

Natural Language Processing (NLTP)

Complete Study Material

Comprehensive Study Guide

Contents

1 Module 1: Foundations of NLP

1.1 1 Introduction to Natural language processing (1)

Slide 1:

Natural Language Text Processing

Slide 2:

Natural Language

Languages that are spoken naturally in human being

Slide 3:

Languages

Human to Human

Machine to machine

Slide 4:

Perspective on NLP: Areas of AI and their inter-dependencies.

Slide 5:

Layer 1

Search

Search algorithms try to find out the best possible strategy, the optimal strategy for computer.

Logic

Logic is a vehicle for reasoning and inferencing. In logic we are concerned with several constructs like if x is true then y is true.

C. Knowledge Representation

knowledge must be extracted and embedded in the Machine.

Slide 6:

Layer 2

Machine Learning

NLP is using lots of Statistical Techniques. Statistical Techniques are Machine Learning techniques; they make use of the knowledge content in the data.

Planning

Planning in AI is the process of producing a series of actions or procedures to accomplish a particular goal. Planning in AI is often used for tasks that require reasoning, problem-solving, and decision-making, such as robotics, games, or scheduling

Slide 7:

Perspective on NLP: Areas of AI and their inter-dependencies.

NLP-concerned with computer being able to process human language like Hindi, Marathi, Gujarathi, French, English, and understand .

Computer Vision-NLP is followed by Computer Vision where machine processes seen and understand how to operate in seen.

Robotics-there is an embedded software inside robots that asking it to perform certain actions like navigating etc.

Slide 8:

Perspective on NLP: Areas of AI and their inter-dependencies.

Expert System-Expert System is concerned with ,the expert level performance of the software on a specific task.

For example –The task could be diagnosis of diseases and curing it.

A doctor is known to operate with number of rules ,a very large no of rules obtained by years of education and practice on patience .

So, the expert system is concerned with emulating this behavior of the expert.

Slide 9:

2 nd layer : feeding Layer

Machine Learning and planning feed into several layers in the outer most category.

For example, Natural Language Processing is fed by Machine Learning and Natural Language Processing is also fed by Knowledge Representation.

The reason for this is that in current world, Natural Language Processing is using lots of Statistical Techniques.

Statistical Techniques are Machine Learning techniques; they make use of the knowledge content in the data.

Slide 10:

Introduction to Natural Language Processing

branch of artificial intelligence that helps computers understand, interpret and manipulate human language.

it deals with the interaction between computers and humans using the natural language.

objective is to read, decipher, understand, and make sense of the human languages in a manner that is valuable.

is an interdisciplinary subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language

Slide 11:

Natural language processing

It refers to the branch of computer science—and more specifically, the branch of artificial intelligence or AI

concerned with giving computers the ability to understand text and spoken words in much the same way human beings can.

is an interdisciplinary subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language

Slide 12:

INPUT Text (Document

/Paragraph

/Sentence

/word

OUTPUT

(Text/ Rating

/Graph

/Audio)

NLP System

Slide 13:

Text (Document

/Paragraph

/Sentence

/word

Speech

Processing

Audio

OUTPUT

(Text/ Rating

/Graph)

NLP System

Slide 14:

Text (Document

/Paragraph

/Sentence

/word

Speech
Processing
OCR
Processing
Audio
Image
OUTPUT
(Text/ Rating
/Graph)
NLP System

Slide 15:

Stages of Natural language Processing

To converse with human a program must understand the syntax(grammar), semantics(word meaning) and morphology (word level analysis), and pragmatics(conversation).

There are certain phases in which Natural language processing is performed .

Slide 16:

Steps in NLP

Morphological Analysis

Syntactic analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

Word-Level Analysis

Sentence-Level Analysis

Sentence-Level Analysis

Sentence-Level Analysis

Sentence-Level Analysis

Slide 17:

Morphological Analysis

studies the structure of words or formation of the words.

How words are built from smaller pieces

Identification ,analysis of root words , affixes (suffixes and prefixes)

Example

Washing – wash+ ing

Browser – Browse + er

Incomplete- In+ Complete

Slide 18:

Morphological Analysis

1. Tokenization

John ate the pizza ! !

2. Stop Word Removal (removing the words that occur commonly across all the documents ,typically, articles and pronouns are generally classified as stop words)

Example

Search Engines

News article categorization Spam Detection

Chatbots Virtual Assistants:

“I want to know the status of my order”→ know status order

Slide 19:

Morphological Analysis

Syntactic analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

3. Stemming

Stemming is a process of reducing words into its base form (Root form/stem form).

Process of converting inflected word to their word stem.

John->John Ate -> eat Pizza->Pizza

Slide 20:

4. Lemmatization

text normalization technique used for Natural Language Processing (NLP).

It can convert any word's inflections to the base root form.

For example:

Playing, Plays, Played ——— Play (Common root form "play")

Slide 21:

Stemming vs Lemmatization

Stemming Real-world Uses:

Search engines

Information retrieval systems

Text mining

Recommendation systems

Lemmatization Real-world Uses:

Chatbots

Machine translation

Medical legal NLP

Sentiment analysis

Slide 22:

Stemming vs Lemmatization

Slide 23:

Morphological Analysis

Syntactic analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

4. N-Gram Language Model

Continuous sequence of N-Items from a given sample text.

1-gram-

John

Ate

the

Pizza

Bigram -

Trigram -

4-Gram -

John Ate the Pizza

John Ate the ?

John

Ate

the

Pizza

Slide 24:

Syntactic analysis

John Ate the Apple

Ate the Apple John

analyzing the grammatical syntax of a sentence to understand its meaning

Slide 25:

Syntactic analysis set of rules needed to ensure a sentence is grammatically correct

John Ate the Apple

Ate the Apple John

Slide 26:

Morphological Analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

She drank Some Milk

She drank Some books

semantics refers to meaning

a computer understands the meaning of a text by analyzing the text as a whole and not just looking at individual words

context in which a word is used is very important

“Does it all sound like a joke to you?”

Example 1

Example 2

Slide 27:

Morphological Analysis

Syntactic analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

Monkeys Eat Banana, when they Wake up.

Who is they here?

-Monkey

Monkeys eat Banana,

when they are ripe.

Who is they here?

-Banana

Resolving the reference as per the context .

Slide 28:

Morphological Analysis

Syntactic analysis

Semantic Analysis

Discourse Analysis

Pragmatic Analysis

Close the Door

-Order

Please Close the Door

- Request ,affirmation

knowledge of the relationship of meaning to the goals and intentions of the speaker.

Slide 29:

Ambiguity in Natural language

Text is ambiguous if there are multiple alternative Ambiguous linguistic structures that can be built for it

situation where a word or a sentence may have more than one meaning.

There are different types of ambiguities

Lexical
Syntactical
Semantic
Discourse Ambiguity
Pragmatic Ambiguity

Slide 30:

Lexical Ambiguity
is the ambiguity of a single word
It can be resolved by parts of speech tagging
Word has more than one meaning/category
E.g.

Book – Noun –Textbook/Novel
Book – Verb - Book ticket/seat
Bank-Noun –Financial institute
Bank-Noun - River Bank
Bank- Verb- Banking Transaction

Slide 31:

Syntactical Ambiguity(Grammar or rules are ambiguous)
it exists in the presence of two or more possible meanings within the sentence.
Specify the possible arrangements of words in sentence used
Example
I saw the girl with the binocular.
In the above example, did I have the binoculars? Or did the girl have the binoculars?

Slide 32:

Semantic Ambiguity
When a sentence has more than one meaning then it is called as semantic ambiguity
Rahul loves his cat and Dipesh does too.
Whether Dipesh loves his cat or Rahul's cat

Slide 33:

Anaphoric Ambiguity
Occurs due to use of Anaphoric entities in discourse.
Anaphora-When same beginning of sentence is repeated in the sentence several times , we makes the pronoun instead of noun .
For example, My mother liked the house very much , but she couldn't purchase it

Example: -

- Monkeys Eat Banana, when they Wake up. -Who is they here? Monkey
 - Monkeys eat Banana, when they are ripe.-Who is they here?-Banana
- Identify When, Where, by whom occurrence was said

Slide 34:

Pragmatic Ambiguity
Understanding speaker's intention.
Eg. You are Late

Slide 35:

Challenges in NLP
Elongated words
I am soooooo sorry.
I t was tooooo yummy.
Shortcuts
Pls -
BTW -
Wat -

K -

Please

By the way What

OK

Slide 36:

Challenges in NLP

Emojis

Slide 37:

Challenges in NLP

Mix Use of Languages

I liked that movie. Salman khan ka acting was Lajabab. Too good. .

Slide 38:

Challenges in NLP

Ellipsis

Peter worked hard and passed exam, Kevin too.

Interpretation

Kevin worked hard

Kevin passed exam

Kevin worked hard also he passed exam

Slide 39:

Challenges in NLP

Punctuational Ambiguity

Women, without her man, is nothing.

Women! without her, man is nothing.

Slide 40:

Applications of NLP

1. Question Answering

Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language.

Slide 41:

Applications of NLP

2. Spam Detection

Spam detection is used to detect unwanted e-mails getting to a user's inbox.

Slide 42:

3. Sentiment Analysis

A combination of NLP (Natural Language Processing) and statistics by assigning the values to the text (positive, negative, or natural), identify the mood of the context (happy, sad, angry, etc.)

Slide 43:

4. Machine Translation (Example: Google Translator)

Machine translation is used to translate text or speech from one natural language to another natural language.

Slide 44:

5. Spelling correction

Microsoft Corporation provides word processor software like MS-word, PowerPoint for the spelling correction.

Chek's the spelling while typing.

Example Wrong Speling will be underlined with the red color as done in this sentence

Slide 45:

6. Chatbot

Implementing the Chatbot is one of the important applications of NLP. It is used by many companies to provide the customer's chat services.

Slide 46:

Practice Questions

Identify the Type Of Ambiguity for Given Sentence

Identify The Type of Preprocessing with respect to given scenario-based corpus

Steps in NLP

Challenges in NLP

Ambiguities in NLP

Slide 47:

Lexical Ambiguity

Occurs when a word has multiple meanings.

Example: "The fisherman went to the bank."

"Bank" could mean a financial institution or the riverbank (edge of a river).

This ambiguity arises due to polysemy (multiple meanings of a word).

Syntactic (Structural) Ambiguity

Occurs when a sentence can have multiple grammatical interpretations due to different ways of parsing it.

Example: "The manager told the employee that he was fired."

Who is fired? The manager or the employee?

The sentence structure allows both interpretations, leading to confusion.

Semantic Ambiguity

Occurs when the meaning of a sentence is unclear due to vague or multiple possible meanings.

Example: "Visiting relatives can be annoying."

Does it mean relatives who visit are annoying or the act of visiting relatives is annoying?

Pragmatic Ambiguity

Occurs when the meaning depends on context, tone, or external knowledge.

Example: "Can you pass the salt?"

Literally, this asks if the person has the ability to pass salt.

But in a real-world context, it is a request to pass the salt.

Referential Ambiguity

Occurs when pronouns (he, she, it, they) refer to multiple possible antecedents.

Example: "John met Mike at the café, and he ordered a coffee."

Who ordered the coffee—John or Mike?

Slide 48:

Practice Questions

Q. Identify the Type Of Ambiguity for Given Sentence

Consider the following sentences and identify type of ambiguity

The manager told the employee that he was fired.

The fisherman went to the bank.

Solution:

"The manager told the employee that he was fired."

Type of Ambiguity: Syntactic Ambiguity / Referential Ambiguity

Unclear whether "he" refers to the manager or the employee.

"The fisherman went to the bank."

Type of Ambiguity: Lexical Ambiguity

The word "bank" can mean a financial institution or a riverbank.

Slide 49:

Consider the following sentences and identify type of ambiguity

He saw the seal at the zoo.

John told Mike that he won the award.

Slide 50:

Identify The Type of Preprocessing with respect to given scenario-based corpus

Q1 .A search engine ignores common words such as “the”, “is”, “and”, “of” while indexing web pages to improve search speed and relevance. Which text preprocessing technique is required?

Q2. A customer review sentiment analysis system removes symbols like !, ?, #, and . before extracting features from text. Which text preprocessing technique is required?

Q3. A medical chatbot converts words such as “diagnoses”, “diagnosed”, and “diagnosing” into their base dictionary form “diagnose” to preserve meaning. Which text preprocessing technique is required?

Q4. A news classification system treats “COVID”, “Covid”, and “covid” as the same term during model training. Which text preprocessing technique is required?

Q5. A document retrieval system reduces words like “playing”, “played”, and “player” to “play” without checking grammatical correctness. Which text preprocessing technique is required?

Slide 51:

Q6

A social media analytics platform removes URLs, HTML tags, emojis, and special characters from tweets before analysis. Which text preprocessing technique is required?

Q7

A plagiarism detection system breaks paragraphs into individual words before computing similarity measures. Which text preprocessing technique is required?

Q8

An e-commerce search engine automatically corrects misspelled queries such as “samsng” to “samsung”. Which text preprocessing technique is required?

Q9

A financial report analysis system replaces values like “10,000”, “2023”, and “5

Q10

A sentiment analysis system captures expressions like “not satisfied” and “very good” as meaningful units instead of individual words. Which text preprocessing technique is required?

Slide 52:

Answers

1.2 2 Language Model_KD**Slide 1:**

Language Modeling

Introduction to N-grams

Slide 2:

Why?

Machine Translation:

$P(\text{high winds tonite}) > P(\text{large winds tonite})$

Spell Correction

The office is about fifteen minuets from my house

$P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$

Slide 3:

Why ?

Speech Recognition

$P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

+ Summarization, question-answering, etc., etc.!!

OCR(optical character recognition) Hand Recognition

In, OCR LM can be used to predict a word that is not easily readable in the given image .

Slide 4:

Why ?

Suggestions while typing

We get suggestion on our cell phone while messaging

Context Sensitive Spelling Correction

Eg. "college is closed on Saturday and Sunday

Spelling error : college has been typed as collage

College : Education institution

Collage : Artwork

Suppose both words college and collage are present in corpus .

Spelling Mistake detection and correction will not work as both words are present in corpus.

Need Spelling Correction based on local context.

$P(\text{College is closed on Saturday and Sunday}) > P(\text{Collage is closed on Saturday and Sunday})$

Slide 5:

Probabilistic Language Modeling

compute the probability of a sentence or sequence of words occurring together:

$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$ Joint Probability

probability of an upcoming word:

$P(w_5 | w_1, w_2, w_3, w_4)$ Conditional Probability

Slide 6:

How to estimate these probabilities

Could we just count and divide?

No! Too many possible sentences!

We'll never see enough data for estimating these

Slide 7:

Drawback of estimating probabilities directly from counts

Large corpora (like the web) allow probability estimation using word counts.

This approach often fails because even huge datasets are not large enough.

Language is productive and constantly creates new sentences.

Many valid sentences or phrases never appear in the data.

Unseen sentences or phrases get zero counts, leading to poor probability estimates.

Solution is join probability

Slide 8:

Joint Probability $P(W)$

How to compute this joint probability:

$P(\text{its, water, is, so, transparent, that})$

Intuition: let's rely on the Chain Rule of Probability

Slide 9:

Reminder: The Chain Rule

Recall the definition of conditional probabilities

$p(B|A) = P(A,B)/P(A)$ **Rewriting:** $P(A,B) = P(A)P(B|A)$

More variables:

$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$

The Chain Rule in General

$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$

Slide 10:

Markov Assumption

In other words, we approximate each component in the product

Slide 11:

Markov Assumption

Simplifying assumption:

Or maybe

Andrei Markov

Slide 12:

N-gram model

A model that applies markov assumption to text

For Example, bigram

The bigram model, approximates the conditional probability of the preceding word $P(w_n|w_{n-1})$.

Slide 13:

Example

instead of computing the probability

$P(\text{the}|\text{Walden Pond's water is so transparent that})$

we approximate it with the probability

$P(\text{the}|\text{that})$

Slide 14:

How do we estimate these bigram or n-gram probabilities?

The Maximum Likelihood Estimate

Slide 17:

An example

Corpus:

$\langle s \rangle$ I am Sam $\langle /s \rangle$

$\langle s \rangle$ Sam I am $\langle /s \rangle$

$\langle s \rangle$ I do not like green eggs and ham $\langle /s \rangle$

Slide 18:**Exercise 1:**

Estimating Bi-gram probabilities

What is the most probable next word predicted by the model for the following word sequence?

$\langle S \rangle$ Do ?

$\langle S \rangle$ I like Henry ?

Slide 19:

1) $\langle S \rangle$ Do ?

Next word prediction probability $W = \text{do}$

I is most Probable

Slide 20:

$\langle S \rangle$ I like Henry ?

Next word prediction probability $W = \text{Henry}$

$\langle /S \rangle$ is more probable

Slide 21:

Questions

Slide 22:

Solution

Slide 23:

Problems with Using Raw Probabilities

1. Numerical Underflow

2. Computational Inefficiency

Slide 24:

Practical Issues

We do everything in log space

Avoid underflow

(also adding is faster than multiplying)

Slide 25:

Probabilities 1 shrink when multiplied repeatedly

Multiplying many n-gram probabilities causes numerical underflow

Using log probabilities avoids very small numbers

Multiplication in probability space becomes addition in log space

Addition is faster and more stable than multiplication

Convert back to probabilities only when final results are needed

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Log Probabilities

Slide 26:

Example

Slide 27:

Example

Slide 28:

Thank You

Slide 29:

Practice Question

Extract bigrams from the given corpus:

Corpus C:

there is a big garden

children play in a garden

they play inside beautiful garden

Slide 30:

Practice Question 1

Extract bigrams from the given corpus:

Corpus C:

there is a big garden

children play in a garden

they play inside beautiful garden

Preprocess

Adding sentence boundaries <s> and </s>:

<s> there is a big garden </s>

<s> children play in a garden </s>

<s> they play inside beautiful garden </s>

Slide 31:

Practice Question

extract bigrams from the given corpus:

Corpus C:

there is a big garden

children play in a garden

they play inside beautiful garden

Adding sentence boundaries <s> and </s>:

<s> there is a big garden </s>

<s> children play in a garden </s>

<s> they play inside beautiful garden </s>

Bigram Frequency Table:

Slide 32:

Practice Question 2

Compute $P(\text{in}|\text{play})$ using a Bigram Model

Corpus C:

there is a big garden

children play in a garden
they play inside beautiful garden

Slide 33:

P(in|play)

Practice Question 2

Compute P(in|play) using a Bigram Model

Corpus C:

there is a big garden
children play in a garden
they play inside beautiful garden

Slide 34:

Practice Question-3

Compute P(they play in a big garden) using a Bigram Model

Corpus C:

there is a big garden
children play in a garden
they play inside beautiful garden

Adding sentence boundaries <s> and </s>:

<s> there is a big garden </s>

<s> children play in a garden </s>

<s> they play inside beautiful garden </s>

Unigram Frequency Table:

Bigram Frequency Table:

Slide 35:

Practice Question

Compute P(they play in a big garden) using a Bigram Model

Corpus C:

there is a big garden
children play in a garden
they play inside beautiful garden

Unigram Frequency Table:

Bigram Frequency Table:

Slide 37:

Calculations are made easy by $a.b.c = \exp(\log(a.b.c))$

$abc = \exp(\log a + \log b + \log c)$

Slide 38:

Practice Question 6

"there is a big garden"

Trigrams: (there, is, a), (is, a, big), (a, big, garden)

"children play in a garden"

Trigrams: (children, play, in), (play, in, a), (in, a, garden)

"they play inside beautiful garden"

Trigrams: (they, play, inside), (play, inside, beautiful), (inside, beautiful, garden)

Total Trigrams

Total unique trigrams = 9

Total trigrams including repetitions = 9 (No duplicates here)

Trigram Counts:

Bigram Counts:

1.3 3 Evaluating Language Models

Slide 1:

Evaluating Language Models

Natural Language Processing

1

Slide 2:

Natural Language Processing

2

Does our language model prefer good sentences to bad ones?

Assign higher probability to “real” or “frequently observed” sentences than “ungrammatical” or “rarely observed” sentences?

We train parameters of our model on a training set.

We test the model’s performance on data we haven’t seen.

A test set is an unseen dataset that is different from our training set, totally unused.

An evaluation metric tells us how well our model does on the test set.

Evaluating Language Models

Slide 3:

Natural Language Processing

3

Evaluating Language Models

Extrinsic Evaluation(Application Based)

Extrinsic Evaluation of a N-gram language model is to use it in an application and measure how much the application improves.

To compare two language models A and B:

Use each of language model in a task such as spelling corrector, MT(Machine Translation) system.

Get an accuracy for A and for B

How many misspelled words corrected properly

How many words translated correctly

Compare accuracy for A and B

The model produces the better accuracy is the better model.

Extrinsic evaluation can be time-consuming.

Slide 4:

Natural Language Processing

4

Evaluating Language Models

Intrinsic Evaluation

An intrinsic evaluation metric is one that measures the quality of a model independent of any application.

When a corpus of text is given and to compare two different n-gram models,

Divide the data into training and test sets,

Train the parameters of both models on the training set, and

Compare how well the two trained models fit the test set.

Whichever model assigns a higher probability to the test set

In practice, probability-based metric called perplexity is used instead of raw probability as our metric for evaluating language models.

Slide 5:

Evaluating Language Models

Perplexity

The best language model is one that best predicts an unseen test set

– Gives the highest $P(\text{testset})$

The perplexity of a language model on a test set is the inverse probability of the test set, normalized by the number of words.

Minimizing perplexity is the same as maximizing probability

The perplexity PP for a test set $W=w_1w_2\dots w_n$ is

PP(W) by chain rule

The perplexity PP for bigrams: PP(W)

Natural Language Processing

5

Slide 6:

Evaluating Language Models

Perplexity as branching factor

Perplexity can be seen as the weighted average branching factor of a language.

– The branching factor of a language is the number of possible next words that can follow any word.

Let's suppose a sentence consisting of random digits

What is the perplexity of this sentence according to a model that assign $P=1/10$ to each digit?

6

Slide 7:

Example

$\langle s \rangle$ I am a human $\langle /s \rangle$

$\langle s \rangle$ I am not a stone $\langle /s \rangle$

$\langle /s \rangle$ I I live in Mumbai $\langle /s \rangle$

Using a bigram language model compute: $P(\text{"I I am not"})$

Slide 8:

Natural Language Processing

8

The n-gram model, like many statistical models, is dependent on the training corpus. the probabilities often encode specific facts about a given training corpus.

N-grams only work well for word prediction if the test corpus looks like the training corpus
In real life, it often doesn't We need to train robust models that generalize!

One kind of generalization: Getting rid of Zeros! Things that don't ever occur in the training set, but occur in the test set
Zeros: things that don't ever occur in the training set but do occur in the test set causes problem for two reasons. First, underestimating the probability of all sorts of words that might occur, Second, if probability of any word in test set is 0, entire probability of test set is 0.

Generalization and Zeros

Slide 9:

Natural Language Processing

9

N-gram models depend on the training corpus

Perform well only if test data matches training data

Zero probabilities occur for unseen words

One zero makes the entire test probability zero

Generalization and Zeros

Slide 10:

Natural Language Processing

10

We have to deal with words we haven't seen before, which we call unknown words.

We can model these potential unknown words in the test set by adding a pseudo-word called $\langle \text{UNK} \rangle$ into our training set too.

One way to handle unknown words is:

Replace words in the training data by <UNK> based on their frequency.

For example we can replace by <UNK> all words that occur fewer than n times in the training set, where n is some small number, or

Equivalently select a vocabulary size V in advance (say 50,000) and choose the top V words by frequency and replace the rest by <UNK>.

Proceed to train the language model as before, treating <UNK> like a regular word.

Unknown Words

Slide 11:

11

Real data contains unknown words not seen in training

Unknown words are modeled using a pseudo-word <UNK>

<UNK> is added to the training vocabulary

Rare words in training are replaced with <UNK>

Or fix vocabulary size V and keep top V frequent words

Replace all remaining words with <UNK>

Train the language model treating <UNK> as a normal word

Unknown Words

Slide 12:

<s>I am Henry</s>

<s> I like college </s>

<s>Do Henry like college </s>

<s>Henry I am</s>

<s>Do I like Henry</s>

<s>Do I like college </s>

<s>I do like Henry </s>

<s> like college</s>

<s>Do I like Henry</s>

Consider the Following Corpus and Identify the most Probable statement from the given sentence

Slide 13:

<s> like college </s>

$= P(\text{like} \mid \text{<s>}) \times P(\text{college} \mid \text{like}) \times P(\text{</s>} \mid \text{college})$

$= 0/7 \times 3/5 \times 3/3 = 0$

<s> Do I like Henry </s>

$= P(\text{do} \mid \text{<s>}) \times P(\text{I} \mid \text{do}) \times P(\text{like} \mid \text{I}) \times P(\text{Henry} \mid \text{like}) \times P(\text{</s>} \mid \text{Henry})$

$= 3/7 \times 2/4 \times 3/6 \times 2/5 \times 3/5$

$= 9/350 = 0.0257$

Slide 14:

Natural Language Processing

14

We lower the probability of common word pairs a little and use that leftover probability to give rare or unseen word pairs a small non-zero probability.

This modification is called smoothing (or discounting).

With smoothing:

Unseen words get a small chance

The model becomes more realistic and useful

Smoothing

Slide 15:

15

There are many ways to do smoothing, and some of them are:

Add-1 smoothing (Laplace Smoothing)

Add-k smoothing,
Smoothing

Slide 16:

Natural Language Processing
16

The simplest way to do smoothing is to add one to all the counts, before we normalize them into probabilities.

– All the counts that used to be zero will now have a count of 1, the counts of 1 will be 2, and so on.

This algorithm is called Laplace smoothing (Add-1 Smoothing).

We pretend that we saw each word one more time than we did, and we just add one to all the counts!

Laplace Smoothing

Slide 17:

17

The unsmoothed maximum likelihood estimate of the unigram probability of the word w_i is its count c_i normalized by the total number of word tokens N :

$$P(w_i) = c_i / N$$

Laplace smoothing adds one to each count. Since there are V words in the vocabulary and each one was incremented, we also need to adjust the denominator to take into account the extra V observations.

$$P_{\text{Laplace}}(w_i) = (c_i + 1) / (N + V)$$

Laplace Smoothing (Add-1 Smoothing)

Where

N = total word tokens in corpus,

V = number of unique words in the vocabulary.

Slide 18:

The normal bigram probabilities are computed by normalizing each bigram counts by the unigram count:

Add-one smoothed bigram probabilities:

Laplace Smoothing for Bigrams

Natural Language Processing

18

Slide 19:

Unique words are: <S>, </S>, I, Henry, do, like, am, college

Total unique words: 8

But we exclude </S>

Total unique words: 7

Give the following bi-gram probabilities estimated by Laplace model.

Slide 20:

1. <S> like college </S>

$$= P(\text{like} \mid \text{<S>}) \times P(\text{college} \mid \text{like}) \times P(\text{</S>} \mid \text{college})$$

$$= (0 + 1) / (7 + 7) \times (3 + 1) / (5 + 7) \times (3 + 1) / (3 + 7)$$

$$= 1/14 \times 4/12 \times 4/10$$

$$= 0.0095$$

2. <S> Do I like Henry </S>

$$= P(\text{do} \mid \text{<S>}) \times P(\text{I} \mid \text{do}) \times P(\text{like} \mid \text{I}) \times P(\text{Henry} \mid \text{like}) \times P(\text{</S>} \mid \text{Henry})$$

$$= (3 + 1) / (7 + 7) \times (2 + 1) / (4 + 7) \times (3 + 1) / (6 + 7) \times (2 + 1) / (5 + 7) \times (3 + 1) / (5 + 7)$$

$$= 4/14 \times 3/11 \times 4/13 \times 3/12 \times 4/12$$

$$= 0.0020$$

Slide 21:

Now consider a Bi-gram Language model (without smoothing). Which of the following sentences has the highest probability estimate?

a. S1: $\langle s \rangle$ michael played at the playground $\langle /s \rangle$ b. S2: $\langle s \rangle$ bob went to the school $\langle /s \rangle$ c. S3: $\langle s \rangle$ the school was huge $\langle /s \rangle$ d. S4: $\langle s \rangle$ zack went to the playground $\langle /s \rangle$

$$P(S1) = 0P(S2) = 0P(S3) = \frac{2}{5} \times \frac{3}{5} \times \frac{2}{5} \times \frac{1}{2} \times 1 = \frac{2}{25} P(S4) = 0$$

Slide 22:

Laplace-smoothed Bigrams

Corpus: Berkeley Restaurant Project Sentences

Natural Language Processing

22

Slide 23:

Laplace-smoothed Bigrams

Corpus: Berkeley Restaurant Project Sentences: Adjusted counts

Natural Language Processing

23

Slide 24:

Laplace-smoothed Bigrams

Corpus: Berkeley Restaurant Project Sentences: Adjusted counts

Natural Language Processing

24

Slide 25:

Problem with Add-1 Smoothing

We fix zeros by pretending every possible bigram happened once:

Where V = vocabulary size.

But here's the issue:

Adding 1 to every possible word is huge when V is big.

From the slide:

$C(\text{want} \rightarrow \text{to})$ changed $608 \rightarrow 238$ after smoothing

$P(\text{to} \mid \text{want})$ dropped $0.66 \rightarrow 0.26$

So we:

Steal probability from common events

Give too much to rare/unseen events

This is called over-smoothing.

Slide 26:

The sharp change in counts and probabilities occurs because too much probability mass is moved to all the zeros.

One alternative to add-one smoothing is to move a bit less of the probability mass from the seen to the unseen events.

Instead of adding 1 to each count, we add a fractional count k (.5? .05? .01?).

This algorithm is called add- k smoothing.

Add- k Smoothing

26

What this does:

Slide 27:

Numerical

Consider the following Corpus

$\langle s \rangle$ I love natural language processing $\langle /s \rangle$

$\langle s \rangle$ language models learn patterns $\langle /s \rangle$

$\langle s \rangle$ I love machine learning $\langle /s \rangle$

$\langle s \rangle$ natural language models are powerful $\langle /s \rangle$

<s> machine learning uses data </s>

1.4 4 EditDistance

Slide 1:

Minimum Edit Distance

Definition of Minimum Edit Distance

Slide 2:

How similar are two strings?

Spell correction

The user typed “graffe”

Which is closest?

graf

graft

grail

giraffe

Computational Biology

Align two sequences of nucleotides

Resulting alignment:

Also for Machine Translation, Information Extraction, Speech Recognition

AGGCTATCACCTGACCTCCAGGCCGATGCCC

TAGCTATCACGACCGCGGTCGATTTGCCCGAC

-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC—

TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC

Slide 3:

Edit Distance

Edit distance gives us a way to quantify intuitions about string similarity.

The minimum edit distance between two strings

Is the minimum number of editing operations

Insertion

Deletion

Substitution

Needed to transform one into the other

Slide 4:

Minimum Edit Distance

Two strings and their alignment:

Slide 5:

Minimum Edit Distance

If each operation has cost of 1

Distance between these is 5

If substitutions cost 2 (Levenshtein)

Distance between them is 8

Slide 6:

Alignment in Computational Biology

Given a sequence of bases

An alignment:

Given two sequences, align each letter to a letter or gap

-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC—

TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC

AGGCTATCACCTGACCTCCAGGCCGATGCCC

TAGCTATCACGACCGCGGTCGATTTGCCCGAC

Slide 7:

Alignment in Computational Biology

Given a sequence of bases

An alignment:

Given two sequences, align each letter to a letter or gap

-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC—
TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC

Slide 8:

Other uses of Edit Distance in NLP

Evaluating Machine Translation and speech recognition

R Spokesman confirms senior government adviser was appointed

H Spokesman said the senior adviser was appointed

S I D I

Named Entity Extraction and Entity Coreference

IBM Inc. announced today

IBM profits

Stanford Professor Jennifer Eberhardt announced yesterday

for Professor Eberhardt...

Slide 9:

How to find the Min Edit Distance?

Searching for a path (sequence of edits) from the start string to the final string:

Initial state: the word we're transforming

Operators: insert, delete, substitute

Goal state: the word we're trying to get to

Path cost: what we want to minimize: the number of edits

Slide 10:

Minimum Edit as Search

But the space of all edit sequences is huge!

We can't afford to navigate naïvely

Lots of distinct paths wind up at the same state.

We don't have to keep track of all of them

Just the shortest path to each of those revisited states.

Slide 11:

Defining Min Edit Distance

For two strings

X of length n

Y of length m

We define $D(i,j)$

the edit distance between $X[1..i]$ and $Y[1..j]$

i.e., the first i characters of X and the first j characters of Y

The edit distance between X and Y is thus $D(n,m)$

Slide 12:

Minimum Edit Distance

Definition of Minimum Edit Distance

Slide 13:

Dynamic Programming for Minimum Edit Distance

Dynamic programming: A tabular computation of $D(n,m)$

Solving problems by combining solutions to subproblems.

Bottom-up

We compute $D(i,j)$ for small i,j

And compute larger $D(i,j)$ based on previously computed smaller values
i.e., compute $D(i,j)$ for all i ($0 < i < n$) and j ($0 < j < m$)

Slide 14:

Defining Min Edit Distance (Levenshtein)

Initialization

$$D(i,0) = i$$

$$D(0,j) = j$$

Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i-1,j) + 1$$

$$D(i,j) = \min D(i,j-1) + 1$$

$$D(i-1,j-1) + 2; \text{ if } X(i) \neq Y(j)$$

$$0; \text{ if } X(i) = Y(j)$$

Termination:

$D(N,M)$ is distance

Slide 15:

The Edit Distance Table

j

i

Slide 16:

The Edit Distance Table

Slide 17:

Edit Distance

Slide 18:

The Edit Distance Table

Slide 19:

Computing alignments

Edit distance isn't sufficient

We often need to align each character of the two strings to each other

We do this by keeping a "backtrace"

Every time we enter a cell, remember where we came from

When we reach the end,

Trace back the path from the upper right corner to read off the alignment

Slide 20:

Edit Distance

Slide 21:

Exercise

Slide 22:

Solution

Slide 23:

MinEdit with Backtrace

Slide 24:

Adding Backtrace to Minimum Edit Distance

Base conditions: Termination:

$$D(i,0) = i \quad D(0,j) = j \quad D(N,M) \text{ is distance}$$

Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i-1,j) + 1$$

$$D(i,j) = \min D(i,j-1) + 1$$

$D(i-1,j-1) + 2$; if $X(i) \neq Y(j)$

0; if $X(i) = Y(j)$

LEFT

ptr(i,j)= DOWN

DIAG

insertion

deletion

substitution

insertion

deletion

substitution

Slide 25:

The Distance Matrix

Slide adapted from Serafim Batzoglou with Permission

y0 yM

x0 xN

Every non-decreasing path

from (0,0) to (M, N)

corresponds to

an alignment

of the two sequences

An optimal alignment is composed of optimal subalignments

Slide 26:

Result of Backtrace

Two strings and their alignment:

Slide 27:

Minimum Edit Distance

Weighted Minimum Edit Distance

Slide 28:

Weighted Edit Distance

Why would we add weights to the computation?

Spell Correction: some letters are more likely to be mistyped than others

Biology: certain kinds of deletions or insertions are more likely than others

Slide 29:

Confusion matrix for spelling errors

Slide 31:

Weighted Min Edit Distance

Initialization:

$D(0,0) = 0$

$D(i,0) = D(i-1,0) + \text{del}[x(i)]$; $1 < i \leq N$

$D(0,j) = D(0,j-1) + \text{ins}[y(j)]$; $1 < j \leq M$

Recurrence Relation:

$D(i-1,j) + \text{del}[x(i)]$

$D(i,j) = \min D(i,j-1) + \text{ins}[y(j)]$

$D(i-1,j-1) + \text{sub}[x(i),y(j)]$

Termination:

$D(N,M)$ is distance

Slide 32:

Question

Find the minimum edit distance in transforming the word DOG to COW using Levenshtein distance, ie., insertion = deletion = 1 and substitution = 2.

Slide 33:

Solution

Slide 34:

Example

1.5 Chapter 1 Ngram

Page 1:

Speech and Language Processing. Daniel Jurafsky James H. Martin. Copyright © 2026. All rights reserved. Draft of January 6, 2026. CHAPTER 3 N-gram Language Models “You are uniformly charming!” cried he, with a smile of associating and now and then I bowed and they perceived a chaise and four to wish for. Random sentence generated from a Jane Austen trigram model Predicting is difficult—especially about the future, as the old quip goes. But how about predicting something that at seems much easier, like the next word someone is going to say? W

Page 2:

2 CHAPTER 3 • N-GRAM LANGUAGE MODELS language model. An n-gram is a sequence of n words: a 2-gram (which we’ll call bigram) is a two-word sequence of words like The water, or water of, and a 3-gram (a trigram) is a three-word sequence of words like The water of, or water of Walden. But we also (in a bit of terminological ambiguity) use the word ‘n-gram’ to mean a probabilistic model that then-1 previous words, and thereby also to assign probabilities to entire sequences. In later chapters we will intr

Page 3:

3.1 • N-GRAMS 3 bility: $P(X_1 \dots X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1 X_2) \dots P(X_n|X_1 \dots X_{n-1})$ (cid:89) $= P(X_k|X_1:k-1)$ (3.3) $k=1$ Applying the chain rule to words, we get $P(w) = P(w)$ $P(w|w)P(w|w) \dots P(w|w_1:n-1)$ (cid:89) $= P(w_k|w_1:k-1)$ (3.4) $k=1$ The chain rule shows the link between computing the joint probability of a sequence and computing the conditional probability of a word given previous words. Equation 3.4 suggests that we could estimate the joint proba- wo

Page 4:

4 CHAPTER 3 • N-GRAM LANGUAGE MODELS size, so $N=2$ means bigrams and $N=3$ means trigrams. Then we approximate the probability of a word given its entire context as follows: $P(w_n|w_1:n-1)P(w_n|w_1:n-1)$ (3.8) Given the bigram assumption for the probability of an individual word, we can compute the probability of a complete word sequence by substituting Eq. 3.7 into Eq. 3.4: $P(w_1:n)$ (cid:89) $P(w_1)$ $P(w_2|w_1) \dots P(w_n|w_1:n-1)$ (3.9) $k=1$ 3.1.2 How to estimate probabilities How do we estimate these bigram or n-gram probabilities? An intuitive way to maximum

Page 5:

3.1 • N-GRAMS 5 Equation 3.12 (like Eq. 3.11) estimates the n-gram probability by dividing the observed frequency of a particular sequence by the observed frequency of a prefix. This ratio is called a relative frequency. We said above that this use of relative frequency frequencies a way to estimate probabilities is an example of maximum likelihood estimation or MLE. In MLE, the resulting parameter is the maximum likelihood estimate of the true parameter. For example, suppose the word Chinese occurs 400

Page 6:

6 CHAPTER 3 • N-GRAM LANGUAGE MODELS i want to eat chinese food lunch spend i 0.002 0.33 0 0.0036 0 0 0 0.00079 want 0.0022 0 0.66 0.0011 0.0065 0.0065 0.0054 0.0011 to 0.00083 0 0.0017 0.28 0.00083 0 0.0025 0.087 eat 0 0 0.0027 0 0.021 0.0027 0.056 0 chinese 0.0063 0 0 0 0.52 0.0063 0 food 0.014 0 0.014 0 0.00092 0.0037 0 0 lunch 0.0059 0 0 0 0.0029 0 0 spend 0.0036 0 0.0036 0 0 0 0 0 Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probability

Page 7:

3.2 • EVALUATING LANGUAGE MODELS: TRAINING AND TEST SETS 7 Longer context Although for pedagogical purposes we have only described bi- trigram models, when there is sufficient training

4-gram condition on the previous two words, or 4-gram or 5-gram models. For these larger 5-gram n-grams, we'll need to assume extra context to the left and right of the sentence end. For example, to compute trigram probability we use two pseudo-words for the first trigram (i.e., $P(I|<s><s>)$). Some

Page 8:

8 CHAPTER 3 • N-GRAM LANGUAGE MODELS The training set is the data we use to learn the parameters of our model; for simple n-gram language models it's the corpus from which we get the counts that we normalize into the probabilities of the n-gram language model. The test set is a different, held-out set of data, not overlapping with the training set, that we use to evaluate the model. We need a separate test set to give us an unbiased estimate of how well the model we trained can generalize when we apply it to some new unknown dataset. A

Page 9:

3.3 • EVALUATING LANGUAGE MODELS: PERPLEXITY 9 to measure a statistically significant difference between two potential models. It's important that the dev set be drawn from the same kind of text as the test set, since its goal is to measure how well we would do on the test set. 3.3 Evaluating Language Models: Perplexity We said above that we evaluate language models based on which one assigns a higher probability to the test set. A better model is better at predicting upcoming words, and so it will be less surprised by (i.e., assign

Page 10:

10 CHAPTER 3 • N-GRAM LANGUAGE MODELS The perplexity of W computed with a bigram language model is still a geometric mean, but now of the inverse of the bigram probabilities: (cid:118) (cid:117) N (cid:117) (cid:89) 1 perplexity(W) = (cid:116) N (3.17) $P(w|w_{1:i}) = \prod_{i=1}^i P(w_i|w_{1:i-1})$ What we generally use for word sequence in Eq. 3.15 or Eq. 3.17 is the entire sequence of words in some test set. Since this sequence will cross many sentence boundaries, if our vocabulary includes a between-sentence token $<EOS>$ or separate begin- and end

Page 11:

3.4 • SAMPLING SENTENCES FROM A LANGUAGE MODEL 11 language that is deterministic (no probabilities), any word can follow any word, and whose vocabulary consists of only three colors: $L = \text{red, blue, green}$ (3.18) The branching factor of this language is 3. Now let's make a probabilistic version of the same LMM where each word follows each other with equal probability 1 (it was trained on a training set with 3 equal counts for the 3 colors), and data = "red red red red blue". Let's first convince ourselves that if we compute the perplexity of this artificial

Page 12:

12 CHAPTER 3 • N-GRAM LANGUAGE MODELS polyphonic however $p = 0.0000018$ the of a to in ... ($p = 0.0003$) 0.06 0.03 0.02 0.02 0.0206 .09 .11 .13 .15 .66 .99 0 1 Figure 3.3 A visualization of the sampling distribution for sampling sentences by repeatedly sampling unigrams. The blue bar represents the relative frequency of each word (we've ordered them from most frequent to least frequent, but the choice of order is arbitrary). The number line shows the cumulative probabilities. If we choose a random number between 0 and 1, it will fall in an interval correspond

Page 13:

3.5 • GENERALIZING VS. OVERFITTING THE TRAINING SET 13 –To him swallowed confess hear both. Which. Of save on trail for are ay device and 1 rote life have gram –Hill helates peaks; or! amore to legless first you enter –Why dost stand forth thy canopy, forsooth; heist his palpable hit the King Henry Live 2 king. Follow. gram –What means, sir. I confess she? then all sorts, heist rim, captain. –Fly, and will rid methese Therefore the sadness of parting, as they say, 3 'tis done. gram –This shall for bid it should be branded, if renown made it

Page 14:

14 CHAPTER 3 • N-GRAM LANGUAGE MODELS (3.22) Bored a fden my phone fin nadie!!! while tweets from English-based languages like Nigerian Pidgin have markedly different vocabulary and n-gram patterns from American English (Jurgens et al., 2017): (3.23) @username Ruawizardorwatgansef: indmornin-utweet, afternoon-u tweet, nytganudeytweet. betagetur IT placement twit twitter Is it possible for the test set before? What happens if the word Jurafsky never occurs in our training set, but pops up in the test set? The answer is

Page 15:

3.6 • SMOOTHING, INTERPOLATION, AND BACKOFF 15 of the word w is its count c normalized by the total number of word tokens N : $i \text{ i c i } P(w) = \frac{i}{N}$ Laplace smoothing merely adds one to each count (hence its alternate name add-one smoothing). Since there are V words in the vocabulary and each one was incremented, we also need to adjust the denominator to take into account the extra V observations. (What happens to our P values if we don't increase the denominator?) $\frac{c+1}{N+V}$ $i \text{ i } P$ Laplace $(w \text{ i}) = (3.24) \frac{N+V}{N+V}$ Now that we have the intuition for the

Page 16:

16 CHAPTER 3 • N-GRAM LANGUAGE MODELS i want to eat chinese food lunch spend
i 0.0015 0.21 0.00025 0.0025 0.00025 0.00025 0.00025 0.00075 want 0.0013 0.00042 0.26 0.00084
0.0029 0.0029 0.0025 0.00084 to 0.00078 0.00026 0.0013 0.18 0.00078 0.00026 0.0018 0.055
eat 0.00046 0.00046 0.0014 0.00046 0.0078 0.0014 0.02 0.00046 chinese 0.0012 0.00062 0.00062
0.00062 0.00062 0.052 0.0012 0.00062 food 0.0063 0.00039 0.0063 0.00039 0.00079 0.002 0.00039
0.00039 lunch 0.0017 0.00056 0.00056 0.00056 0.00056 0.0

Page 17:

3.6 • SMOOTHING, INTERPOLATION, AND BACKOFF 17 language modeling, generating counts with poor variances and often inappropriate discounts (Gale and Church, 1994). 3.6.3 Language Model Interpolation There is an alternative source of knowledge we can draw on to solve the problem of zero frequency n -grams. If we are trying to compute $P(w | w_1 \dots w_n)$ but we have no examples of a particular trigram $w_2 w_1 w_n$, we can instead estimate its $w_2 w_1$ probability by using the bigram probability $P(w_2 | w_1)$. Similarly, if w_1

Page 18:

18 CHAPTER 3 • N-GRAM LANGUAGE MODELS has zero counts, we approximate it by backing off to the $(n-1)$ -gram. We continue backing off until we reach a history that has some counts. For a backoff model to discount give a correct probability distribution, we have to discount the higher-order n -grams to save some probability mass for the lower order n -grams. In practice, instead of discounting, it's common to use a much simpler non-discounted backoff algorithm stupid backoff called stupid backoff (Brant et al., 2007). Stupid backoff gives

Page 19:

3.7 • ADVANCED: PERPLEXITY'S RELATION TO ENTROPY 19 Horse 1 1 Horse 5 1 2 64
Horse 2 1 Horse 6 1 4 64 Horse 3 1 Horse 7 1 8 64 Horse 4 1 Horse 8 1 16 64 The entropy of the random variable X that ranges over horses gives us a lower bound on the number of bits and is $H(X) = -\sum_{i=1}^8 p(i) \log_2 p(i) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{4} \log_2 \frac{1}{4} - \frac{1}{8} \log_2 \frac{1}{8} - \frac{1}{16} \log_2 \frac{1}{16} - \frac{1}{64} \log_2 \frac{1}{64} = 2.585$ bits (3.33) A code that averages 2.585 bits per race can be built with short encodings for more probable horses, and longer encodings for less probable horses.

Page 20:

20 CHAPTER 3 • N-GRAM LANGUAGE MODELS The Shannon-McMillan-Breiman theorem (Algoet and Cover, 1991) states that if the language is regular in certain ways (to be exact, if it is both stationary and ergodic), $H(L) = \lim_{n \rightarrow \infty} \frac{1}{n} \log_2 P(w_1 \dots w_n)$ (3.38) That is, we can't take a single sequence that is long enough instead of summing over all possible sequences. The intuition of the Shannon-McMillan-Breiman theorem is that a long-enough sequence of words will contain in it many other shorter sequences and that each of these shorter sequences

Page 21:

3.8 • SUMMARY 21 lower cross-entropy. (The cross-entropy can never be lower than the true entropy, so a model cannot err by underestimating the true entropy.) We are finally ready to see the relation between perplexity and cross-entropy as we saw it in Eq. 3.40. Cross-entropy is defined in the limit as the length of the observed word sequence goes to infinity. We approximate this cross-entropy by relying on a (sufficiently long) sequence of fixed length. This approximation to the cross-entropy of a model $M = P(w | w')$ on a sequence of words

Page 22:

22 CHAPTER 3 • N-GRAM LANGUAGE MODELS trigram probability that a given letter would be a vowel given the previous one or two letters. Shannon (1948) applied n -gram stochastic approximation to

word sequences. Based on Shannon's work, Markov models were commonly used in engineering, linguistic, and psychology in the 1950s. In a series of extremely influential papers starting with Chomsky (1956) and including Chomsky (1957) and Miller (1958), it was shown that "finite-state Markov process

Page 23:

EXERCISES 23 Exercises 3.1 Write out the equation for trigram probability estimation (modifying Eq. 3.11). Now write out all the non-zero trigram probabilities for the I am Sam corpus on page 4. 3.2 Calculate the probability of the sentence I want Chinese food. Give two probabilities, one using Fig. 3.2 and the 'useful probabilities' on page 6, and another using the add-1 smoothed table in Fig. 3.7. Assume the additional add-1 smoothed probabilities $P(i|j) = 0.40$. 3.3 Which of the two probabilities you computed in the previous

Page 24:

24 CHAPTER 3 • N-GRAM LANGUAGE MODELS 3.9 Run your n-gram program on two different small corpora (one might use email text or newsgroups). Now compare the statistics of the two corpora. What are the differences in the most common unigrams between the two? How about interesting differences in bigrams? 3.10 Add an option to your program to generate random sentences. 3.11 Add an option to your program to compute the perplexity of a test set. 3.12 You are given a training set of 100 numbers that consists of 91 zeros and 9 ones. Compute the

Page 25:

Exercises 25 Algoet, P. H. and T. M. Cover. 1988. A sandwich proof of Jelinek, F. and R. L. Mercer. 1980. Interpolated estimation of the Shannon-McMillan-Breiman theorem. The Annals of Probability, 16(2):899–909. Gelsema and L. N. Kanal, eds, Proceedings, Workshop Bahl, L. R., F. Jelinek, and R. L. Mercer. 1983. A maximum likelihood approach to continuous speech recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence, 5(2):157–166. Jelinek, F., R. L.

Page 26:

26 Chapter 3 • N-gram Language Models Stolcke, A. 2002. SRILM—an extensible language modeling toolkit. ICSLP. Trnka, K., D. Yarrington, J. McCaw, K. F. McCoy, and C. Pennington. 2007. The effects of word prediction on communication rate for AAC. NAACL-HLT. Witten, I. H. and T. C. Bell. 1991. The zero-frequency problem: Estimating the probabilities of novel events in adaptive text compression. IEEE Transactions on Information Theory, 37(4):1085–1094.

1.6 Module 1 Notes

- Introduction:
- Generic Natural Language Processing (NLP) system [CO1]
- NLP is the field that enables computers to understand and respond to human language. It is crucial for applications like email filtering (e.g., spam detection). NLP has evolved from traditional rule-based systems to using machine learning and deep learning techniques, especially from the 1980s onward. Natural language processing (NLP) is a branch of artificial intelligence that helps computers understand, interpret and manipulate human language.
-
- Natural Language Processing, usually shortened as NLP, is a branch of artificial intelligence that deals with the interaction between computers and humans using the natural language.
-
- The ultimate objective of NLP is to read, decipher, understand, and make sense of the human languages in a manner that is valuable. It is an interdisciplinary subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language.

-
- Text Analytics and NLP
- Text analytics focuses on extracting meaningful insights from text data, which may not always be human language. In contrast, NLP includes both text and speech recognition. The two branches of NLP are: -
- Natural Language Understanding (NLU): Comprehension of spoken or written language by machines.
- Natural Language Generation (NLG): Machines producing human-understandable text.
- Stages in NLP, [CO1]
- To converse with human a program must understand the syntax(grammar), semantics (word meaning) and morphology (word level analysis), and pragmatics(conversation).
- There are certain phases in which Natural language processing is performed.
-
- 1. Morphological Analysis
- It deals with morphemes which are the smallest units of meaning in a word. It is important for understanding structure of words and their parts by identifying free morphemes (independent words like "cat") and bound morphemes (like prefixes or suffixes e.g. "un-" or "-ing").
- Key tasks in morphological analysis:
- Stemming: Reducing words to their root form like "running" to "run".

2 Module 2: Semantic Analysis and Word Embeddings

2.1 1 Vector Semantics *Final*

Slide 1:

Computational Semantics and Semantic Parsing

Slide 2:

What Is Semantic Analysis in NLP?

To understand the meaning of Natural Language

Captures the meaning of the given text while considering context, logical structuring of sentences and grammar roles

Due to the vast complexity and subjectivity involved in human language, interpreting it is quite a complicated task for machines

Slide 3:

What is Semantics ? Word

A single distinct meaningful element of speech or writing, used with others to form a sentence.

Words that occur in similar contexts tend to have similar meanings.

Words which are synonyms (like oculist and eye-doctor) tended to occur in the same environment.

Slide 8:

Vector Semantics:

It represent a word as a point in some multidimensional semantic space.

Vectors for representing words are generally called embeddings, because the word is embedded in a particular vector space.

Vector semantic models can be learned automatically from text without any complex labeling or supervision.

As a result of these advantages, vector models is now a standard way to represent the meaning of words in NLP.

Slide 10:

Vector Space

Slide 11:

Word Space

Target words: automobile, car, soccer, football

Term vocabulary: wheel, transport, passenger, tournament, London, goal, match

Slide 12:

Constructing Word spaces

Informal algorithm for constructing word spaces

Pick the words you are interested in: target words

Define a context window, number of words surrounding target word

The context can in general be defined in terms of documents, paragraphs or sentences.

Count number of times the target word co-occurs with the context words: co-occurrence matrix

Build vectors out of (a function of) these co-occurrence counts

Slide 13:

Constructing Word spaces: distributional vectors

Slide 14:

Vector Space

Slide 15:

Computing similarity

Slide 16:

Vector Space Model without distributional similarity

Words are treated as atomic symbols

One-hot representation

Slide 17:

Building a DSM step-by-step

Slide 18:

Building a DSM step-by-step

Slide 19:

Many design choices

Slide 20:

The parameter space

Slide 21:

Documents as context: Word $\times$ document

Slide 22:

Words as context: Word $\times$ Word

Slide 24:

fool

Slide 25:

Context weighting: documents as context

Slide 27:

Bag of Words

Slide 28:

Bag of words Model

Text modelling technique .

Convert a text into its equivalent vector of numbers.

It is a method of feature extraction with text data.

Slide 29:

Understanding Bag of Words with an example

Example(1) without preprocessing:

Sentence 1: "Welcome to Great Learning, Now start learning"

Sentence 2: "Learning is a good practice"

Slide 30:

Solution:

Example:

Sentence 1: "Welcome to Great Learning, Now start learning"

Sentence 2: "Learning is a good practice"

Bag of Words with an example

Slide 31:

Step 1: Go through all the words in the above text and make a list of all the words in our model vocabulary.

Welcome

To

Great

Learning

,

Now

start

learning

is

a

good

practice

Slide 32:

Frequency Table for both sentences

writing the above frequencies in the vector form

Sentence 1 [1,1,1,1,1,1,1,1,0,0,0]

Sentence 2 [0,0,0,0,0,0,0,1,1,1,1]

Slide 33:

Example(2) with preprocessing:

Example(2) with preprocessing:

Sentence 1: "Welcome to Great Learning, Now start learning"

Sentence 2: "Learning is a good practice"

Step 1: Convert the above sentences in lower case as the case of the word does not hold any information.

Step 2: Remove special characters and stopwords from the text. Stopwords are the words that do not contain much information about text like 'is', 'a', 'the' and many more'.

Slide 34:

Occurrence matrix

Writing the above frequencies in the vector

Sentence 1 [1,1,2,1,1,0,0]

Sentence 2 [0,0,1,0,0,1,1]

Slide 35:

Disadvantages of bag of Words

Treats all words equally → Common words (e.g., the, is, and) get same importance as meaningful words.

No importance weighting → Cannot highlight rare but informative words.

High frequency bias → Words appearing many times dominate even if not meaningful.
No document-level context → Doesn't consider how unique a word is across documents.
Sparse and noisy features → Large vocabulary with many low-value words.

Slide 36:

Disadvantages of bag of Words

Slide 40:

Practice Problem

Compute TFIDF value for word Cat given in a small corpus with three documents:

Document 1: "The cat sat on the mat"

Document 2: "The dog sat on the log"

Document 3: "The cat and the dog played"

Slide 41:

Compute Term Frequency (TF)

The Term Frequency (TF) is calculated as:

TF = Number of times term appears in document / Total terms in document

Let's compute TF for the word "cat" in each document:

Document 1: "The cat sat on the mat" Total words = 6, Occurrences of "cat" = 1
D1) $1/6 = 0.1667$

Document 2: "The dog sat on the log" Total words = 6, Occurrences of "cat" = 0
D2) $0/6 = 0$

Document 3: "The cat and the dog played" Total words = 7, Occurrences of "cat" = 1
D3) $1/7 = 0.1429$

Slide 42:

Step 3: Compute Inverse Document Frequency (IDF)

The Inverse Document Frequency (IDF) is calculated as:

IDF = $\log(\text{Total number of documents} / \text{Number of documents containing the term})$

For the term "cat":

The total number of documents = 3

Documents containing "cat" = 2 (D1 and D3)

IDF(cat) = $\log(3/2) = \log(1.5) = 0.1761$

Slide 43:

Compute TF-IDF Score

The TF-IDF score is obtained by multiplying TF and IDF:

TFIDF = TF × IDF

For "cat":

Document 1: $\text{TFIDF}(\text{cat}, \text{D1}) = 0.1667 \times 0.1761 = 0.0294$

Document 2: $\text{TFIDF}(\text{cat}, \text{D2}) = 0 \times 0.1761 = 0$

Document 3: $\text{TFIDF}(\text{cat}, \text{D3}) = 0.1429 \times 0.1761 = 0.0251$

Final TF-IDF Scores for "cat"

Slide 44:

Example : TFIDF

News Article Classification Problem

Slide 45:

Example : TFIDF

News Article Classification Problem

Slide 46:

Consider The Document Frequency

Inverse Document Frequency : -

Slide 47:

TF-IDF

Slide 48:

Why Log of IDF

Slide 49:

Numerical TF-IDF

Consider 3 documents:

D1: "data science is fun"

D2: "machine learning is fun"

D3: "data mining and machine learning"

Step 1: Vocabulary (Unique Terms)

data, science, is, fun, machine, learning, mining, and

Slide 50:

Step 2: Term Frequency (TF)

$TF = (\text{Number of times term appears in document}) / (\text{Total terms in that document})$

Document D1: "data science is fun"

Document D2: "machine learning is fun"

Document D3: "data mining and machine learning"

Slide 51:

Step 3: Document Frequency (DF)

Slide 53:

TF-IDF for D2

TF-IDF for D3

Slide 54:

Final TF-IDF Vector Form

Slide 55:

Point Mutual Information

An alternative weighting function to tf-idf

Ask whether a context word is particularly informative about the target word.

Pointwise mutual information is one of the most important concepts in NLP.

It is a measure of how often two events x and y occur, compared with what we would expect if they were independent

Slide 56:

Do events x and y co-occur more than if they were independent?

$PMI(X, Y) = \log (P(x, y) / (P(x) P(y)))$

PMI between two words: (Church Hanks 1989)

Do words word and word co-occur more than if they were independent?

$PMI(\text{word}, \text{word}) = \log (P(\text{word}, \text{word}) / (P(\text{word}) P(\text{word})))$

Pointwise mutual information:

Slide 57:

Computing PPMI on a term-context matrix

Matrix F with W rows (words) and C columns (contexts)

f_{ij} is # of times w_i occurs in context c_j

57

Slide 58:

$p(w=\text{information}, c=\text{data}) =$

$p(w=\text{information}) =$

$p(c=\text{data}) =$

58

$= .3399$

$3982/111716$

$7703/111716$

$= .6575$

5673/11716

= .4842

Slide 59:

59

PMI(information,data) =

Resulting PPMI matrix (negatives replaced by 0)

Slide 60:

Weighting PMI

PMI is biased toward infrequent events

Very rare words have very high PMI values

Two solutions:

Give rare words slightly higher probabilities

Use add-one smoothing (which has a similar effect)

60

Slide 61:

Weighting PMI: Giving rare context words slightly higher probability

61

Slide 62:

$p(w=\text{information},c=\text{data}) = 6/19$ $p(w=\text{information}) = 11/19 = .58$ $p(c=\text{data}) = 7/19 =$

.37

$p(w,\text{context})$

$p(w)$

= .32

ij

p

fij

W C

fij

i1 j1

i

$p(w)$

C

fij

j1

N

$p(cj) \quad i1$

W

fij

N

Slide 63:

ij

$\text{pmi} \quad \log$

2

pij

p p

$i * j$

$\text{pmi}(\text{information},\text{data}) = \log_2 (.32 / (.37*.58)) = .57$

PPMI(w,context)

$p(w,\text{context})$

$p(w)$

Slide 64:

Add-2 Smoothed Count

$(w, context)$

$p(w, context)$ [add02]

$p(w)$

Slide 65:

PPMI versus add-2 smoothed PPMI

$PPMI(w, context)$

Slide 66:

Numerical Example: Consider the following word-context frequencies from a corpus:

Slide 68:

PPMI and Rare Words

PPMI tends to favor rare co-occurrences, which means it gives higher scores to words that rarely occur but co-occur with certain context words more often than expected by chance.

Example:

Let's say we have the following corpus:

markdown

1. The dog barked at the cat.
2. The cat sat on the mat.
3. The dog and the cat watched the ****unicorn****.

"dog", "cat": These words are common and occur in several sentences.

"unicorn": This word is rare—it appears only once, in a single unique sentence.

Slide 69:

PPMI Calculation:

Even though "unicorn" is rare, it gets a high PPMI score with the context "cat" because it occurs with "cat" more frequently than expected (given the general rarity of "unicorn").

This is useful in some NLP tasks (e.g., word similarity) because rare but important relationships are highlighted.

PPMI Example Value:

$PPMI("unicorn", "cat") = 3.5$ (High value)

$PPMI("dog", "cat") = 0.2$ (Low value due to frequent co-occurrence)

Slide 70:

Weighting PMI

PMI is biased toward infrequent events

Very rare words have very high PMI values

Two solutions:

Give rare words slightly higher probabilities

Use add-one smoothing (which has a similar effect)

Slide 71:

How PPMI is Different from TF-IDF

Slide 72:

Computing word similarity: Dot product and cosine

The dot product between two vectors is a scalar:

The dot product tends to be high when the two vectors have large values in the same dimensions

Dot product can thus be a useful similarity metric between vectors

Slide 73:

Problem with raw dot-product

Dot product favors long vectors

Dot product is higher if a vector is longer (has higher values in many dimension)

Vector length:

Frequent words (of, the, you) have long vectors (since they occur many times with other words).

So dot product overly favors frequent words

Slide 74:

Alternative: cosine for computing word similarity

Based on the definition of the dot product between two vectors a and b

Slide 75:

Cosine as a similarity metric

-1: vectors point in opposite directions

+1: vectors point in same directions

0: vectors are orthogonal

But since raw frequency values are non-negative, the cosine for term-term matrix vectors ranges from 0–1

75

Slide 76:

Cosine examples

76

Slide 77:

Visualizing cosines (well, angles)

Slide 78:

Word2Vec

Word Embedding is a language modeling technique used for mapping words to vectors of real numbers.

It represents words or phrases in vector space with several dimensions.

Word embeddings can be generated using various methods like neural networks, co-occurrence matrix, probabilistic models, etc.

Slide 79:

Word2Vec

Word2Vec creates vectors of the words that are distributed numerical representations of word features

these word features could comprise of words that represent the context of the individual words present in our vocabulary.

Word embeddings eventually help in establishing the association of a word with another similar meaning word through the created vectors.

Slide 80:

Word2Vec

Word2Vec consists of models for generating word embedding.

These models are shallow two-layer neural networks having one input layer, one hidden layer, and one output layer.

Word2Vec utilizes two architectures :

CBOW (Continuous Bag of Words)

Skip Gram

Slide 82:

Issues in BOW /TF_IDF

Semantics meaning is not captured

Similar words should have similar kind of vectors

Sparse Matrix is generated

Increase no of dimensions

Vectors created in word2Vec

Limited dimension

Sparsity is reduce (Create dense vector)

Semantic Meaning is maintained

Slide 84:

How does CBOW work?

Word2Vec is an unsupervised model

you can give a corpus without any label information and the model can create dense word embeddings,

Word2Vec internally leverages a supervised classification model to get these embeddings from the corpus.

Slide 85:

How does CBOW work

The CBOW architecture comprises a deep learning classification model in which we take in context words as input, X, and try to predict our target word, Y.

Slide 87:

Architecture of CBOW

Slide 88:

Math Behind The Above Calculation

Slide 89:

Skip-Gram Model

The skip-gram model is a simple neural network with one hidden layer trained in order to predict the probability of a given word being present when an input word is present.

Intuitively, you can imagine the skip-gram model being the opposite of the CBOW model.

In this architecture, it takes the current word as an input and tries to accurately predict the words before and after this current word.

This model essentially tries to learn and predict the context words around the specified input word.

2.2 2 SemanticAnalysisPart2

Slide 1:

Semantic Analysis

—

Lexical Semantics

Slide 2:

WORDNET:

WordNet is a database of lexical relations for English (Fellbaum, 1998).

WordNet consists of Three databases: Noun, Verb, Adjective Adverb.

Slide 3:

WORDNET:

A hierarchically organized lexical database

- On-line thesaurus + aspects of a dictionary
- Word senses and sense relations
- Some other languages available or under development(Arabic, Finnish, German, Portuguese...)

Slide 6:

SYNSETS

Different words with similar meaning is a set of synonyms representing a sense

Example:

Chump person who is gullible easy to take advantage of

Chump1 , fool 2, gull 1, guy 1, mark 9

9th sense of mark is this meaning

1st sense of guy is same as 2nd sense of fool

Slide 7:

2

Relationships between word meanings

Homonymy

Polysemy

Synonymy

Antonymy

Hypernymy

Hyponymy

Meronymy

Slide 8:

lemma and wordform

Semantic Analysis

8

Slide 9:

Lemmas have senses

One lemma “bank” can have many meanings:

Sense 1:

- ...a bank can hold the investments in a custodial 1

Account...

Sense 2:

“...as agriculture burgeons on the east bank the river will shrink even more” 2

- Sense (or word sense)
- A discrete representation of an aspect of a word’s meaning.
- The lemma bank here has two senses 14

Semantic Analysis

9

Slide 10:

Lemma Pepper

Sense 1: spice from pepper plant

Sense 2: the pepper plant itself

Sense 3: another similar plant (Jamaican pepper)

Sense 4: another plant with peppercorns (California pepper)

Sense 5: capsicum (i.e. chili, paprika, bell pepper, etc)

A sense or word sense is discrete representation of one aspect of meaning of word.

Slide 11:

Word Senses

A sense is discrete representation of one aspect of meaning of word.

A sense or “concept” is the meaning component of a word

There are relations between senses

Semantic Analysis

9

Slide 12:

Although a form can have more than one meaning, there is always a primary or original meaning that it expresses.

The original meaning of a form in a language is normally called “denotation”.

With respect to words (lexical terms), the primary (original) meaning is the meaning that we can find in a dictionary.

Example: “Rose” “A garden plant with thorns on its stem and pleasant-smelling flowers, or a flower from this plant”.

Semantic Analysis

12

“Denotation” (Primary Dictionary Meaning)

Slide 13:

Besides, there are additional meaning, which is known as “connotation”.

With respect to words, connotation or additional meanings of a word are not listed in a dictionary; and therefore, we cannot find them in a dictionary

Normally, a person expresses a connotative meaning through a word, phrase, clause or sentence based on certain characteristics of the entity or event that he/she is referring to.

Example: one may call a beautiful girl a “rose” or a “lily”.

13

Connotation

Slide 14:

HOMONYMY

Sense relation in which one form has different meaning; Same Spelling has different Meaning (treated as such in dictionaries)

no relatedness in meaning

e.g. bank1-side of a river

bank2-financial institution

Semantic Analysis

14

Slide 15:

Semantic Analysis

15

Slide 16:

Semantic Analysis

16

Slide 17:

POLYSEMY

One word (Same Spelling) having several closely related senses

Native speaker has clear intuition that the different senses are related to each other

Polysemy-close relatedness in m. which is usually connected to metaphorical extension

meaning relatedness has to be synchronic

Semantic Analysis

17

Slide 18:

The following sentences contain some examples of polysemy.

He drank a glass of milk.

He forgot to milk the cow.

His cottage is near a small wood.

The statue was made out of a block of wood.

He fixed his hair.

They fixed a date for the wedding.

Semantic Analysis

18

Slide 19:

25

Polysemy

The bank is constructed from red brick I withdrew the money from the bank

Are those the same sense?

Which sense of bank is this?

Is it distinct from (homonymous with) the river bank sense?

How about the savings bank sense?

A single lexeme with multiple related meanings (bank the building, bank the financial institution)

Slide 20:

Semantic Analysis

20

Slide 21:

Polysemy is the state or phenomenon in which the words that have more than one meaning.

In other words, it can be described as multiple meanings of words. The words are considered to be related etymologically.

The concrete form of polysemy is called “polyseme”.

Examples: Simple (e.g. English is extremely plain subject).

Plain With nothing added/not decorated in any way (e.g. This blouse is too plain)

Semantic Analysis

21

Slide 22:

HOMONYMY-POLYSEMY DISTINCTION

Homonymy-polysemy distinction: closeness or relatedness of senses of ambiguous w.

Homonymy- different senses of an ambiguous

are far apart from each other and not related via

speaker’s intuition;

matter of accident or coincidence e.g. bank, mug; kit, bar

Semantic Analysis

22

Slide 23:

Semantic Analysis

23

Slide 24:

Relation: Synonymy

Synonyms have the same meaning in some or all contexts.

filbert / hazelnut

couch / sofa

big / large

automobile / car

vomit / throw up

Water / H₂O

Slide 25:

Relation: Synonymy

Note that there are probably no examples of perfect synonymy.

Even if many aspects of meaning are identical

Still may not preserve the acceptability based on notions of politeness, slang, register, genre, etc.

The Linguistic Principle of Contrast:

Difference in form -> difference in meaning

Slide 26:

Synonymy is a relation

between senses rather than words

Consider the words big and large

- Are they synonyms?

How big is that plane?

Would I be flying on a large or small plane?

- **How about here:**

Miss Nelson became a kind of big sister to Benjamin.

?Miss Nelson became a kind of large sister to Benjamin.

- Why?

big has a sense that means being older, or grown up • large

lacks this sense

Semantic Analysis

26

Slide 27:

Hyponymy and Hypernymy

One sense is a hyponym of another if the first sense is more specific, denoting a subclass of the other

- car is a hyponym of vehicle
- mango is a hyponym of fruit

Conversely hypernym/superordinate (“hyper is super”)

vehicle is a hypernym of car

fruit is a hypernym of mango

Semantic Analysis

27

Slide 28:

Hyponymy (i.e. category membership)

It may be problematic to identify the superordinate terms:

brother sister < sibling (formal)

uncle aunt < ?

cow bull < cow/cattle (collective)/bovine (technical)

human being animal < animal (vs. vegetable, mineral)

fish

snapper

trout bass [bæs] sole

salmon [sæm n]

chinook [(t) nuk]

spring coho [k h]

king

sockeye [sk a]

hypernym

(co)hyponyms

Slide 29:

Hyponymy more formally

Extensional:

The class denoted by the superordinate extensionally

includes the class denoted by the hyponym

Entailment:

A sense A is a hyponym of sense B if being an A entails being a B •

Hyponymy is usually transitive

(A hypo B and B hypo C entails A hypo C)

- **Another name:** the IS-A hierarchy

A IS-A B (or A ISA B)

B subsumes A

Semantic Analysis

43

Slide 30:

Antonyms

Senses that are opposites with respect to one feature of meaning

Otherwise, they are very similar!

dark/light short/long fast/slow rise/fall hot/cold up/down in/out

• **More formally:** antonyms can • define a binary opposition or be at opposite ends of a scale

• long/short, fast/slow

• **Be reversives:**

rise/fall, up/down

Semantic Analysis

44

Slide 31:

Metaphor and Metonymy

31

Specific types of polysemy

Metaphor:

Germany will pull Slovenia out of its economic slump. (compares Slovenia's economic difficulties to a physical "slump" or pit.)

I spent 2 hours on that homework. (time being treated as money or a resource)

Metonymy -the act of referring to something by the name of something else that is closely connected with it, for example using the White House for the US President

The White House announced yesterday.

This chapter talks about part-of-speech tagging

Bank (building) and bank (financial institution)

Slide 32:

How do we know when a word has more than one sense?

32

ATIS examples

Which flights serve breakfast?

Does America West serve Philadelphia?

The “zeugma” test:

?Does United serve breakfast and San Jose?

Slide 33:

Synonyms

33

Words that have the same meaning in some or all contexts.

hazelnut adolescent large

filbert

youth

big

automobile car

Two lexemes are synonyms if they can be successfully substituted for each other in all situations

Slide 34:

Synonyms

34

But there are few (or no) examples of perfect synonymy.

Why should that be?

Even if many aspects of meaning are identical

Still may not preserve the acceptability based on notions of politeness, slang, register, genre, etc.

Example:

Big and large?

That's my big sister

That's my large sister

Slide 35:

Antonyms

35

light

Words that are opposites with respect to one feature of their meaning

Otherwise, they are very similar!

Dark

Boy girl

Hot cold

Up

In

down out

Slide 36:

Word Similarity Computation

36

For various computational applications it's useful to find words which are similar to another word.

Machine translation (to find near-synonyms)

Information retrieval (to do "query expansion")

Two ways to do this:

Automatic computation based on distributional similarity

Lsa.colorado.edu

Use a thesaurus which lists similar words.

WordNet (we'll return to this in a few slides)

2.3 3 Word Sense Disambiguation

Slide 1:

Word Sense Disambiguation

Slide 3:

Word Sense Disambiguation Problem

WSD is the problem of identifying which "sense" (meaning) of a word is activated by the use of the word in a particular context or scenario.

deals with determining the intended meaning of a word in a given context.

It is the process of identifying the correct sense of a word from a set of possible senses, based on the context in which the word appears.

Slide 4:

Sense ambiguity

Many words have several meanings or senses

The meaning of bass depends on the context

Are we talking about music, or fish?

An electric guitar and bass player stand off to one side, not really part of the scene, just as a sort of nod to gringo expectations perhaps.

And it all started when fishermen decided the striped bass in Lake Mead were too skinny.

Disambiguation

The task of disambiguation is to determine which of the senses of an ambiguous word is invoked in a particular use of the word.

Slide 5:

Application
Machine Translation
Information Retrieval
Question Answering
Text Summarization
Sentiment Analysis
Chatbots Virtual Assistants
Search Engines

Example:

Apple released a new product.
Is Apple → fruit or company ?

Slide 6:

Algorithms
Knowledge Based Approaches
Overlap Based Approaches
Machine Learning Based Approaches
Supervised Approaches
Semi-supervised Algorithms
Unsupervised Algorithms
Hybrid Approaches

Slide 7:

Approaches:

- Dictionary- and knowledge-based methods: These rely primarily on dictionaries, thesauri, and lexical knowledge bases, without using any corpus evidence.
- **Supervised methods:** These make use of sense-annotated corpora to train from.
- **Semi-supervised or minimally-supervised methods:** These make use of a secondary source of knowledge such as a small annotated corpus as seed data in a bootstrapping process, or a word-aligned bilingual corpus

Slide 8:

WORDNET:

WordNet is a database of lexical relations for English (Fellbaum, 1998).

WordNet consists of Three databases: Noun, Verb, Adjective Adverb.

Slide 9:

WORDNET:

A hierarchically organized lexical database

- On-line thesaurus + aspects of a dictionary
- Word senses and sense relations
- Some other languages available or under development(Arabic, Finnish, German, Portuguese...)

Slide 12:

SYNSETS

Different words with similar meaning is a set of synonyms representing a sense

Example:

Chump person who is gullible easy to take advantage of
Chump1 , fool 2, gull 1, guy 1, mark 9
9th sense of mark is this meaning
1st sense of guy is same as 2nd sense of fool

Slide 13:

HIERARCHICAL SYNSET RELATIONS: NOUNS

Hypernym/hyponym (between concepts)

The more general 'meal' is a hypernym of the more specific 'breakfast'

Instance hypernym/hyponym (between concepts and instances)

Austen is an instance hyponym of author

Member holonym/meronym (groups and members)

professor is a member meronym of (a university's) faculty

Part holonym/meronym (wholes and parts)

wheel is a part meronym of (is a part of) car.

Substance meronym/holonym (substances and components)

flour is a substance meronym of (is made of) bread

Slide 14:

HIERARCHICAL SYNSET RELATIONS: VERBS

Hypernym/troponym (between events):

travel/fly, walk/stroll

Flying is a troponym of traveling:

it denotes a specific manner of traveling

Entailment (between events):

snore/sleep

Snoring entails (presupposes) sleeping

Slide 17:

Dictionary-based WSD: Lesk algorithm

Slide 19:

Dictionary / Knowledge /Thesaurus Based Approaches

Lesk algorithm

Walkers algorithm

We have a dictionary/thesaurus that contains glosses and examples:

bank1

Gloss: a financial institution that accepts deposits and channels the money into lending activities

Examples: "he cashed the check at the bank", "that bank holds the mortgage on my home"

bank2

Gloss: sloping land (especially the slope beside a body of water)

Examples: "they pulled the canoe up on the bank", "he sat on the bank of the river and watched the current"

We could use anything in the thesaurus

§ Example sentences

§ Derivational relations and sentence frames

Slide 20:

Knowledge Based Approaches

Overlap Based Approaches

Require a Machine Readable Dictionary (MRD).

For Example WordNet (all the information should in form that machine can understand)

Steps:

Take the ambiguous word.

List all possible senses from dictionary.

Compare definition words with surrounding context.

Select sense with maximum word overlap.

Find the overlap between the features of different senses of an ambiguous word (sense bag) and the features of the words in its context (context bag).

The features could be sense definitions, example sentences, hypernyms etc.

The features could also be given weights.

The sense which has the maximum overlap is selected as the contextually appropriate sense.

Slide 21:

Lesk's Algorithm

Sense Bag: contains the words in the definition of a candidate sense of the ambiguous word.

Context Bag: contains the words in the definition of each sense of each context word.

Example : -On burning coal we get ash.

Slide 22:

She drank Some Milk

She drank Some books

Slide 23:

Walker's Algorithm

A Thesaurus Based approach

What is Thesaurus ?

A thesaurus is a book, software program, or online service that provides alternative or similar words to a word.

For example, searching for "hope" may return synonyms like "achievement," "faith," "ambition," and "optimism."

Thesaurus vs. dictionary

A thesaurus is used to group different words with the same meaning (synonyms) and similar words.

On the other hand, a dictionary explains the definition of a word.

For example, looking up the word "computer" in a thesaurus may give words like PC, CPU, calculator, abacus, and laptop that could be used in place of the word computer.

Looking up the word "computer" in a dictionary would define the word like what is found on our computer definition.

Slide 24:

Walker's Algorithm -A Thesaurus Based approach

Steps

Identify the ambiguous word.

Extract all possible senses of the word from a dictionary / WordNet.

Identify context words surrounding the ambiguous word.

Measure semantic similarity between each sense of the ambiguous word and each context word.

Select the sense with maximum total similarity score.

semantic similarity Calculation:

Step 1: For each sense of the target word find the thesaurus category to which that sense belongs

Step 2: Calculate the score for each sense by using the context words.

A context word will add 1 to the score of the sense if the thesaurus category of the word matches that of the sense.

E.g. The money in this bank fetches an interest of 8

Target word: bank

Clue words from the context: money, interest, annum, fetch

Slide 25:

Key Differences:

Slide 26:

MRD – A Resource for Knowledge-based WSD

For each word in the language vocabulary, an MRD provides:

A list of meanings

Definitions (for all word meanings)

Typical usage examples (for most word meanings)

WordNet definitions/examples for the noun plant

buildings for carrying on industrial labor; "they built a large plant to manufacture automobiles"

a living organism lacking the power of locomotion

something planted secretly for discovery by another; "the police used a plant to trick the thieves"; "he claimed that the evidence against him was a plant"

an actor situated in the audience whose acting is rehearsed but seems spontaneous to the audience

Slide 30:

1. Constructing the Graph:

Identify ambiguous words in the sentence.

Extract all their possible senses using WordNet / BabelNet.

Build a semantic graph connecting all senses.

Add weighted edges based on semantic relationships.

Run graph algorithms to compute node importance.

Choose highest-ranked sense.

E.g.

The sentence "She drank some milk" contains two key words: "drink" and "milk".

These words are inserted into a graph.

Each word has multiple senses (meanings), which are represented as nodes (e.g., "drink", "drink", etc., and "milk", "milk", etc.).

Directed edges link the words to their possible senses.

Additional edges connect the senses based on their relationships, such as synonyms, hypernyms, or other lexical relations.

Edge Weights can be assigned based on Semantic similarity, Relation type, Path length

2. Choosing the Most Central Sense:

The algorithm assigns some initial probability to each word ("drink" and "milk").

Using PageRank, a centrality algorithm, the most important or frequently visited sense in the graph is identified.

The sense with the highest PageRank score is chosen as the correct meaning. (good page rank is given based on Connectivity, Edge weights, Importance of neighboring nodes)

Slide 31:

WSD Using Random Walk Algorithm

Slide 34:

Page rank Algorithm : ->

Slide 36:

KB Approaches –Conclusions

36

CF

Drawbacks of WSD using Selectional Restrictions

Needs exhaustive Knowledge Base.

Drawbacks of Overlap based approaches

Dictionary definitions are generally very small.

Dictionary entries rarely take into account the distributional constraints of different word senses (e.g. selectional preferences, kinds of prepositions, etc. cigarette and ash never co-occur in a dictionary).

Suffer from the problem of sparse match.

Proper nouns are not present in a MRD. Hence these approaches fail to capture the strong clues provided by proper nouns.

E.g. "Sachin Tendulkar" will be a strong indicator of the category "sports".

Sachin Tendulkar plays cricket.

Slide 37:

Supervised methods

Supervised methods assumes that the context can provide enough evidence on its own to disambiguate words

Probably every machine learning algorithm has been applied to WSD, including associated techniques such as feature selection, parameter optimization, and ensemble learning.

Support vector machines and memory-based learning have been shown to be the most successful approaches, to date, probably because they can cope with the high dimensionality of the feature space.

However, these supervised methods are subject to a new knowledge acquisition bottleneck since they rely on substantial amounts of manually sense tagged corpora for training, which are laborious and expensive to create.

Slide 39:

Word Sense Disambiguation

Classification

Slide 40:

Dan Jurafsky

Classification: definition

Input:

a word w and some features f

a fixed set of classes $C = c_1, c_2, \dots, c_J$

Output: a predicted class c_C

Slide 41:

Dan Jurafsky

Classification Methods: Supervised Machine Learning

Input:

a word w in a text window d (which we'll call a "document")

a fixed set of classes $C = c_1, c_2, \dots, c_J$

A training set of m hand-labeled text windows again called "documents" $(d_1, c_1), \dots, (d_m, c_m)$

Output:

a learned classifier : $d \rightarrow c$

2

2

Slide 42:

Dan Jurafsky

Classification Methods: Supervised Machine Learning

Any kind of classifier

Naive Bayes

Logistic regression

Neural Networks

Support-vector machines

k-Nearest Neighbors

...

Slide 43:

Applying Naive Bayes to WSD

$P(c)$ is the prior probability of that sense

Counting in a labeled training set.

$P(w|c)$ conditional probability of a word given a particular sense

$P(w|c) = \text{count}(w, c) / \text{count}(c)$

We get both of these from a tagged corpus like SemCor

Can also generalize to look at other features besides words.

Then it would be $P(f|c)$
 Conditional probability of a feature given a sense

2.4 PMI

Page 1:

CHAPTER J Pointwise Mutual Information (PMI) An alternative weighting function to tf-idf, PPMI (positive PMI), is used for term-term matrices, when the vector dimensions correspond to words rather than documents. PPMI draws on the intuition that the best way to weigh the association between two words is to ask how much more often they occur in our corpus than we would have a priori expected them to appear by chance. pointwise mutual information (Fano, 1961) is one of the most important concepts in NLP. It is

Page 2:

2 APPENDIX J • POINTWISE MUTUAL INFORMATION (PMI) More formally, let's assume we have a co-occurrence matrix F with W rows (words) and C columns (contexts), where f_{ij} gives the number of times word w_i occurs with context c_j . This can be turned into a PPMI matrix where $PPMI_{ij}$ gives the PPMI value of word w_i with context c_j (which we can also express as $PPMI(w_i, c_j)$ or $PPMI(i, j)$) as follows: $PPMI_{ij} = \log_2 \frac{f_{ij}}{f_i \cdot f_j}$ where $f_i = \sum_j f_{ij}$ and $f_j = \sum_i f_{ij}$.

Page 3:

3 $p(w, context)$ $p(w)$ computer data result pie sugar $p(w)$ cherry 0.0002 0.0007 0.0008 0.0377 0.0021 0.0415 strawberry 0.0000 0.0000 0.0001 0.0051 0.0016 0.0068 digital 0.1425 0.1436 0.0073 0.0004 0.0003 0.2942 information 0.2838 0.3399 0.0323 0.0004 0.0011 0.6575 $p(context)$ 0.4265 0.4842 0.0404 0.0437 0.0052 Figure J.3 Replacing the counts in Fig. J.1 with joint probabilities, showing the marginals in the right column and the bottom row. computer data result pie sugar cherry 0 0 0 4.38 3.30 strawberry 0 0 0 4.10 5.5

Page 4:

4 Appendix J • Pointwise Mutual Information (PMI) Church, K. W. and P. Hanks. 1989. Word association norms, mutual information, and lexicography. ACL. Church, K. W. and P. Hanks. 1990. Word association norms, mutual information, and lexicography. Computational Linguistics, 16(1):22–29. Dagan, I., S. Marcus, and S. Markov. Contextual word similarity and estimation from sparse data. ACL. Fano, R. M. 1961. Transmission of Information: A Statistical Theory of Communications. MIT Press. Levy, O., Y. Goldberg, and I. Dagan. 2015. Improving distributed

2.5 Word Senses

Page 1:

Speech and Language Processing. Daniel Jurafsky James H. Martin. Copyright © 2026. All rights reserved. Draft of January 25, 2026. CHAPTER I Word Senses and WordNet Lady Bracknell. Are your parents living? Jack. I have lost both my parents. Lady Bracknell. To lose one parent, Mr. Worthing, may be regarded as a misfortune; to lose both looks like carelessness. Oscar Wilde, The Importance of Being Earnest ambiguous Words are ambiguous: the same word can be used to mean different things. In Chapter 5 we saw that the word “mouse

Page 2:

2 APPENDIX I • WORD SENSES AND WORDNET analytic direction. I.1 Word Senses word sense A sense (or word sense) is a discrete representation of one aspect of the meaning of a word. Loosely following lexicographic tradition, we represent each sense with a superscript: bank¹ and bank², mouse¹ and mouse². In context, it's easy to see the different meanings: mouse¹: a mouse controlling a computer system in 1968. mouse²: a quiet animal like a mouse bank¹: ... a bank can hold the investments in a custodial account... bank²: ... as agriculture burgeons on the

Page 3:

I.1 • WORD SENSES 3 Yet despite their circularity and lack of formal representation, glosses can still be useful for computational modeling of senses. This is because a gloss is just a sentence, and from sentences we can learn something about the meaning of the sense. Dictionaries often give example sentences along with

glosses, and these can again be used to help build a sense representation. The second way that the thesaurus offers for defining a sense is—like the dictionary definitions—defining a sense through hits

Page 4:

4 APPENDIX I • WORD SENSES AND WORDNET I.2 Relations Between Senses This section explores the relations between word senses, especially those that have received significant computational investigation. **Synonymy** We introduced in Chapter 5 the idea that when two senses of two different words (synonyms (lemmas)) are identical, or nearly identical, we say the two senses are synonyms. Synonyms include such pairs as couch/sofa vomit/throwup filbert/hazelnut car/automobile And we mentioned t

Page 5:

I.2 • RELATIONS BETWEEN SENSES 5 and hyponym) are very similar and hence easily confused; for this reason superordinate is often used instead of hypernym. Superordinate vehicle fruit furniture mammal Subordinate car mango chair dog We can define hypernymy more formally by saying that the class denoted by the superordinate extensionally includes the class denoted by the hyponym. Thus, the class of animals includes as members all dogs, and the class of moving actions includes all walking actions. Hypernymy c

Page 6:

6 APPENDIX I • WORD SENSES AND WORDNET I.3 WordNet: A Database of Lexical Relations The most commonly used resource for sense relations in English and many other WordNet languages is the WordNet lexical database (Fellbaum, 1998). English WordNet consists of three separate databases, one each for nouns and verbs and a third for adjectives and adverbs; closed class words are not included. Each database contains a set of lemmas, each one annotated with a set of senses. The WordNet 3.0 release has 117,798 nouns, 11,529 verbs, 22,

Page 7:

I.3 • WORDNET: A DATABASE OF LEXICAL RELATIONS 7 **Supersense** called supersenses, because they act as senses which can be useful when word senses are too fine-grained (Ciaramita and Johnson 2003, Ciaramita and Altomonte 2004). Supersenses have also been defined for adjectives (Tsvetkov et al., 2014) and prepositions (Schneider et al., 2018). **Category Example** Category Example Category Example ACT service GROUP place PLANT tree ANIMAL dog LOCATION area POSSESSION price ARTIFACT car MOTIVE reason

Page 8:

8 APPENDIX I • WORD SENSES AND WORDNET **bass3**, **basso** (an adult male singer with the lowest voice) => singer, vocalist, vocalizer, vocaliser => musician, instrumentalist, player => performer, performing artist => entertainer => person, individual, someone... => organism, being => living thing, animate thing, => whole, unit => object, physical object => physical entity => entity **bass7** (member with the lowest range of a family of instruments) => musical instrument, instrument => device => instrumentality

Page 9:

I.4 • WORD SENSE DISAMBIGUATION 9 I.4.1 WSD: The Task and Datasets In this section we introduce the task setup for WSD, and then turn to algorithms. The inventory of sense tags depends on the task. For sense tagging in the context of translation from English to Spanish, the sense tag inventory for an English word might be the set of different Spanish translations. For automatic indexing of medical articles, the sense-tag inventory might be the set of MeSH (Medical Subject Headings) thesaurus entries. Or we can u

Page 10:

10 APPENDIX I • WORD SENSES AND WORDNET **y y 5 6 y 3** **stand1**: **side1**: **y y 1** **bass1**: 4 **upright** **relative low range** ... **region** **electric1**: ... **player1**: **stand5**: ... **using** **bass4**: **in game** **bear** **side3**: **electricity** **sea fish** **player2**: ... **of body** **electric2**: **y** ... **musician** **stand10**: ... **tense** **2** **bass7**: **player3**: **put** **side11**: **electric3**: **instrument** **actor** **upright** **slope** **thrilling** **guitar1** **x x x x x x 1 2 3 4 5 6** **an electric guitar and bass player** **stand off to one side** Figure I.8 The all-words WSD task, mapping from input words (x

Page 11:

I.4 • WORDSENSEDISAMBIGUATION 11 sense whose sense embedding has the highest cosine with: $\text{sense}(t) = \arg\max_s \text{cosine}(t, v_s)$ (I.14) $\text{ssenses}(t)$ Fig. I.9 illustrates the model. 5 4 find v find v 1 find v 9 find v c c c c c I found the jar empty ENCODER I found the jar empty Figure I.9 The nearest-neighbor algorithm for WSD. In green are the contextual embeddings precomputed for each sense of each word; here we just show a few of the senses for find. A contextual embedding is computed for the target word found, and then the nearest nei

Page 12:

12 APPENDIX I • WORDSENSES AND WORDNET I.5 Alternate WSD algorithms and Tasks I.5.1 Feature-Based WSD Feature-based algorithms for WSD are extremely simple and function almost as well as contextual language model algorithms. The best performing IMS algorithm (Zhong and Ng, 2010), augmented by embeddings (Iacobacci et al. 2016, Raganato et al. 2017b), uses an SVM-classifier to choose the sense for each input word with the following simple features of the surrounding words:

- part-of-speech tags (for a window of 3 words on each side, stoppin

Page 13:

I.5 • ALTERNATE WSD ALGORITHMS AND TASKS 13 function `SIMPLIFIEDLESK(word, sentence)` return best-sense $\leftarrow$ most frequent sense for word $\text{max-overlap} \leftarrow 0$ context $\leftarrow$ set of words in sentence for each sense in senses of word do $\text{signature} \leftarrow$ set of words in the gloss and examples of sense $\text{overlap} \leftarrow \text{COMPUTE_OVERLAP}(\text{signature}, \text{context})$ if $\text{overlap} > \text{max-overlap}$ then $\text{max-overlap} \leftarrow \text{overlap}$ $\text{best-sense} \leftarrow \text{sense}$ end return (best-sense) Figure I.10 The Simplified Lesk algorithm. The `COMPUTE\_OVERLAP` function returns the number of words in common between two sets, ignor

Page 14:

14 APPENDIX I • WORDSENSES AND WORDNET F There's a lot of trash on the bed of the river—I keep a glass of water next to my bed when I sleep F Justify the margins—The end justifies the means T Air pollution—Open a window and let in some air T The expanded window will give us time to catch the thieves—You have a two-hour window of clear weather to finish working on the lawn Figure I.11 Positive (T) and negative (F) pairs from the WiC dataset (Pilehvar and Camacho-Collados, 2019). word-in-context first cluster the word senses into coarser clusters, so that th

Page 15:

I.6 • USING THESAURUSES TO IMPROVE EMBEDDINGS 15 I.6 Using Thesauruses to Improve Embeddings Thesauruses have also been used to improve both static and contextual word embeddings. For example, static word embeddings have a problem with antonyms. A word like expensive is often very similar in embedding cosine to its antonym like cheap. Antonymy information from thesauruses can help solve this problem; Fig. I.12 shows nearest neighbor to some target words after one such method. Before

Page 16:

16 APPENDIX I • WORDSENSES AND WORDNET 1. Compute a context vector for t . 2. Retrieve all sense vectors s for w . 3. Assign to the sense represented by the sense vector s that is closest to t . All we need is a clustering algorithm and a distance metric between vectors. Clustering is a well-studied problem with a wide number of standard algorithms that can be applied to inputs structured as vectors of numerical values (Duda and Hart, 1973). A frequently used technique in language applications is known as agglomerative clustering. In t

Page 17:

HISTORICAL NOTES 17 • An important baseline for WSD is the most frequent sense, equivalent, in WordNet, to take the first sense. • Another baseline is a knowledge-based WSD algorithm called the Lesk algorithm which chooses the sense whose dictionary definitions share the most words with the target word's neighborhood. • Word sense induction is the task of learning word senses unsupervised. Historical Notes Word sense disambiguation traces its roots to some of the earliest applications of digital computers. The insight that unde

Page 18:

18 APPENDIX I • WORDSENSES AND WORDNET resources include Amsler's 1981 use of the Merriam Webster's Dictionary of Contemporary English (Boguraev and Briscoe, 1989). Supervised approaches to

disambiguation began with the use of decision trees by Black (1988). In addition to the IMS and contextual-embedding based methods for supervised WSD, recent supervised algorithms include encoder-decoder models (Raganato et al., 2017a). The need for large amounts of annotated text in supervised methods led early on to investigate

Page 19:

Exercises 19 Agirre, E. and O. L. de Lacalle. 2003. Clustering WordNet Hirst, G. 1987. Semantic Interpretation and the Resolution of Ambiguity. Cambridge University Press. Agirre, E. and P. Edmonds, eds. 2006. Word Sense Disambiguation: Algorithms and Applications. Kluwer. Hirst, G. 1988. Resolving lexical ambiguity computationally with spreading activation and polaroid words. In S. L. Small, G. W. Cottrell, and M. K. Tanenhaus, eds., Lexical Ambiguity Resolution, 73–108. Morgan Kaufmann.

Page 20:

20 Appendix I • Word Senses and WordNet Manandhar, S., I. P. Klapaftis, D. Dligach, and S. Pradhan. Ponzetto, S. P. and R. Navigli. 2010. Knowledge-rich word 2010. SemEval-2010 task 14: Word sense induction sensedisambiguation rivaling supervised systems. ACL. disambiguation. SemEval. Pu, X., N. Pappas, J. Henderson, and A. Popescu-Belis. Martin, J. H. 1986. The acquisition of polysemy. ICML. 2018. Integrating weakly supervised word sense disambiguation Masterman, M. 1957. The thesaurus in syntax and semantics. Transactions of the American Society for Computational Linguistics.

Page 21:

Exercises 21 Weaver, W. 1949/1955. Translation. In W. N. Locke and A. D. Boothe, eds., Machine Translation of Languages, 15–23. MIT Press. Reprinted from a memorandum written by Weaver in 1949. Wilks, Y. 1975a. An intelligent analyzer and understander of English. CACM, 18(5):264–274. Wilks, Y. 1975b. Preference semantics. In E. L. Keenan, ed., The Formal Semantics of Natural Language, 329–350. Cambridge Univ. Press. Wilks, Y. 1975c. A preferential, pattern-seeking, semantics for natural language inference. Artificial Intelligence, 6(1)

3 Module 3: Text Classification with Naive Bayes

3.1 Text Classification using Naive Bayes (3)

Slide 1:

Chapter 2 Text Classification

Slide 2:

What is Text Classification?

Have you performed The Task of Text Classification ?

Slide 3:

Some Examples of Text Classification?

Slide 4:

Is this spam? How will we decide ?

Slide 5:

What is the subject of this medical article?

Antagonists and Inhibitors

Blood Supply

Chemistry

Drug Therapy

Embryology

Epidemiology

...

5

MeSH Subject Category Hierarchy

?

MEDLINE Article

Slide 6:

Positive or negative movie review?

...zany characters and richly applied satire, and some great plot twists

It was pathetic. The worst part about it was the boxing scenes...

...awesome caramel sauce and sweet toasty almonds. I love this place!

...awful pizza and ridiculously overpriced...

6

+

+

Slide 7:

Positive or negative movie review?

...zany characters and richly applied satire, and some great plot twists

It was pathetic. The worst part about it was the boxing scenes...

...awesome caramel sauce and sweet toasty almonds. I love this place!

...awful pizza and ridiculously overpriced...

7

+

+

Slide 8:

Why sentiment analysis?

Movie: is this review positive or negative?

Products: what do people think about the new iPhone?

Public sentiment: how is consumer confidence?

Politics: what do people think about this candidate or issue?

Prediction: predict election outcomes or market trends from sentiment

8

Slide 9:

Scherer Typology of Affective States

Emotion: brief organically synchronized ... evaluation of a major event

angry, sad, joyful, fearful, ashamed, proud, elated

Mood: diffuse non-caused low-intensity long-duration change in subjective feeling

cheerful, gloomy, irritable, listless, depressed, buoyant

Interpersonal stances: affective stance toward another person in a specific interaction

friendly, flirtatious, distant, cold, warm, supportive, contemptuous

Attitudes: enduring, affectively colored beliefs, dispositions towards objects or persons

liking, loving, hating, valuing, desiring

Personality traits: stable personality dispositions and typical behavior tendencies

nervous, anxious, reckless, morose, hostile, jealous

Slide 10:

Basic Sentiment Classification

Sentiment analysis is the detection of attitudes

Simple task we focus on in this chapter

Is the attitude of this text positive or negative?

We return to affect classification in later chapters

Slide 11:

Text Classification

Sentiment analysis

Spam detection

Authorship identification

Language Identification

Assigning subject categories, topics, or genres

...

Slide 12:

Text Classification: definition

Input:

a document d

a fixed set of classes $C = c_1, c_2, \dots, c_J$

Output: a predicted class $c \in C$

Slide 13:

Classification Methods: Hand-coded rules

Rules based on combinations of words or other features

spam: black-list-address OR (“dollars” AND “you have been selected”)

Accuracy can be high

If rules carefully refined by expert

But building and maintaining these rules is expensive

Slide 14:

Classification Methods: Supervised Machine Learning

Input:

a document d

a fixed set of classes $C = c_1, c_2, \dots, c_J$

A training set of m hand-labeled documents $(d_1, c_1), \dots, (d_m, c_m)$

Output:

a learned classifier : $d \mapsto c$

14

Slide 15:

Classification Methods: Supervised Machine Learning Types

Any kind of classifier

Naïve Bayes

Logistic regression

Neural networks

k-Nearest Neighbors

SVM...

Slide 16:

How to use naïve bayes classifier for text classification?

Slide 17:

Generative and Discriminative Classifiers

Naive Bayes is a generative classifier

by contrast:

Logistic regression/SVM is a discriminative classifier

Slide 19:

Generative and Discriminative Classifiers

Suppose we’re distinguishing cat from dog images

imagenet

imagenet

Slide 20:

Generative Classifier:

Build a model of what’s in a cat image

Knows about whiskers, ears, eyes

Assigns a probability to any image:

how cat-y is this image?

Also build a model for dog images

Now given a new image:

Run both models and see which one fits better

Slide 21:

Discriminative Classifier

Just try to distinguish dogs from cats

Oh look, dogs have collars!

Let's ignore everything else

Slide 22:

Finding the correct class c from a document d in Generative vs Discriminative Classifiers

Naive Bayes

22

Slide 23:

How SVM Makes Decision

$y = \text{sign}(wx + b)$ Where :

w = weight vector

b = bias

$\text{sign}() \rightarrow$ decides class (+1 or 1)

Slide 24:

Why is Naïve Bayes generative?

NB assumes that features are conditionally independent given the class.

It first estimates $P(X|Y)$ (the likelihood of the features given the class) and $P(Y)$ (the prior probability of each class).

Then, it uses Bayes' theorem to compute

Since it explicitly models how the data is generated, it is a generative model.

Slide 25:

Why is Logistic Regression discriminative?

It does not model how the data is generated; instead, it directly finds the best decision boundary.

It learns a function of the form:

It maximizes the likelihood of the observed data, adjusting weights to separate classes optimally.

Since it directly maps input X to output Y without modeling $P(X)$ or $P(XY)$, it is a discriminative model.

Slide 26:

Naive Bayes Intuition

Simple ("naive") classification method based on Bayes rule

Relies on very simple representation of document

Bag of words

Slide 27:

The Bag of Words Representation

27

Slide 28:

The bag of words representation

(

) = c

Slide 29:

Supervised Learning

Slide 30:

Bayes' Rule Applied to Documents and Classes

For a document d and a class c

Slide 31:

Naive Bayes Classifier (I)

MAP is “maximum a posteriori” = most likely class

Bayes Rule

Dropping the denominator

Slide 32:

Naive Bayes Classifier (II)

Document d represented as features $x_1..x_n$

"Likelihood"

"Prior"

Slide 33:

Naïve Bayes Classifier (IV)

How often does this class occur?

$O(|X|n \cdot |C|)$ parameters

We can just count the relative frequencies in a corpus

Could only be estimated if a very, very large number of training examples was available.

Slide 34:

Multinomial Naive Bayes Independence Assumptions

Bag of Words assumption: Assume position doesn't matter

Conditional Independence: Assume the feature probabilities $P(x_i|c_j)$ are independent of others, given the class.

Slide 35:

Multinomial Naive Bayes Classifier

Slide 36:

Applying Multinomial Naive Bayes Classifiers to Text Classification

positions all word positions in test document

Slide 37:

Problems with multiplying lots of probs

There's a problem with this:

Multiplying lots of probabilities can result in floating-point underflow!

$.0006 * .0007 * .0009 * .01 * .5 * .000008 \dots$

Idea: Use logs, because $\log(ab) = \log(a) + \log(b)$

We'll sum logs of probabilities instead of multiplying probabilities!

Slide 38:

We actually do everything in log space

Instead of this:

This:

Notes:

1) Taking log doesn't change the ranking of classes!

The class with highest probability also has highest log probability!

2) It's a linear model:

Just a max of a sum of weights: a linear function of the inputs

So naive bayes is a linear classifier

Slide 39:

Parameter estimation

Create mega-document for topic j by concatenating all docs in this topic

Use frequency of w in mega-document

fraction of times word w_i appears

among all words in documents of topic c_j

Slide 40:

Learning the Multinomial Naive Bayes Model

First attempt: maximum likelihood estimates
simply use the frequencies in the data

Sec.13.3

Slide 41:

Problem with Maximum Likelihood

What if we have seen no training documents with the word fantastic and classified in the topic positive (thumbs-up)?

Zero probabilities cannot be conditioned away, no matter the other evidence!

Sec.13.3

Slide 42:

Laplace (add-1) smoothing for Naïve Bayes

Slide 43:

Multinomial Naïve Bayes: Learning

Calculate $P(c_j)$ terms

For each c_j in C do

docsj all docs with class = c_j

Calculate $P(w_k | c_j)$ terms

Textj single doc containing all docsj

For each word w_k in Vocabulary

n_k # of occurrences of w_k in Textj

From training corpus, extract Vocabulary

Slide 44:

Unknown words

What about unknown words

that appear in our test data

but not in our training data or vocabulary?

We ignore them

Remove them from the test document!

Pretend they weren't there!

Don't include any probability for them at all!

Why don't we build an unknown word model?

It doesn't help: knowing which class has more unknown words is not generally helpful!

Slide 45:

Stop words

Some systems ignore stop words

Stop words: very frequent words like the and a.

Sort the vocabulary by word frequency in training set

Call the top 10 or 50 words the stopword list.

Remove all stop words from both training and test sets

As if they were never there!

But removing stop words doesn't usually help

So in practice most NB algorithms use all words and don't use stopword lists

Slide 46:

Text Classification and Naive Bayes

Sentiment and Binary Naive Bayes

Slide 47:

Let's do a worked sentiment example!

Slide 48:

Step1:

1. Prior from training:

$$P(-) = 3/5$$

$$P(+) = 2/5$$

Slide 49:

Step 2 and 3

2. Drop "with"

3. Likelihoods from training:

Slide 50:

4. Scoring the test set:

Slide 51:

Solve the following

Slide 52:

Training Data

Slide 55:

Step 3: Word Frequency Table

Spam Class

D1 + D2 → buy cheap meds cheap meds online

Total words in Spam = 6

Ham Class

D3 + D4 → meeting schedule today project meeting today

Total words in Ham = 6

Slide 56:

Optimizing for sentiment analysis

For tasks like sentiment, word occurrence seems to be more important than word frequency.

The occurrence of the word fantastic tells us a lot

The fact that it occurs 5 times may not tell us much more.

Binary multinomial naive bayes, or binary NB

Clip our word counts at 1

Slide 57:

Binary Multinomial Naïve Bayes: Learning

Calculate $P(c_j)$ terms

For each c_j in C do

docsj all docs with class = c_j

From training corpus, extract Vocabulary

Calculate $P(w_k | c_j)$ terms

Remove duplicates in each doc:

For each word type w in docj

Retain only a single instance of w

Slide 58:

Binary Multinomial Naive Bayes on a test document d

58

First remove all duplicate words from d

Then compute NB using the same equation:

Slide 59:

Binary multinomial naive Bayes

Slide 60:

Sentiment Classification: Dealing with Negation

I really like this movie

I really don't like this movie

Negation changes the meaning of "like" to negative.

Negation can also change negative to positive-ish

Don't dismiss this film
Doesn't let us get bored

Slide 64:

Sentiment Classification: Dealing with Negation

Simple baseline method:

Add NOT_ to every word between negation and following punctuation:

didn't like this movie , but I

didn't NOT_like NOT_this NOT_movie but I

Das, Sanjiv and Mike Chen. 2001. Yahoo! for Amazon: Extracting market sentiment from stock message boards. In Proceedings of the Asia Pacific Finance Association Annual Conference (APFA).

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. EMNLP-2002, 79—86.

3.2 Naive Bayes Classifiers

Page 1:

Speech and Language Processing. Daniel Jurafsky James H. Martin. Copyright © 2026. All rights reserved. Draft of January 6, 2026. CHAPTER B Naive Bayes, Text Classification, and Sentiment Classification lies at the heart of both human and machine intelligence. Deciding what letter, word, or image has been presented to our senses, recognizing faces or voices, sorting mail, assigning grades to homeworks; these are all examples of assigning a category to an input. The potential challenges of this task are high

Page 2:

2 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT Finally, one of the oldest tasks in text classification is assigning a library subject category or topic label to a text. Deciding whether a research paper concerns epidemiology or instead, perhaps, embryology, is an important component of information retrieval. Various sets of subject categories exist, such as the MeSH (Medical Subject Headings) thesaurus. In fact, as we will see, subject category classification is the task for which the naive Bayes algorithm was invented in 1

Page 3:

B.1 • NAIVE BAYES CLASSIFIERS 3 how the features interact. The intuition of the classifier is shown in Fig. B.1. We represent a text document bag of words as if it were a bag of words, that is, an unordered set of words with their position ignored, keeping only their frequency. In the example in the figure, instead of representing the word order in all the phrases like “I love this movie”, “I would recommend it”, we simply note that the word I occurred 5 times in the entire excerpt, the word it 6 times, the words love, recommend, and

Page 4:

4 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT We can conveniently simplify Eq. B.3 by dropping the denominator $P(d)$. This is possible because we will be computing $P(d|c)P(c)$ for each possible class. But $P(d)$ doesn't change for each class, since the same document d , which must have the same probability $P(d)$. Thus, we can choose the class that maximizes this simpler formula: $c^* = \arg\max_c P(c|d) = \arg\max_c P(d|c)P(c)$ (B.4) We call Naive Bayes a generative model because we can read Eq. B.4 as

Page 5:

B.2 • TRAINING THE NAIVE BAYES CLASSIFIER 5 by walking an index through every word position in the document: $\text{positions} \leftarrow \text{all word positions in test document}$ (cid:89) $c_{NB} = \arg\max_c P(c) \prod_i P(w_i | c)$ (B.9) Naive Bayes calculations, like calculations for language modeling, are done in log space, to avoid underflow and in-

crease speed. Thus Eq. B.9 is generally instead expressed as (cid:88) $c \text{ NB} = \text{argmax}_c \log P(c) + \log P(w_i | c)$ (B.10) cC ipositions By considering features in logspace, Eq. B.10 computes

Page 6:

6 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

But since naive Bayes naively multiplies all the feature likelihoods together, zero probabilities in the likelihood term for any class will cause the probability of the class to be zero, no matter the other evidence! The simplest solution is the add-one (Laplace) smoothing introduced in Chapter 3. While Laplace smoothing is usually replaced by more sophisticated algorithms in language modeling, it is commonly used in naive Bayes text categorization

Page 7:

B.4 • OPTIMIZING FOR SENTIMENT ANALYSIS 7 function TRAIN_NAIVE_BAYES(D, C) returns V, log P(c), log P(w|c) for each class cC # Calculate P(c) terms N = number of documents in D doc Nc = number of documents from D in class c Nc log-prior[c] ← log N doc V ← vocabulary of D bigdoc[c] ← append(d) for d in D with class c for each word w in V # Calculate P(w|c) terms count(w, c) ← # of occurrences of w in bigdoc[c] count(w, c) + 1 loglikelihood[w, c] ← log (cid:80) (count(w (cid:48), c) + 1) w (cid:48) in V return logprior, loglikelihood, V function TEST_NAIVE_BAYES(testdoc,

Page 8:

8 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

First, for sentiment classification and a number of other text classification tasks, whether a word occurs or not seems to matter more than its frequency. Thus it often improves performance to clip the word counts in each document at 1 (see the end of the chapter for pointers to these results). This variant is called binary multinomial naive Bayes or binary naive Bayes. The variant uses the same algorithm as in Bayes Fig. B.2 except that for

Page 9:

B.5 • NAIVE BAYES FOR OTHER TEXT CLASSIFICATION TASKS 9 these negation words and the predicates they modify, but this simple baseline works quite well in practice. Finally, in some situations we might have insufficient labeled training data to train accurate naive Bayes classifiers using all words in the training set to estimate positive and negative sentiment. In such cases we can instead derive the positive sentiment and negative word features from sentiment lexicons, lists of words that are pre-lexicons annotated with positive

Page 10:

10 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

- Email subject line contains “online pharmaceutical”
- HTML has unbalanced “head” tags
- Claims you can be removed from the list

languageid For other tasks, like language id—determining what language a given piece of text is written in—the most effective naive Bayes features are not words at all, but character n-grams, 2-grams (‘zw’) 3-grams (‘nya’, ‘Vo’), or 4-grams (‘iez’, ‘thei’), or, even simpler, byte n-grams, where instead of using a character

Page 11:

B.7 • EVALUATION: PRECISION, RECALL, F-MEASURE 11 $P(\text{“I love this fun film”} | +) = 0.1 \times 0.1 \times 0.01 \times 0.05 \times 0.1 = 5 \times 10^{-7}$ $P(\text{“I love this fun film”} | -) = 0.2 \times 0.001 \times 0.01 \times 0.005 \times 0.1 = 1.0 \times 10^{-6}$ As it happens, the positive model assigns a higher probability to the sentence: $P(s | \text{pos}) > P(s | \text{neg})$. Note that this is just the likelihood part of the naive Bayes model; once we multiply in the prior a full naive Bayes model might well make a different classification decision. B.7 Evaluation: Precision, Recall, F-measure To introduce the method

Page 12:

12 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

gold standard labels gold positive gold negative system p s o y s s i t t e i m ve true
 positive false positive precision = $\frac{tp}{tp + fp}$ output system labels negative false
 negative true negative $\frac{tn}{tp + tn}$ recall = accuracy = $\frac{tp + tn}{tp + fp + tn + fn}$ Figure B.4
 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels. That's why instead of accuracy we generally turn to two other metrics shown in precision Fig. B.4: prec

Page 13:

B.7 • EVALUATION: PRECISION, RECALL, F-MEASURE 13 and hence F-measure is

$$F = \frac{2 \cdot P \cdot R}{P + R}$$
 (cid:18) 1 (cid:19) (2+1) PR F = or with 2 = F = (B.18) 1 + (1) 1 2P + R P
 R Harmonic mean is used because the harmonic mean of two values is closer
 to the minimum of the two values than the arithmetic mean is. Thus it weighs the lower of
 the two numbers more heavily, which is more conservative in this situation. B.7.1 Evalu-
 ating with more than two classes Up to now we have been describing text classification tasks with
 only two classes. But lots of classification ta

Page 14:

14 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT
 Class 1: Urgent Class 2: Normal Class 3: Spam Pooled true true true true true
 true true true urgent not normal not spam not yes no system system system sys-
 tem 8 11 60 55 200 33 268 99 urgent normal spam yes system 8 340 system 40 212
 system 51 83 system 99 635 not not not no 8 60 200 microaverage 268 precision =
 = .42 precision = .52 precision = .86 = .73 $\frac{8+11}{268+99}$ $\frac{60+55}{200+33}$ precision
 268+99 macroaverage $\frac{.42+.52+.86}{3} = .60$ precision

Page 15:

B.9 • STATISTICAL SIGNIFICANCE TESTING 15 Training Iterations Testing
 1 Dev Training 2 Dev Training 3 Dev Training 4 Dev Training Test 5 Training Dev
 Training Set 6 Training Dev 7 Training Dev 8 Training Dev 9 Training Dev 10
 Training Dev Figure B.7 10-fold cross-validation B.9 Statistical Significance Testing
 In building systems we often need to compare the performance of two systems. How can we
 know if the new system we just built is better than our old one? Or better
 than some other system described in the literature

Page 16:

16 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT
 we would encounter the value of $P(X=x|H_0)$ that we found, if we repeated the experiment p-value a
 great many times. We formalize this likelihood as the p-value: the probability, as-
 suming the null hypothesis H_0 is true, of seeing the (x) that we saw or one even 0 greater $P((X)(x)|H_0 \text{ is true})$ (B.21) So in our example, this p-value is the probability that we would see (x) assuming
 A is not better than B. If (x) is huge (let's say A has a very respectable F of .9 1
 and B has a terrible F of

Page 17:

B.9 • STATISTICAL SIGNIFICANCE TESTING 17 Considering text classifi-
 cation example with a test set of 10 documents. The first row of Fig. B.8 shows the re-
 sults of two classifiers (A and B) on this test set. Each document is labeled by one
 of the four possibilities (A and B both right, both wrong, A right and B wrong, A wrong
 and B right). A slash through a letter ((cid:19)B) means that that classifier got the an-
 swer wrong. On the first document both A and B get the correct class (AB), while on the second document
 it wrong (A(

Page 18:

18 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

$$p\text{-value}(x) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(x(i) > x)$$
 (x(i)) exceeds the expected value of (x) by (x) or more: b 1 (cid:88) (cid:16) (cid:17) p-
 value(x) = 1 (x(i))(x)(x) b i=1 b 1 (cid:88) (cid:16) (cid:17) = 1 (x(i))2(x) (B.22) b
 i=1 So in our example we have 10,000 test sets $x(i)$ and a threshold of .01, and in only 47 of the test-

sets do we find that A is accidentally better $(x(i) \geq 2(x))$, the resulting p-value of .0047 is smaller than .01, indicating sufficiently surprising and unlikely to have happened

Page 19:

B.11 • SUMMARY 19 toxicity icity detection is the task of detecting hate speech, abuse, harassment, or other detection kinds of toxic language. While the goal of such classifiers is to help reduce societal harm, toxicity classifiers can themselves cause harms. For example, researchers have shown that some widely used toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like women (Park et al., 2018), blind people (Hutchinson et al., 2020) or gay people (Dixon et al.,

Page 20:

20 APPENDIX B • NAIVE BAYES, TEXT CLASSIFICATION, AND SENTIMENT

- Statistical significance tests should be used to determine whether we can be confident that one version of a classifier is better than another.
- Designers of classifiers should carefully consider harms that may be caused by the model, including its training data and other components, and report model characteristics in a model card.

Historical Notes Multinomial naive Bayes text classification was proposed by Maron (1961) at the RAND Corporation for the

Page 21:

EXERCISES 21 2014, Dorr et al. 2017). Feature selection is a method of removing features that are unlikely to generalize well. Features are generally ranked by how informative they are about information for decision. A very common metric, information gain, tells us how many bits of gain in information the presence of the word gives us for guessing the class. Other feature selection metrics include 2, pointwise mutual information, and GINI index; see Yang and Pedersen (1997) for a comparison and Guyon and Elisseeff (2003) for an introduction to

Page 22:

22 Appendix B • Naive Bayes, Text Classification, and Sentiment Aggarwal, C. C. and C. Zhai. 2012. A survey of text classification algorithms. In C. C. Aggarwal and C. Zhai, eds, 1998. A Bayesian approach to filtering junk email. AAAI- Mining text data, 163–222. Springer. 98 Workshop on Learning for Text Categorization. Bayes, T. 1763. An Essay Toward Solving a Problem in the Hu, M. and B. Liu. 2004. Mining a Doctrine of Chances, volume 53. Reprinted in Facsimiles reviews. KDD. of Two Papers by

Page 23:

Exercises 23 Pang, B., L. Lee, and S. Vaithyanathan. 2002. Thumbs up? Sentiment classification techniques. EMNLP. Park, J. H., J. Shin, and P. Fung. 2018. Reducing gender bias in abusive language detection. EMNLP. Pennebaker, J. W., R. J. Booth, and M. E. Francis. 2007. Linguistic Inquiry and Word Count: LIWC 2007. Austin, TX. Popp, D., R. A. Donovan, M. Crawford, K. I. M. Peele. 2003. Gender, race, and speech style stereotypes. Sex Roles, 48(7-8):317–325. Sahami, M., S. T. Dumais, D. Heckerman, and E. Horvitz. 199

4 Module 4: Recurrent Neural Networks

4.1 1. Chapter 4

Slide 1:

Deep Learning Architectures
for Sequence Processing

Slide 2:

Recap of Feed Forward Neural Networks

Inputs → pass through hidden layers → output

Each layer has weights + activation functions

Limitation: FFNNs assume inputs are independent (no memory of past inputs).

Works well for images, tabular data, but not for sequences.

Example to give students:

Predicting tomorrow's weather using only today's temperature (FFNN).

But in reality, weather depends on past few days → we need memory.

Slide 3:

Introduce the Idea of Sequences

Some problems need context:

Sentences in language (words depend on previous words).

Stock price prediction (depends on past days).

Speech recognition (next sound depends on earlier sounds).

“If I say ‘The cat sat on the _____’, what word do you expect?” They’ll naturally answer ‘mat’. That shows context matters.

Slide 4:

Recurrent Neural Networks

Works well with sequence of data for example Natural language data, Time series data, sales forecasting, stock forecasting .

Problems in NLP using Machine Learning

For example, SPAM Classifier

NLP Text Preprocessing technique used to convert text data to vector are Bag of Words ,TFIDF,wprd2Vec

Steps followed for Classification

Step 1 Covert the text into vector

Step 2 Apply machine learning algorithm(naïve Bayes)

Problem : -While converting text to vector sequence information is discarded

If the sequence information is decarded Accuracy of the result may get affected.

In many NLP applications sequence information is very important..

And for such applications RNN is important concept

Slide 5:

RNN

where the output from the previous step is fed as input to the current step

when in language models, it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.

main and most important feature of RNN is its Hidden state which remembers some information about a sequence.

The state is also referred to as Memory State since it remembers the previous input to the network.

It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output.

Slide 6:

Architecture of RNN

where:

ht -> current state

ht-1 -> previous state

xt -> input state

Slide 7:

Simple recurrent neural network illustrated as a feed-forward network

Most significant change: new set of weights, U

connect the hidden layer from the previous time step to the current hidden layer.

determine how the network should make use of past context in calculating the output for the current input.

INPUT

OUTPUT

WEIGHT MATRIX

WEIGHT MATRIX

HIDDEN LAYER

PREVIOUS TIME STEP

weights from the hiddenlayer to the output layer

Slide 8:

RNNs: Passing hidden state to next time step

While processing, it passes the previous hidden state to the next step of the sequence.

The hidden state acts as the memory.

It holds information on previous data the network has seen before.

Slide 9:

Simple-RNN abstraction

y2

y1

y3

Slide 10:

RNNs: Understanding a single cell

Slide 11:

Unrolled RNN

Slide 12:

Training

we'll use a training set, a loss function, and back-propagation

to obtain the gradients needed to adjust the weights in these recurrent networks.

3 sets of weights to update while training :

W , the weights from the input layer to the hidden layer,

U, the weights from the previous hidden layer to the current hidden layer, and finally

V , the weights from the hidden layer to the output layer.

Slide 13:

Steps in Training

to compute the loss function for the output at time t we need the hidden layer from time t - 1.

the hidden layer at time t influences both the output at time t and the hidden layer at time t + 1 (and hence the output and loss at t + 1).

to assess the error accruing to h_t , we'll need to know its influence on both the current output as well as the ones that follow.

Slide 14:

Backpropagation algorithm – two pass algorithm for training the weights in RNNs.

Step 1:- we perform forward inference, computing h_t , y_t , accumulating the loss at each step-in time, saving the value of the hidden layer at each step for use at the next time step.

Step 2:- we process the sequence in reverse, computing the required gradients as we go, computing and saving the error term for use in the hidden layer for each step backward in time.

This general approach is commonly referred to as Backpropagation Through Time

Slide 15:

Language Modeling

Language Modelling (LM) is the task of predicting what word comes next or more generally, a system that assigns probability to a piece of a text sequence.

N-gram is the simplest language model, and its performance is limited by its lack of complexity.

Simplistic models like this one cannot achieve fluency, enough language variation and correct writing style for long texts.

For these reasons, neural networks (NN) are explored as the new gold standard despite their complexity and Recurrent Neural Networks (RNN) became a fundamental architecture for sequences of any kind.

Slide 16:

RNNs as Language Models

Processes sequences one word at a time

Predicts the next word in a sequence

Uses: Current word Previous hidden state

Hidden state captures information from all previous words

Overcomes limited context problem of N-gram models maintains long-term context from the beginning of the

Slide 17:

RNN as language Model

Input sequence x consists of word embeddings (one-hot vectors) of size $|V| \times 1$

Output y is a probability distribution over the vocabulary

At each step:

Retrieve word embedding using embedding matrix (E)

Combine with previous hidden state

Compute new hidden state

Hidden state is used to generate the output layer

Output is passed through Softmax function

Produces probabilities for all words in the vocabulary

Slide 18:

Training RNNs as language models.

Slide 19:

Generation with RNN-Based Language Models

Generate random sentences using a trained language model to understand its behavior

Start with beginning-of-sentence token ($<s>$) Sample first word from Softmax output distribution

Feed the sampled word's embedding as input for the next step

Repeat the process:

Generate next word Sample from probability distribution

Continue until:

End-of-sentence token ($</s>$) is generated

A fixed length limit is reached

Slide 20:

RNN as a Language Model

Steps for using RNN as Language Model

Consider the following sentences

"The cat sat on the mat."

Let's assume our vocabulary consists of 5 unique words:

The, cat, sat, on, mat

We will manually compute:

Word Representation

Forward Pass in RNN

Softmax Output Calculation

Loss Calculation (Cross-Entropy)

Backpropagation (BPTT) Calculation

Slide 21:

Step 1: Assign Numerical Representation

We convert words into one-hot vectors:

Slide 22:

Our training data consists of input-output pairs:

Slide 23:

Step 2: Forward Pass in RNN

An RNN updates its hidden state h_t using the formula:

where:

h_t = hidden state at time t

h_{t-1} = previous hidden state

x_t = input one-hot vector

W_h = hidden state weight matrix (random initialization)

W_x = input weight matrix (random initialization)

b_h = bias term

$\tanh()$ = activation function

Let's assume:

Hidden state size = 3

Initial hidden state: $h_0 = [0, 0, 0]$

Random weight matrices:

Slide 24:

Step 2.1: Compute h_1 for Input "The"

Using the one-hot vector for "The":

$x_1 = [1, 0, 0, 0, 0]$

Hidden state calculation:

$h_1 = \tanh(W_h * h_0 + W_x * x_1 + b_h)$

Slide 25:

Step 3: Compute Softmax Output

The output layer computes:

$y_t = \text{softmax}(W_o * h_t + b_o)$

where:

W_o = output weight matrix

b_o = bias

Slide 26:

Step 4: Compute Cross-Entropy Loss

Given correct output "cat" (index = 1):

$L = -\log P(\text{"cat"})$

$L = -\log(0.206) = 1.58L$

Step 5: Backpropagation Through Time (BPTT)

Gradient calculations are computed for W_x, W_h, W_o using the chain rule.

Slide 27:

Step 1: Assign Word Embeddings

Let's assume we use a 3-dimensional embedding for each word:

So instead of using one-hot vectors, we directly pass the embedding vector to the RNN.

Slide 35:

Autoregressive Generation

The technique of the generation since the word generated at each autoregressive generation time step is conditioned on the word selected by the network from the previous step.

Slide 36:

What is Autoregressive RNN?

Autoregressive RNN is a sequence generation model where the next word is predicted based on the previously generated words.

The key characteristic is that the model uses its own generated outputs as inputs for the next step.

It is commonly used in text generation, language modeling, and machine translation.

in actual generation (inference, real time use), the model predicts words one by one without knowing the ground truth at each step, so per-step loss is not useful.

Slide 37:

Step-by-Step Numerical Example

Problem Statement

Let's say we train an RNN language model on the sentence:

"I love deep learning"

We want to use autoregressive RNN to generate text starting from the seed word "I".

Slide 38:

Step 2: Initialize RNN Parameters

We assume a simple Recurrent Neural Network (RNN) with the following parameters:

Hidden state size = 3

Weight matrices:

Input-to-hidden weights: W_x (3×3 matrix)

Hidden-to-hidden weights: W_h (3×3 matrix)

Hidden-to-output weights: W_y (3×3 matrix)

Biases: b_h, b_y

Slide 39:

Step 3: Compute Hidden State for First Word ("I")

Hidden State Equation

$$h_t = \tanh(W_x * x_t + W_h * h_{t-1} + b_h)$$

Initial hidden state (h_0) = $[0, 0, 0]$

For word "I" ($x_1 = [0.1, 0.3, 0.5]$)

Slide 41:

Step 5: Continue the Process

The model feeds "love" as input into the RNN.

It generates "deep" next.

Then "learning".

The process continues until a stopping condition is met.

Slide 42:

Conclusion

Autoregressive RNNs generate text one word at a time.

The hidden state captures context from previous words.

The softmax layer selects the most probable next word.

The model recursively feeds predictions back as input.

Slide 43:

Sequence Labeling

In sequence labeling, the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence.

Canonical examples of sequence labeling include part-of-speech tagging and named entity recognition

In an RNN approach to sequence labeling, inputs are word embeddings and the outputs are tag probabilities generated by a softmax layer over the given tag set,

Slide 44:

Part-of-speech tagging as sequence labeling with a simple RNN.

The inputs at each time step are pre-trained word embeddings corresponding to the input tokens.

The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared U, V and W weight matrices that comprise the network.

The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer.

Slide 45:

RNNs for Sequence Classification

RNNs is to classify entire sequences rather than the tokens within them

document-level topic classification, spam detection, message routing for customer service applications, and deception detection

To apply RNNs in this setting, the text to be classified is passed through the RNN a word at a time generating a new hidden layer at each time step.

The hidden layer for the final element of the text, h_n , is taken to constitute a compressed representation of the entire sequence. In the simplest approach to classification, h_n , serves as the input to a subsequent feedforward network that chooses a class via a SoftMax over the possible classes

Slide 46:

there are no intermediate outputs for the words in the sequence preceding the last element.

there are no loss terms associated with those elements.

Instead, the loss function used to train the weights in the network is based entirely on the final text classification task.

Specifically, the output from the SoftMax output from the feedforward classifier together with a cross-entropy loss drives the training.

The error signal from the classification is backpropagated all the way through the weights in the feedforward classifier through, to its input, and then through to the three sets of weights in the RNN

Slide 47:

Performing Sentiment analysis using RNN

Sentiment analysis using Recurrent Neural Networks (RNNs) involves processing textual data to classify whether the sentiment of a sentence is positive, negative, or neutral.

Slide 48:

Step 1: Understanding the Sentiment Analysis Task

Objective:

To classify given text (e.g., movie reviews, tweets) into sentiment categories such as:

Positive sentiment (e.g., "I love this movie")

Negative sentiment (e.g., "I hate this film")

Slide 49:

Step 2: Preprocessing the Text Data

Before feeding text into an RNN model, we must convert it into numerical form.

Tokenization

Definition: Splitting a sentence into words (tokens).

Example:

Input: "This movie is great"

Output tokens: ["this", "movie", "is", "great"]

Slide 50:

Convert Words into Word Embeddings

Instead of one-hot encoding, we use word embeddings (like Word2Vec, GloVe, or trainable embeddings) that convert words into dense vector representations.

Example:

So, the sentence "This movie is great" becomes a matrix of word embeddings.

Slide 51:

Step 3: Define RNN Model

RNN processes sequences one word at a time and maintains a hidden state h_t .

3.1 Forward Pass in RNN

For each word, the hidden state is updated using:

$$h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b_h)$$

where:

x_t = Word embedding of the current word.

h_t = Hidden state at time t .

W_x, W_h, b_h = Learnable parameters.

Slide 52:

Each word in a sentence updates the hidden state as:

Time Step 1: "This"

$$h_1 = \tanh(W_x * x_1 + b_h)$$

Time Step 2: "movie"

$$h_2 = \tanh(W_h * h_1 + W_x * x_2 + b_h)$$

Time Step 3: "is"

$$h_3 = \tanh(W_h * h_2 + W_x * x_3 + b_h)$$

Time Step 4: "great"

$$h_4 = \tanh(W_h * h_3 + W_x * x_4 + b_h)$$

At the final time step, the hidden state h_T represents the entire sentence.

Slide 53:

Step 4: Sentiment Prediction

The final hidden state h_T is passed through a fully connected layer and a sigmoid activation function to get the final sentiment score:

$$y = (W_y * h_T + b_y)$$

where:

W_y, b_y = Learnable parameters.

(Sigmoid function converts the output to a probability between 0 and 1).

If $y > 0.5 \rightarrow$ Positive Sentiment

If $y < 0.5 \rightarrow$ Negative Sentiment

Slide 54:

Step 5: Loss Function Backpropagation

To train the model, we use Binary Cross-Entropy Loss:

where:

Y_{true} = Actual sentiment label (1 for positive, 0 for negative).

y = Predicted probability.

Gradients are computed using Backpropagation Through Time (BPTT), and parameters are updated using Gradient Descent:

where (learning rate) is a small value like 0.01.

Slide 55:

Step 6: Model Training

Train on a dataset of labeled reviews (e.g., IMDB reviews dataset).

Adjust hyperparameters like learning rate, embedding size, and number of RNN units.

Use dropout to prevent overfitting.

Slide 56:

Step 7: Model Evaluation

Evaluate performance using accuracy, precision, recall, and F1-score.

Test with new sentences like:

"I love this film" → Predicted Sentiment: Positive (1)

"This movie is bad" → Predicted Sentiment: Negative (0)

Slide 57:

Step 1: Understanding Sentiment Analysis with RNN

Objective: Predict sentiment (e.g., positive/negative) from a given text.

Input: Text data (reviews, tweets, etc.).

Output: Sentiment score (e.g., binary classification: positive = 1, negative = 0).

Example:

Input: "I love this movie"

Output: Positive sentiment (1)

Slide 58:

Step 2: Preprocessing the Data

Before feeding text into an RNN model, we must convert it into numerical form.

2.1 Tokenization

Tokenization splits the text into individual words: Sentence: "This movie is great" Tokens: ["this", "movie", "is", "great"]

2.2 Word Embeddings (Instead of One-Hot Encoding)

Each word is mapped to a fixed-size vector using pretrained embeddings (like Word2Vec, GloVe) or a trainable embedding layer.

Slide 60:

Step 3: Forward Pass in RNN

An RNN processes each word sequentially and updates a hidden state.

$h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b_h)$

where:

x_t = Word embedding at time t .

h_t = Hidden state at time t .

W_x, W_h, b_h = Learnable parameters.

Let's initialize:

$h_0 = [0, 0, 0]$

Slide 65:

Stacked RNN

Stacked recurrent networks.

The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output

The optimal number of stacked RNNs is specific to each application and to each training set.

However, as the number of stacks is increased the training costs rise quickly.

Slide 66:

RNNs: Vanishing/exploding gradients

Product of so many derivatives of tanh functions (<1) will lead to exponentially small values ($\rightarrow 0$). Vanishing Gradient problem

Product of so many derivatives of tanh functions (>1) will lead to exponentially large values. Exploding Gradient problem

Exploding Gradient problem can be simply solved by clipping the gradients

However, vanishing gradients is a complex issue and disallow capturing long-range dependencies

Proper initialization of the W matrix can reduce the effect of vanishing gradients

Use ReLU (derivative either 0 or 1) instead of sigmoid or tanh activation functions

Use Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU) architectures

Slide 67:

Managing Context in RNNs: LSTMs and GRUs

to make use of information distant from the current point of processing is difficult while training RNN.

the information encoded in hidden states tends to be local, more relevant to the most recent parts of the input sequence and recent decisions.

Slide 68:

Long short-term memory (LSTM)

A type of RNN

The differences are the operations within the LSTM's cells

What to keep and what to forget?

Slide 69:

LSTM: The main idea

The cell state and various gates

The cell state act as a transport highway that transfers relative information all the way down the sequence chain.

The gates decide which information is allowed on the cell state

Gates contains sigmoid activations

Forget info: if sigmoid activation $\rightarrow 0$

Keep info: if sigmoid activation $\rightarrow 1$

Slide 70:

Consider the following example

step by step and calculation of all intermediate values for an LSTM cell when processing the input sequence "Movie is good" using given weights and biases.

Slide 71:

Consider the following example

step by step and calculation of all intermediate values for an LSTM cell when processing the input sequence "Movie is good" using given weights and biases.

Slide 72:

Step 2: LSTM Forward Pass for "Movie"

We compute all gate activations for time step $t = 1$.

Slide 73:

LSTM: Forget gate

This gate decides what information should be thrown away or kept.

Information from previous hidden state and the current input is passed through the sigmoid function.

Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

Slide 74:

LSTM: Input gate

First, the previous hidden state and current input is passed into a sigmoid function

Also pass the hidden state and current input into the tanh function to squish values between -1 and 1.

Multiply the tanh output with the sigmoid output

Sigmoid output will decide which information is important to keep from the tanh output

Slide 76:

LSTM: Cell state

The cell state gets pointwise multiplied by the forget vector.

Take the output from the input gate and do a pointwise addition which updates the cell state to new values

Slide 77:

LSTM: Output gate

The output gate decides what the next hidden state should be

First, we pass the previous hidden state and the current input into a sigmoid function.

Then we pass the newly modified cell state to the tanh function.

Multiply the tanh output with the sigmoid output

Slide 78:

Example

Traditional RNN Fails to predict because of short term memory

Slide 79:

Example

Slide 80:

Example

Slide 81:

LSTM: The operations

Forget gate

Slide 82:

LSTM: The operations

Input gate

Slide 83:

LSTM: Updating weights

Slide 84:

LSTM: The operations

Output gate

Slide 85:

LSTM: Updating weights

Slide 86:

LSTM: Updating weights

Slide 87:

LSTM: Updating weights

Gradient behaves similar to the forget gate

Slide 88:

LSTM: Updating weights

This can be controlled initially using +ve bias term!

Slide 89:

Gated Recurrent Unit (GRU)

New generation RNNs

GRU's got rid of the cell state and used the hidden state to transfer information

Little speedier to train than LSTMs

But not a clear winner

Update Gate: decides what information to throw away and what new information to add

Reset Gate: decides how much past information to forget.

Slide 90:

GRU Mathematical Explanation to the Diagram

A GRU cell consists of:

Reset Gate (r_t , red sigmoid in the left part of the diagram)

Update Gate (z_t , red sigmoid in the center)

Candidate Hidden State ($\tilde{h}_t$)

New Hidden State (h_t , final output)

Slide 92:

Purpose of the Reset Gate

The reset gate (r_t) determines how much of the past hidden state (h_{t-1}) should be forgotten before computing the new candidate hidden state ($\tilde{h}_t$).

If r_t is close to 0, the past hidden state is ignored, meaning the GRU relies mostly on the current input.

If r_t is close to 1, the past hidden state is preserved and influences the new candidate hidden state.

Slide 93:

For example, if $r_t =$

The first value 0.42 means that for the first hidden unit, about 42

The second value 0.55 means that for the second hidden unit, 55

Slide 95:

Purpose of the Candidate Hidden State

The candidate hidden state ($\tilde{h}_t$) represents the new memory that could be used to update the final hidden state.

It is computed based on the current input x_t and a modified version of the previous hidden state h_{t-1} , which is influenced by the reset gate r_t .

If r_t is close to 0, the past hidden state is largely ignored. If r_t is close to 1, the past hidden state has more influence.

Slide 98:

Understanding $1-z_t$ and z_t

The update gate z_t plays a critical role in determining how much of the previous hidden state h_{t-1} should be kept and how much of the newly computed candidate hidden state $\tilde{h}_t$ should be used.

The final hidden state h_t is computed as:

This equation is a weighted sum where:

z_t controls how much of $\tilde{h}_t$ (the new memory) is retained.

$1-z_t$ controls how much of h_{t-1} (the old memory) is kept.

Slide 100:

Interpretation of $1-z_t$ and z_t

$1-z_t$ acts as a forgetting factor for the old state h_{t-1} .

If z_t is small, then $1-z_t$ is large, meaning more of the old state is preserved.

If z_t is large, then $1-z_t =$ is small, meaning more of the old state is discarded.

zt determines how much of h t (new memory) should be incorporated into the hidden state.

If zt is close to 1, the new memory is fully used, making the network update its knowledge rapidly.

If zt is close to 0, the network relies mostly on past memory, preventing unnecessary updates.

This mechanism makes the GRU effective at handling long-term dependencies in sequential data.

Slide 101:

Example

Slide 102:

GRU

Slide 103:

Example 2

Slide 104:

Different types of RNNs

Slide 105:

one-to-one: Binary Classification [single images/words → single output]

one-to-many: Image Captioning [image → Sequence of words](variability sized)

many-to-one: Sentiment Classification [sequence of words (variable sized) → single sentiment]

many-to-many (non-matching): Machine Translation [sequence of words → sequence of words], where both sequences have variable length and these lengths may be difference

many-to-many (matching): Video classification on a...

4.2 2. Bidirectional RNN

Slide 1:

Bi-directional Recurrent Neural Network

In order for the network to use information from both the past and future context in its predictions, BRNNs process input sequences in both the forward and backward directions.

This is the main distinction between BRNNs and conventional recurrent neural networks.

Slide 2:

BRNN

A BRNN has two distinct recurrent hidden layers, one of which processes the input sequence forward and the other of which processes it backward.

After that, the results from these hidden layers are collected and input into a prediction-making final layer.

Any recurrent neural network cell, such as Long Short-Term Memory (LSTM) or Gated Recurrent Unit, can be used to create the recurrent hidden layers.

Slide 3:

BRNN functions

The BRNN functions similarly to conventional recurrent neural networks in the forward direction, updating the hidden state depending on the current input and the prior hidden state at each time step.

The backward hidden layer, on the other hand, analyses the input sequence in the opposite manner, updating the hidden state based on the current input and the hidden state of the next time step.

Compared to conventional unidirectional recurrent neural networks, the accuracy of the BRNN is improved since it can process information in both directions and account for both past and future contexts.

Because the two hidden layers can complement one another and give the final prediction layer more data, using two distinct hidden layers also offers a type of model regularization.

Slide 4:

Gradient Updating

In order to update the model parameters, the gradients are computed for both the forward and backward passes of the backpropagation through the time technique that is typically used to train BRNNs.

The input sequence is processed by the BRNN in a single forward pass at inference time, and predictions are made based on the combined outputs of the two hidden layers.

Slide 6:

Working of Bidirectional Recurrent Neural Network

Inputting a sequence: A sequence of data points, each represented as a vector with the same dimensionality, are fed into a BRNN. The sequence might have different lengths.

Dual Processing: Both the forward and backward directions are used to process the data. Based on the input at that step and the hidden state at step $t-1$, the hidden state at time step t is determined in the forward direction. The input at step t and the hidden state at step $t+1$ are used to calculate the hidden state at step t in a reverse way.

Computing the hidden state: A non-linear activation function on the weighted sum of the input and previous hidden state is used to calculate the hidden state at each step. This creates a memory mechanism that enables the network to remember data from earlier steps in the process.

Determining the output: A non-linear activation function is used to determine the output at each step from the weighted sum of the hidden state and a few output weights. This output has two options: it can be the final output or input for another layer in the network.

Training: The network is trained through a supervised learning approach where the goal is to minimize the discrepancy between the predicted output and the actual output. The network adjusts its weights in the input-to-hidden and hidden-to-output connections during training through backpropagation.

Slide 7:

To calculate the output from an RNN unit, we use the following formula:

Slide 8:

The training of a BRNN is like backpropagation through a time algorithm. BPTT algorithm works as follows:

Slide 9:

Applications of Bidirectional Recurrent Neural Network

Bi-RNNs have been applied to various natural language processing (NLP) tasks, including:

Sentiment Analysis: By considering both the prior and subsequent context, BRNNs can be utilized to categorize the sentiment of a particular sentence.

Named Entity Recognition: By considering the context both before and after the stated thing, BRNNs can be utilized to identify those entities in a sentence.

Part-of-Speech Tagging: The classification of words in a phrase into their corresponding parts of speech, such as nouns, verbs, adjectives, etc., can be done using

BRNNs.

Machine Translation: BRNNs can be used in encoder-decoder models for machine translation, where the decoder creates the target sentence, and the encoder analyses the source sentence in both directions to capture its context.

Speech Recognition: When the input voice signal is processed in both directions to capture the contextual information, BRNNs can be used in automatic speech recognition systems.

Slide 10:

Advantages of Bidirectional RNN

Context from both past and future: With the ability to process sequential input both forward and backward, BRNNs provide a thorough grasp of the full context of a sequence. Because of this, BRNNs are effective at tasks like sentiment analysis and speech recognition.

Enhanced accuracy: BRNNs frequently yield more precise answers since they take both historical and upcoming data into account.

Efficient handling of variable-length sequences: When compared to conventional RNNs, which require padding to have a constant length, BRNNs are better equipped to handle variable-length sequences.

Resilience to noise and irrelevant information: BRNNs may be resistant to noise and irrelevant data that are present in the data. This is so because both the forward and backward paths offer useful information that supports the predictions made by the network.

Ability to handle sequential dependencies: BRNNs can capture long-term links between sequence pieces, making them extremely adept at handling complicated sequential dependencies.

Slide 11:

Disadvantages of Bidirectional RNN

Computational complexity: Given that they analyze data both forward and backward, BRNNs can be computationally expensive due to the increased amount of calculations needed.

Long training time: BRNNs can also take a while to train because there are many parameters to optimize, especially when using huge datasets.

Difficulty in parallelization: Due to the requirement for sequential processing in both the forward and backward directions, BRNNs can be challenging to parallelize.

Overfitting: BRNNs are prone to overfitting since they include many parameters that might result in too complicated models, especially when trained on short datasets.

Interpretability: Due to the processing of data in both forward and backward directions, BRNNs can be tricky to interpret since it can be difficult to comprehend what the model is doing and how it is producing predictions.

4.3 Gated Recurrent Unit (3)

Slide 1:

Gated Recurrent Unit

Slide 8:

Element Wise Multiplication

Slide 10:

The Basic Idea

It uses gating mechanisms to selectively update the hidden state of the network at each time step.

The gating mechanisms are used to control the flow of information in and out of the network.

The GRU has two gating mechanisms, called the reset gate and the update gate.

The reset gate determines how much of the previous hidden state should be forgotten, while the update gate determines how much of the new input should be used to update the hidden state.

The output of the GRU is calculated based on the updated hidden state.

Slide 11:

The equations used to calculate the reset gate, update gate, and hidden state of a GRU are as follows:

where W_r , W_z , and W_h are learnable weight matrices,

x_t is the input at time step t ,

h_{t-1} is the previous hidden state,

and h_t is the current hidden state.

Slide 12:

Collective information passed onto the next Gated Recurrent Unit

The different gates of a GRU are as described below: -

Update Gate(z):

It determines how much of the past knowledge needs to be passed along into the future.

It is analogous to the Output Gate in an LSTM recurrent unit.

Reset Gate(r):

It determines how much of the past knowledge to forget.

It is analogous to the combination of the Input Gate and the Forget Gate in an LSTM recurrent unit.

Current Memory Gate($\tilde{h}$):

It is often overlooked during a typical discussion on Gated Recurrent Unit Network. It is incorporated into the Reset Gate just like the Input Modulation Gate is a sub-part of the Input Gate and is used to introduce some non-linearity into the input and to also make the input Zero-mean. Another reason to make it a sub-part of the Reset gate is to reduce the effect that previous information has on the current information that is being passed into the future.

Slide 13:

Working of a Gated Recurrent Unit:

Take input the current input and the previous hidden state as vectors.

Calculate the values of the three different gates by following the steps given below:-

For each gate, calculate the parameterized current input and previously hidden state vectors by performing element-wise multiplication (Hadamard Product) between the concerned vector and the respective weights for each gate.

Apply the respective activation function for each gate element-wise on the parameterized vectors. Below given is the list of the gates with the activation function to be applied for the gate.

Slide 14:

Step 3

The process of calculating the Current Memory Gate is a little different.

First, the Hadamard product of the Reset Gate and the previously hidden state vector is calculated. Then this vector is parameterized and then added to the parameterized current input vector.

Slide 15:

Step 4

To calculate the current hidden state, first, a vector of ones and the same dimensions as that of the input is defined.

This vector will be called ones and mathematically be denoted by 1.

First, calculate the Hadamard Product of the update gate and the previously hidden state vector.

Then generate a new vector by subtracting the update gate from ones and then calculate the Hadamard Product of the newly generated vector with the current memory gate.

Finally, add the two vectors to get the currently hidden state vector.

Slide 16:

Training

the training process for a GRU network is also diagrammatically like that of a basic Recurrent Neural Network and differs only in the internal working of each recurrent unit.

The Back-Propagation Through Time Algorithm for a Gated Recurrent Unit Network is like that of a Long Short Term Memory Network and differs only in the differential chain formation.

5 Module 5: Morphological and Syntactic Analysis

5.1 $1_{Morphologicalanalysis}(1)(2)$

Slide 1:

Morphology

Morphology is the study of words

Study's internal structure of words

How words change their form to generate new word

Different role they play in sentence ,strictly following linguistic rule.

Slide 2:

Morphemes

words are built from smaller meaningful grammatical units called morphemes.

dogs

2 morphemes, 'dog' and 's'

's' is a plural marker on nouns

Slide 3:

Morpheme

A Smallest unit carries meaning

While doing analysis of a word, we split the given word into parts.

Each part is called a morpheme.

Eg.:

uneducated $\rightarrow$

un + educate + d.

it has three morphemes.

Slide 4:

Free Morpheme

Can appear as a word by itself; often can combine with other morphemes too.

house (house-s), walk (walk-ed), of, the, or

Independent/can stand by themselves as single words.

When combined with bound morphemes

free morphemes are called stems/root.

Slide 5:

Lexical Morpheme

Visualize/Content word

Nouns –Glass

Adjectives -Black,

Verbs-Dance,

Adverb - beautifully.

‘open’ class of morphemes because we can add new words to the language easily

Eg.(girl, play, google, e-mail, blog).

Slide 6:

Grammatical Morpheme

Also called as Functional morphemes

consist of functional/Grammer words

Conjunctions- And/Or

Prepositions-On/Under/At

Articles-An/The

Pronouns- She/He.

Limited words

This is a ‘closed’ class of morphemes

Slide 7:

Bound Morpheme

Cannot appear as a word by itself.

-s (dog-s), -ly (quick-ly), -ed (walk-ed)

cannot stand alone

Not complete in themselves

Depends on free Morpheme

Affixes are bound morpheme

typically attached to free morpheme.

They can prefixes, Infix and suffixes (re-, un-, dis-, pre-, -ness, -less, -ly).

Slide 8:

Prefix

Attached before lexical morpheme

Be-come

Un-happy

Slide 9:

Infix

unladylike

3 morphemes

un- ‘not’

lady ‘well-behaved woman’

-like ‘having the characteristic of’

Slide 10:

Suffix

Words attached after root

Teach-er

Use-less

Slide 11:

Suffix

8 Suffixes

Noun

Plural – Book->Books

Possessional- Ram’s

Adjective

er – Tall->Taller

est- Tall->Tallest

Verb

S= Play->Plays

ed= Play->Played

Ing= Play->Playing

en= Brake->Broken

Slide 12:

Derivational Morpheme

make new words of a different grammatical category from a stem

Class Changing

Care careful

(Verb) (Adjective)

Care carelessly

(Verb) (Adverb)

Class Maintaining

Child Childhood

(Noun) (Noun)

Slide 13:

Inflectional Morpheme

Morpheme when attached to root doesn't change its class

There are eight inflectional morphemes in English.

They are all suffixes.

Slide 14:

Regular Inflectional Morpheme

Refers to those inflections which follows a standard pattern

Eg.

Dance+ed->Danced

Walk+ed->Walked

Boy+s->Boys

Slide 15:

Irregular Inflectional Morpheme

Refers to those inflections which does not follow a standard pattern

Eg.

Wife+s - > Wives

Child+s -> Children

Mouse+s -> Mice

Slide 16:

Completely change the morpheme

Occurrence of phonemically unrelated morpheme

Eg.

Go+ed->Went

Good+er->Better

am+ed->was

Suppletion Inflectional Morpheme

Slide 17:

Identify Inflectional or Derivational Morphology

I jumped into the puddle this morning

This is unbelievable

I have john's umbrella

Emma goes to school

She is working carelessly

Slide 18:

Identify Inflectional or Derivational Morphology

I jumped into the puddle this morning

This is unbelievable

I have john's umbrella

Emma goes to school

She is working carelessly

Inflectional Morphology

Derivational Morphology

Inflectional Morphology

Inflectional Morphology

Derivational Morphology

Slide 20:

Regular Expressions

Slide 21:

Some Questions on Regular Expression.....

Slide 22:

What is Regular Expression?

What does the command `ab+c` search for?

`ac,abc,abbc`, and so on

`ab,abc,abcc` and so on

`abc,abbc,abbbc` and so on

None of the above

The number of matches does the command `a1,3` give with the string `aabbaaaa`?

3

2

1

4

Slide 23:

What is the output of the below code?

```
print(re.match('On','one'))
```

'e'

Match Object

'n'

None

What does the sequence finds the match?

Decimal digits

Non-decimal digits

Division

None of the above

Which of the following command is used to search a match for 1,2,3,4?

`[1-4]`

`(1-3)`

`[1234]`

Both a and c

Slide 24:

What is the output of the below code?

```
re.sub('a','u','aeiou!')
```

'ueiou!'

'eiou!'

'eio!'

None of the above

Slide 26:

What is Regular Expression?

A Formal Language for specifying text search strings

A regular expression is an algebraic notation for characterizing a set of strings.

Regular expression search function will search through the corpus, returning all texts that match the pattern or the first match.

The corpus can be a single document or a collection

Slide 27:

Regular expressions

The simplest kind of regular expression is sequence of simple characters

A formal language for specifying text strings

How can we search for any of these?

woodchuck

woodchucks

Woodchuck

Woodchucks

[Ww]oodchucks?

Regular expressions are case sensitive

All the above are different regular expression

Slide 28:

Regular Expression

Slide 29:

Regular Expressions: Disjunctions

Letters inside square brackets []

Ranges [A-Z]

Slide 30:

Regular Expressions: Negation in Disjunction

Negations [^ss]

Carat means negation only when first in []

Slide 31:

Regular Expressions: More Disjunction

Woodchuck is another name for groundhog!

The pipe | for disjunction

Slide 32:

Regular Expressions: ? *+.

Stephen C Kleene

Kleene *, Kleene +

Slide 33:

Regular Expressions: Anchors \$

993, 99

If your pattern is 99 it will be matched to both above string if you want to match only 99 you can use 99

-

Slide 36:

Example

Find me all instances of the word "the" in a text.

the

Misses capitalized examples

[tT]he

Incorrectly returns other or theology

$[^a - zA - Z][tT]he[^a - zA - Z]$

Slide 37:

Errors

The process we just went through was based on fixing two kinds of errors:

Matching strings that we should not have matched (there, then, other)

False positives (Type I errors)

Not matching things that we should have matched (The)

False negatives (Type II errors)

Slide 38:

Errors cont.

In NLP we are always dealing with these kinds of errors.

Reducing the error rate for an application often involves two antagonistic efforts:

Increasing accuracy or precision (minimizing false positives)

Increasing coverage or recall (minimizing false negatives).

Slide 41:

Summary

Regular expressions play a surprisingly large role

Sophisticated sequences of regular expressions are often the first model for any text processing text

For hard tasks, we use machine learning classifiers

But regular expressions are still used for pre-processing, or as features in the classifiers

Can be very useful in capturing generalizations

41

Slide 42:

More Regular Expressions:

Substitutions and ELIZA

Slide 43:

An important use of regular expressions is in substitutions.

For example, the substitution operator `s/regexpl/pattern/` used in Python and in Unix commands like `vim` or `sed` allows a string characterized by a regular expression to be replaced by another string:

Slide 44:

Substitutions

Substitution in Python and UNIX commands:

`s/regexpl/pattern/`

e.g.:

`s/colour/color/`

Slide 45:

Capture Groups

Say we want to put angles around all numbers:

the 35 boxes the `<35>` boxes

Use parens `()` to "capture" a pattern into a numbered register (1, 2, 3...)

Use `%` to refer to the contents of the register

`s/([0-9]+)/<%>/`

Slide 46:

Capture groups: multiple registers

`/the (.*?)er they (.*?) the er we /`

Matches

the faster they ran, the faster we ran

But not

the faster they ran, the faster we ate
Sham went to Paris with Ram, Ram loves Paris."
/sham went to (Paris) with (Ram), loves

Slide 47:

The harder he worked, the harder she worked.
the faster they ran, the faster we ran
The sooner they arrived, the sooner we arrived.

Slide 48:

But suppose we don't want to capture?

Parentheses have a double function: grouping terms, and capturing

Non-capturing groups: add a ?: after paren:

/(?: some|a few) (people|cats) like some /
matches

some cats like some cats

but not

some cats like some some

Slide 49:

Simple Application: ELIZA

Early NLP system that imitated a Rogerian psychotherapist

Joseph Weizenbaum, 1966.

Uses pattern matching to match, e.g.,:

"I need X"

and translates them into, e.g.

"What would it mean to you if you got X?"

Slide 50:

Simple Application: ELIZA

Men are all alike.IN WHAT WAY

They're always bugging us about something or other. CAN YOU THINK OF A SPECIFIC
EXAMPLE

Well, my boyfriend made me come here.YOUR BOYFRIEND MADE YOU COME HERE

He says I'm depressed much of the time.I AM SORRY TO HEAR YOU ARE DEPRESSED

Slide 51:

How ELIZA works

s/. * I'M (depressed|sad) . */I AM SORRY TO HEAR YOU ARE /

s/. * I AM (depressed|sad) . */WHY DO YOU THINK YOU ARE /

s/. * all . */IN WHAT WAY?/

s/. * always . */CAN YOU THINK OF A SPECIFIC EXAMPLE?/

Slide 52:

Write the regular expression for the following

Write the regular expression that matches a string that has an a followed by zero or more
b's

Write the regular expression for given language $L = E, a, aa, b, bb, ab, ba, aba, bab, \dots$, any
combination of a and b.

Write the regular expression for the language $L = a, aba, aab, aba, aaa, abab, \dots$

Write the regular expression for the language $L = a, aa, aaa, \dots$

Write the regular expression for the language $L = E, 0, 1, 00, 11, 10, 100, \dots$

Write the regular expression for the language accepting all the string which are starting with
1 and ending with 0, over $\Sigma = \{0, 1\}$.

Slide 53:

Write the regular expression for the following

Write the regular expression that matches a string that has an a followed by zero or more b's $R = ab^*$

Write the regular expression for given language $L = E, a, aa, b, bb, ab, ba, aba, bab, \dots$, any combination of a and b. Answer:-Solution

The regular expression will be $(a + b)^*$

Write the regular expression for the language $L = a, aba, aab, aba, aaa, abab, \dots$ $R = a + ab^*$

Write the regular expression for the language $L = a, aa, aaa, \dots$ $R = a^+$

Write the regular expression for the language $L = E, 0, 1, 00, 11, 10, 100, \dots$

$R = (1^* 0^*)$

Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over $\Sigma = 0, 1$. $R = 1(0+1)^* 0$

Slide 54:

Finite State Automata

Finite Automata (FA) is the simplest machine to recognize patterns.

The finite automata or finite state machine is an abstract machine that has five elements or tuples.

It has a set of states and rules for moving from one state to another, but it depends upon the applied input symbol.

Basically, it is an abstract model of a digital computer.

Finite automata have two states, Accept state or Reject state.

When the input string is processed successfully, and the automata reached its final state, then it will accept.

Slide 55:

The above figure shows the following features of automata:

Input

Output

States of automata

State relation

Output relation

Slide 56:

A Finite Automata consists of the following:

Q : Finite set of states.

Σ : set of Input Symbols.

q : Initial state.

F : set of Final States.

δ : Transition Function.

Slide 57:

Deterministic Finite Automata (DFA):

DFA consists of 5 tuples Q, Σ, q, F, δ .

Q : set of all states.

Σ : set of input symbols. (Symbols which machine takes as input)

q : Initial state. (Starting state of a machine)

F : set of final state.

δ : Transition Function, defined as $\delta : Q \times \Sigma \rightarrow Q$.

In a DFA, for a particular input character, the machine goes to one state only.

A transition function is defined on every state for every input symbol.

Also in DFA null (or ϵ) move is not allowed, i.e., DFA cannot change state without any input character.

Slide 58:

For example, below DFA with $\Sigma = 0, 1$ accepts all strings ending with 0.

Slide 64:

State Transition Diagram

Slide 71:

Draw a deterministic finite automata which accept 00 and 11 at the end of a string containing 0, 1 in it, e.g., 01010100 but not 000111010.

Slide 73:

FSA as language Recognizer

Recognition is the process of determining if a string should be accepted by a machine •

Or... it's the process of determining if a string is in the language, we're defining with the machine

Or... it's the process of determining if a regular expression matches a string

- Those all amount the same thing in the end

Slide 74:

Recognition

Simply a process of starting in the start state

Examining the current input

Consulting the table

Going to a new state and updating the input pointer.

Until you run out of input

Slide 81:

Finite-state can capture the generalization here:

Eg.

I eat Sushi

Ram like Mango

Noun+ Verb Noun+

Slide 82:

Language is recursive

§ the ball

§ the ball in the garden

§ the ball in the garden behind the house

§ the ball in the garden behind the house next to school

§ the ball

§ the big ball

§ the big, red ball

§ the big, red, heavy ball

Slide 84:

Morphological parsing

Morphological parsing is to find the lexical form of a word from its surface form.

Given the input, for example, cats, we would like to produce cat +N +PL.

Stem—Prefix+Stem+Suffix

Slide 85:

Morphological parsing

Two-level morphology,

– Representing a word as a correspondence between a lexical level

- Representing a simple concatenation of morphemes making up a word, and

– The surface level

- Representing the actual spelling of the final word.

• Morphological parsing is implemented by building mapping rules that maps letter sequences like cats on the surface level into morpheme and features sequence like cat +N +PL on the lexical level

Slide 86:

Morphological Parsing

-cats ——— cat +N +PLU
 -cat ——— cat +N +SG
 -goose ——— goose +N +SG
 -geese ——— goose +N +PLU
 -catch ——— catch +V
 -caught ——— catch +V +PAST

Surface Form

Lexical Form

Slide 87:

Parts of A Morphological Processor

- **Lexicon:** The list of stems with basic information about categories (noun, verb, adjective, ...) sub-categories (regular noun, irregular noun, ...)
- **Morphotactics:** Explains ordering -which classes of morphemes can follow other classes of morphemes inside a word.
- **Orthographic Rules (Spelling Rules):** These spelling rules are used to model changes that occur in a word (normally when two morphemes combine).

Slide 88:

Lexicon

- A lexicon is a repository for words (stems).
- They are grouped according to their main categories.
-noun, verb, adjective, adverb, ...
- They may be also divided into sub-categories.
-regular-nouns, irregular-singular nouns, irregular-plural nouns
- The simplest way to create a morphological parser, put all possible words (together with its inflections) into a lexicon.

Slide 89:

Morphotactic

In a natural language, a word may have many morphemes attached to it as affixes (prefix, suffix, infix).

Morphotactic is about placing morphemes with stem to form a meaningful word.

It gives the details about which classes of morpheme can follow which other classes of morpheme.

For example, in English language the plural morpheme (s or es) follows the stem.

Slide 90:

Morphotactics

Slide 91:

Orthographic Rules

It refers to the spelling rules used in a particular language to model the spelling changes that occur in a word.

For example, when a stem 'cook' is attached with the morpheme 'ing', it becomes a new valid word with the spelling 'cooking'.

What happened here was the concatenation of two strings.

This is not true for all words. If we attach the morpheme 'ing' with the word 'care', the spelling changes to 'caring' rather than 'careing'.

In this case, it was not just concatenation, but it also involved removal of the letter 'e'.

Slide 94:

Finite state transducers

A finite state transducer essentially is a finite state automaton that works on two (or more) tapes. The most common way to think about transducers is as a kind of “translating machine”.

They read from one of the tapes and write onto the other. This, for instance, is a transducer that translates as into bs:

Slide 95:

Finite state transducers

A finite state transducer (FST) is a finite state machine where transitions are conditioned on a pair of symbols

The machine moves between the states based on input symbol, while it outputs the corresponding output symbol

An FST encodes a relation, a mapping from a set to another. The relation defined by an FST is called a regular (or rational) relation

Slide 99:

Two-Level Morphology

Two-level morphology represents the correspondence between lexical and surface levels.

- We use a finite-state transducer to find mapping between these two levels.

- **A FST is a two-tape automaton:**

- Reads from one tape and writes to other one.

- For morphological processing, one tape holds lexical representation, the second one holds the surface form of a word.

Slide 100:

Morphological segmentation (or Stemming)

Taking a surface input and breaking it down into its morphemes

cat- cat+N+SL

cats- cat+N+PL

Plays- Play+V+Singular

Played- Play+V+ PastParticipat

foxes – fox+N+PL

Slide 102:

#

+Sg

Slide 103:

#

+Sg

Slide 104:

For each spelling rule we will have a FST, and these FSTs run parallel.

Slide 105:

Orthographic Rules

- For each spelling rule we will have a FST, and these FSTs run parallel.

- **We represent these rules using two-level morphology rules:**

$a \Rightarrow b / c \text{ ___ } d$

rewrite a as b when it occurs between c and d.

English Spelling Rules:

–E insertion –e added after s, z, x, ch, sh before s –watch/watches

Slide 106:

Stemming

Stemming is suffix stripping operation

Process of reducing word into its base form (Root form/stem form)

This is achieved by cutting of beginning or end of the word

Eg:

Playing- Play

Boys- Boy

Popular Algorithm is Porter Stemmer

Slide 107:

Porter Stemmer

- Five sets of rules, applied in order

- Within each set, if more than one of the rules can apply, only the one with the longest matching suffix (S1) is followed

Advantage: easy to see understand, easy to implement.

Slide 108:

Convention

Consonant(C): other than A, E, I, O or U, and other than Y preceded by a consonant.

So in TOY the consonants are T and Y

Vowel(V) : Any other letter.

Slide 109:

Any word in English has this forms:

[C](VC_m)[V]

[]denotes arbitrary presence of content

C-Consonant

V-Vowel

m will be called the measure of any word or word part

m=0 TR, EE, TREE, Y, BY.

m=1 TROUBLE, OATS, TREES, IVY.

m=2 TROUBLES, PRIVATE, OATEN, ORRERY.

Slide 110:

RULES

The rules for removing a suffix will be given in the form (condition) S1 -> S2 if a word ends with the suffix S1 will be replaced by S2, if it satisfies the given condition.

e.g. (m > 1) EMENT -> Here S1 is 'EMENT' and S2 is null.

REPLACEMENT to REPLAC,

since REPLAC is a word part for which m = 2.

Slide 111:

E.g. (m>1 or *S)

Slide 112:

Step 1a :

sses -> ss (**Example :** caresses -> caress)

ies -> i (**Example :** ponies -> poni ; ties -> ti)

ss -> ss (**Example :** caress -> caress)

s -> (**Example :** cats -> cat)

PORTER STEMMER

Slide 113:

Step 1b :

(m>0) eed -> ee (**Example :** agreed -> agree; feed -> feed)

(*v*) ed -> (**Example :** plastered -> plaster ; bled -> bled)

(*v*) ing -> (**Example :** motoring -> motor ; sing -> sing)

s -> (**Example :** cats -> cat)

Slide 114:

If the second or third of the rules in Step 1b is successful, the following is done: Cleaning Step

at -> ate (**Example :** conflat(ed) -> conflate)

bl -> ble (**Example :** troubl(ed) -> trouble)

iz -> ize (**Example :** siz(ed) -> size)

s -> (**Example** : cats -> cat)
 (*d ! (*l or *s or *z)) -> single letter
 (Example : hopp(ing) -> hop ; tann(ed) -> tan ; fall(ing) -> fall ; hiss(ing) -> hiss ; fizz(ed)
 -> fizz)
 (m=1 and *o) -> e
 (**Example** : fil(ing) -> file); fail(ing) -> fail

Slide 115:

Step 1c : Y Elimination

(v) y -> i (**Example** : happy -> happi ; sky -> sky)

Step 1 deals with plurals and past participles. The subsequent steps are much more straightforward.

Step 2 : Derivational Morphology -1

(m>0) ational -> ate (**Example** : relational -> relate

(m>0) ization -> ize (generalization -> generalize)

(m>0) biliti -> ble (sensibiliti -> sensible)

Slide 116:

Step 3 :

(m>0) icate -> ic (**Example** : triplicate -> triplic)

(m>0) ful -> (**Example** : hopeful -> hope)

(m>0) ness -> (**Example** : goodness -> good)

Slide 117:

Step 4 : Derivational Morphology-II

((m>1) ance -> (**Example** : allowance -> allow)

(m>1) ment -> (**Example** : adjustment -> adjust)

(m>1) ent -> (**Example** : dependent -> depend)

(m>1) ive -> (**Example** : effective -> effect)

Slide 118:

The suffixes are now removed. All that remains is a little tidying up.

Step 5a :

(m>1) e -> (**Example** : probate -> probat ; rate -> rate)

(m=1 and not *o) ness -> (**Example** : goodness -> good)

Step 5b

(m > 1 and *d *l) -> single letter

(**Example** : controll -> control ; roll -> roll)

Slide 119:

Disadvantage

This indiscriminate cutting will be successful at some occasion and fail in some
 Eg.

Studies- Studi

Giving- Giv

Intelligence-Intelligen

Produced immediate representation of the word may not have any meaning

Slide 120:

Stemming Lemmatization

Cutting of Suffixes to extract stem

Eg.

Studies- Studi

Giving- Giv

Caring- Car

Produced immediate representation of the word may not have any meaning

Cutting of Suffixes to extract Lemma

Eg.

Studies- Study

Giving- Give

Caring- Care

Produced immediate representation of the word having meaning

Slide 121:

Lemmatization

Same as Stemming but immediate representation have some meaning

Stemming and lemmatization both of these concepts are used to normalized the given word by removing affixes

Slide 122:

N-Gram Language Model

Predicting Nth word from N-1 words

Predicting 3rd word from previous 2 words – Model is called Trigram

Predicting 2nd word from previous 1 words – Model is called Bigram

He is going to _____ Predicting 5th word from last four words is 5 gram Language Model going to _____ Predicting 3rd word from last two words is trigram

Slide 123:

Application of N-Gram

Optical Character Recognition

Image Text

If words are missing or not clear can be predicted

Grammar correction

Spelling is not correct then based on context can be suggested

If spelling is right

E.g. Deer sir instead of Dear sir can be correctly as, Sir is normally followed by Dear not Deer

Speech to Text

Words with different pronunciation. e.g. Eye am Fine I am Fine

Translation

Multiple Synonym of words

E.g., He is biggest minister of Pakistan Prime

Suggestion while typing in mobile text prediction mode.

Slide 124:

He is going to school Meaningful

He school is to going Doesn't Make sense

For N-gram models

$P(w_1, w_2, \dots, w_n)$

By the Chain Rule we can decompose a joint probability, as follows:

$P(w_1, w_2, \dots, w_n) = P(w_1) P(w_2|w_1) P(w_3|w_1w_2) \dots P(w_n|w_1, \dots, w_{n-1})$

Join Probability of sentence

$P(\text{He is going to school}) = P(\text{He})P(\text{is}|\text{He})P(\text{going}|\text{He is})P(\text{to}|\text{He is going})P(\text{school}|\text{He is going to})$

Looking back so much

5.2 *Porter Stemmer Algorithm*(3)

Slide 1:

Porter Stemmer Algorithm

Slide 2:

Stemming

Stemming is suffix stripping operation

Process of reducing word into its base form (Root form/stem form)

This is achieved by cutting of beginning or end of the word

Eg:

Playing- Play

Boys- Boy

Popular Algorithm is Porter Stemmer

Slide 3:

Porter Stemmer

- Five sets of rules, applied in order

- Within each set, if more than one of the rules can apply, only the one with the longest matching suffix (S1) is followed

Advantage: easy to see understand, easy to implement.

Slide 4:

Algorithm

Few definitions.

Consonant-

it is a letter other than A, E, I, O or U, and other than Y preceded by a consonant.

So, in TOY the consonants are T and Y, and in SYZYGY they are S, Z and G.

If a letter is not a consonant it is a vowel.

Slide 5:

Algorithm

A consonant will be denoted by c, a vowel by v.

A list ccc... of length greater than 0 will be denoted by C,

and a list vvv... of length greater than 0 will be denoted by V.

Any word, or part of a word, therefore, has one of the four forms:

CVCV ... C

CVCV ... V

VCVC ... C

VCVC ... V

Slide 6:

Algorithm

These may all be represented by the single form $[C]VCVC \dots [V]$ where the square brackets denote arbitrary presence of their contents.

Using $(\$VC^m\$)$ to denote VC repeated m times, this may again be written as $[C](\$VC^m\$)[V]$ m will be called the m

The case $m = 0$ covers the null word. Here are some examples: (m is no of VC pairs)

$m=0$ TR, EE, TREE, Y, BY.

$m=1$ TROUBLE, OATS, TREES, IVY.

$m=2$ TROUBLES, PRIVATE, OATEN, ORRERY.

Slide 8:

The rules for removing a suffix will be given in the form (condition) $S1 \rightarrow S2$ This means that if a word ends with the suffix S1, and the stem before S1 satisfies the given condition, S1 is replaced by S2. The condition is usually given in terms of m,

e.g. ($m > 1$) EMENT $\rightarrow$ Here S1 is 'EMENT' and S2 is null.

This would map REPLACEMENT to REPLAC, since REPLAC is a word part for which $m = 2$.

Slide 9:

The 'condition' part may also contain the following:

*S - the stem ends with S (and similarly for the other letters).

v - the stem contains a vowel.

$m=2$ TROUBLES, PRIVATE, OATEN, ORRERY.

*d - the stem ends with a double consonant (e.g. -TT, -SS).

*o - the stem ends cvc, where the second c is not W, X or Y (e.g. -WIL, -HOP).

Slide 10:

And the condition part may also contain expressions with and, or and not, so that:

(m>1 and (*S or *T)) : tests for a stem with m>1 ending in S or T, while

(*d and not (*L or *S or *Z)) : tests for a stem ending with a double consonant other than L, S or Z.

Elaborate conditions like this are required only rarely.

Slide 11:

In a set of rules written beneath each other, only one is obeyed, and this will be the one with the longest matching S1 for the given word.

For example, with

SSSES -> SS

IES -> I

SS -> SS

S ->

(here the conditions are all null) CARESSES maps to CARESS since SSES is the longest match for S1. Equally CARESS maps to CARESS (S1=SS) and CARES to CARE (S1=S).

In the rules below, examples of their application, successful or otherwise, are given on the right in lower case. The algorithm now follows:

Slide 12:

Step 1a :

SSSES -> SS (Example : caresses -> caress)

IES -> I (Example : ponies -> poni ; ties -> ti)

SS -> SS (Example : caress -> caress)

S -> (Example : cats -> cat)

Slide 13:

Step 1b :

(m>0) EED -> EE (Example : feed -> feed ; agreed -> agree)

(v) ED -> (Example : plastered -> plaster ; bled -> bled)

(v) ING -> (Example : motoring -> motor ; sing -> sing)

S -> (Example : cats -> cat)

Slide 14:

If the second or third of the rules in Step 1b is successful, the following is done:

AT -> ATE (Example : conflat(ed) -> conflate)

BL -> BLE (Example : troubl(ed) -> trouble)

IZ -> IZE (Example : siz(ed) -> size)

S -> (Example : cats -> cat)

(*d and not (*L a q a or *S or *Z)) -> single letter

(Example : hopp(ing) -> hop ; tann(ed) -> tan ; fall(ing) -> fall ; hiss(ing) -> hiss ; fizz(ed) -> fizz)

(m=1 and *o) -> E (Example : fail(ing) -> fail ; fil(ing) -> file)

Slide 15:

The rule to map to a single letter causes the removal of one of the double letter pair.

The -E is put back on -AT, -BL and -IZ, so that the suffixes -ATE, -BLE and -IZE can be recognized later.

This E may be removed in step 4.

Slide 16:

Step 1c :

(v) Y -> I (Example : happy -> happi ; sky -> sky)

Step 1 deals with plurals and past participles. The subsequent steps are much more straightforward.

Slide 17:

Step 2 :

(m>0) ATIONAL -> ATE (Example : relational -> relate)
(m>0) TIONAL -> TION (Example : conditional -> condition ; rational -> rational)
(m>0) ENCI -> ENCE (Example : valenci -> valence)
(m>0) ANCI -> ANCE (Example : hesitanci -> hesitance)
(m>0) IZER -> IZE (Example : digitizer -> digitize)
(m>0) ABLI -> ABLE (Example : conformabli -> conformable)
(m>0) ALLI -> AL (Example : radicalli -> radical)
(m>0) ENTLI -> ENT (differentli -> different)
(m>0) ELI -> E (vileli -> vile)
(m>0) OUSLI -> OUS (analogousli -> analogous)
(m>0) IZATION -> IZE (vietnamization -> vietnamize)

Slide 18:

Step 2: Continue

(m>0) ATION -> ATE (predication -> predicate)
(m>0) ATOR -> ATE (operator -> operate)
(m>0) ALISM -> AL (feudalism -> feudal)
(m>0) IVENESS -> IVE (decisiveness -> decisive)
(m>0) FULNESS -> FUL (hopefulness -> hopeful)
(m>0) OUSNESS -> OUS (callousness -> callous)
(m>0) ALITI -> AL (formaliti -> formal)
(m>0) IVITI -> IVE (sensitiviti -> sensitive)
(m>0) BILITI -> BLE (sensibiliti -> sensible)

Slide 19:

Step 2: Continue

The test for the string S1 can be made fast by doing a program switch on the penultimate letter of the word being tested. This gives a fairly even breakdown of the possible values of the string S1. It will be seen in fact that the S1-strings in step 2 are presented here in the alphabetical order of their penultimate letter. Similar techniques may be applied in the other steps.

Slide 20:

Step 3 :

(m>0) ICATE -> IC (Example : triplicate -> triplic)
(m>0) ATIVE -> (Example : formative -> form)
(m>0) ALIZE -> AL (Example : formalize -> formal)
(m>0) ICITI -> IC (Example : electriciti -> electric))
(m>0) ICAL -> IC (Example : electrical -> electric)
(m>0) FUL -> (Example : hopeful -> hope)
(m>0) NESS -> (Example : goodness -> good)

Slide 21:

Step 4 :

(m>1) AL -> (Example : revival -> reviv)
(m>1) ANCE -> (Example : allowance -> allow)
(m>1) ENCE -> (Example : inference -> infer)
(m>1) ER -> (Example : airliner -> airlin)
(m>1) IC -> (Example : gyroscopic -> gyroscop)
(m>1) ABLE -> (Example : adjustable -> adjust)
(m>1) IBLE -> (Example : defensible -> defens)
(m>1) ANT -> (Example : irritant -> irrit)
(m>1) EMENT -> (Example : replacement -> replac)

(m>1) MENT -> (Example : adjustment -> adjust)

(m>1) ENT -> (Example : dependent -> depend)

Slide 22:

Step 4 : Continued

(m>1 and (*S or *T)) ION -> (Example : adoption -> adopt)

(m>1) OU -> (Example : homologou -> homolog)

(m>1) ISM -> (Example : communism -> commun)

(m>1) ATE -> (Example : activate -> activ)

(m>1) ITI -> (Example : angulariti -> angular)

(m>1) OUS -> (Example : homologous -> homolog)

(m>1) IVE -> (Example : effective -> effect)

(m>1) IZE -> (Example : bowdlerize -> bowdler)

The suffixes are now removed. All that remains is a little tidying up.

Slide 23:

Step 5a :

(m>1) E -> (Example : probate -> probat ; rate -> rate)

(m=1 and not *o) E -> (Example : cease -> ceas)

Slide 24:

Step 5b

(m > 1 and *d and *L) -> single letter

(Example : controll -> control ; roll -> roll)

5.3 3_{*SyntaxAnalysis*}2023(2)

Slide 1:

Syntax Analysis

Slide 2:

Steps in NLP

John Ate the Apple

Ate the Apple John

Slide 3:

Tagging

Tagging is a kind of classification that may be defined as the automatic assignment of description to the tokens.

Here the descriptor is called tag, which may represent one of the part-of-speech, semantic information and so on.

parts of speech are defined instead based on their grammatical relationship with neighboring words or the morphological properties about their affixes

Slide 4:

Part-of-Speech (PoS) tagging

the process of assigning one of the parts of speech to the given word

POS tagging is a task of labelling each word in a sentence with its appropriate part of speech

Examples of Parts Of Speech – nouns, verb, adverbs, adjectives, pronouns, conjunction and their sub-categories

Slide 5:

POS Tagging Example

Annotate each word in a sentence with a part-of-speech.

A POS tagger considers surrounding words while assigning a tag.

[("I", "Pronoun"), ("like", "Verb"), ("to", "To"), ("read", "Verb"), ("books", "Noun")]

Slide 6:

Categories of Parts of Speech

Closed class

Open class

Slide 7:

Open Class (Mostly Content word)

Called open class because we will keep on adding new words in the language example
google,skype,photoshop

Noun,verbs,adjective,adverbs

Mostly content bearing –they refer to objects,actions,and features in the world

Slide 8:

Close class (function words)

Pronoun, determiner, prepositions, connectives

Limited in number

Used to tie the concepts of the sentences together we use these closed words

Slide 9:

POS tags: level of Details

How much detail we want to specify the particular category

Level of details of POS Tag

Coarse grained Fine grained

Decision is dependent on the application

Slide 10:

Coarse Grained(Small Tag Set)

Here we do not provide the pos tag in much details(Grammatical Details)

For example –we can identify a word as NOUN but we can not tell whether it Singular Noun
or Plural Noun

Slide 11:

Fine grained (Large Tag set)

In fine grained level we provide grammatical details of a word

For example if Verb then is it past tense ,future or present tense

NOTE : May Lead to confusion due to too many POS tags

Slide 12:

POS TAG / Word Classes

9 traditional word classes of parts of speech

Noun, verb, adjective, preposition, adverb, article, pronoun, conjunction, interjection

N noun chair, bandwidth, pacing

V verb study, debate, munch

ADJ adjective purple, tall, ridiculous

ADV adverb unfortunately, slowly

P preposition of, by, to

PRO pronoun I, me, mine

DET determiner the, a, that, those

12

Slide 13:

Part-of-speech tagging

process of assigning a part-of-speech to each word in part-of-speech tagging a text.

input sequence $x_1, x_2, \dots, x_n$ of (tokenized) words and a tagset,

Output is a sequence $y_1, y_2, \dots, y_n$ of tags,

each output y_i corresponding exactly to one input x_i as shown below

Slide 14:

POS Tagging is disambiguation task

Words are ambiguous(more than one possible part-of-speech)

the goal is to find the correct tag for the situation

For example,

book can be a verb (book that flight)

Or

a noun (hand me that book)

The goal of POS-tagging is to resolve these ambiguity resolution ambiguities, choosing the proper tag for the context

Slide 15:

Part-of-Speech Tag sets

There are various tag sets to choose.

- The choice of the tag set depends on the nature of the application.
- We may use small tag set (more general tags) or
- large tag set (finer tags).

• **Some of widely used part-of-speech tag sets:**

- Penn Treebank has 45 tags
- Brown Corpus has 87 tags
- C7 tag set has 146 tags

- In a tagged corpus, each word is associated with a tag from the used tag set.

Slide 16:

Penn Treebank Tagset

16

Slide 17:

POS Tagging Approaches

Rule-based tagging

E.g. EnCG ENGTWOL tagger

Stochastic, or, Probabilistic tagging

HMM (Hidden Markov Model) tagging

Transformation-based tagging

Learned rules (statistic and linguistic)

E.g., Brill tagger

17

Slide 18:

POS Tagging Approaches

Slide 19:

Rule Based POS Tagging Approaches

Input: - Sentence

Tag dictionary /Lexicon-Using Tag dictionary POS tags will be assigned to the words in the given sentence

For Example : -

I play cricket everyday (Play is Verb)

I want to perform a play (Play is Noun)

(Here play has two tags Noun and Verb Ambiguous POS)

Handwritten rules-

For example an Ambiguous word is noun if it follows a determiner

Slide 20:

Rule-Based Tagging

The rule-based approach uses handcrafted sets of rules to tag input sentence.

Stage 1:

Typically...start with a dictionary of words and possible tags

Assign all possible tags to words using the dictionary

Stage 2:

Write rules by hand to selectively remove tags

Stop when each word has exactly one (correct) tag

20

Slide 21:

Rules based on context

Please book that flight

I bought a Book.

Book

W_{n-1}

W_n

W_{n+1}

Book

T_{n-1}

T_n

T_{n+1}

• **Rule-2:** if (the previous tag is an article)

then eliminate all verb tags

Rule-1: if (the previous word is “to”)

then eliminate all Noun tags

Slide 22:

Apply Rules Eliminating Some POS

E.g., Eliminate VBN if VBD is an option when VBN|VBD follows “<start> PRP”

NN

RB

JJ VB

PRP VBD TO VB DT NN

She promised to back the bill

22

Slide 23:

Rule-Based Part-of-Speech Tagging: Example

Pronoun

Verb

Article

Verb Noun

Pronoun

Verb

Article

Verb Noun

Slide 24:

Rule Base Tagger

It is oldest approach and uses hand written rules for tagging

Depends on dictionary or lexicon to get possible tags for each word to tagged

Language experts will sit together to find out what are the symbol patterns.

Hand written rules are used to identify the correct tag when the word has more than one possible tag.

Disambiguation is done by analysing the linguistic features of the word, its preceding word following word and other aspects .

Slide 25:

Rule Base Tagger

For example, if the preceding word is article, then the word must be a noun.

We can also understand Rule-based POS tagging by its two-stage architecture

The first stage In the first stage, create a dictionary to assign each word a list of potential parts-of-speech.

The second stage In the second stage, create a large list of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word as per requirements.

TAGGIT the first large rule-based tagger used context pattern rules.

TAGGIT used the set of 71 tags and 3300 disambiguation rules.

These rules disambiguated 77

Slide 26:

Example

University Corpus

I want to read a book.

Book-Noun or Verb

Rule : -Before book ,determiner (a) is there ,therefore it is a noun.

Slide 27:

Limitation of Rule based Approach

Hard coded rules are required

Rules has to be updated based on language change

It needs to check lexicon every time which is not efficient approach

Strong language expert team is required.

Slide 28:

Hidden Markov Model

An HMM is a probabilistic sequence model:

given a sequence of units (words, letters, morphemes, sentences, whatever), it computes a probability distribution over possible sequences of labels and chooses the best label sequence.

Slide 29:

Markov Chain

Markov chains, named after Andrey Markov, a stochastic model that depicts a sequence of possible events

where predictions or probabilities for the next state are based solely on its previous event state, not the states before.

In simple words, the probability that $n+1$ th steps will be x depends only on the n th steps not the complete sequence of steps that came before n .

This property is known as Markov Property or Memorylessness.

Let us explore our Markov chain with the help of a diagram,

Slide 30:

A diagram representing a two-state(here, E and A) Markov process.

Here the arrows originated from the current state and point to the future state and the number associated with the arrows indicates the probability of the Markov process changing from one state to another state.

For instance, if the Markov process is in state E, then the probability it changes to state A is 0.7, while the probability it remains in the same state is 0.3. Similarly, for any process in state A, the probability to change to E state is 0.4 and the probability to remain in the same state is 0.6.

Slide 31:

Markov Chain

tells us something about the probabilities of sequences of random variables, states, each of which can take on values from some set

These sets can be words or tags or symbol representing anything for example weather

Markov Assumption

if we want to predict the future in the sequence, all that matters is the current state.

All the states before the current state have no impact on the future except via the current state.

Slide 33:

Markov chain is specified by the following components:

Slide 34:

The Hidden Markov Model

A Markov chain is useful when we need to compute a probability for a sequence of observable events

In many cases, however, the events we are interested in are hidden : we don't observe them directly

For example we don't normally observe part-of-speech tags in a text. Rather, we see words, and must infer the tags from the word sequence. We call the tags hidden because they are not observed

Slide 35:

A hidden Markov model

A hidden Markov model (HMM) allows us to talk about both observed events (like words that we see in the input) and hidden events (like part-of-speech tags) that we think of as causal factors in our probabilistic model.

Slide 36:

HMM is specified by the following components

Slide 37:

A first-order hidden Markov model instantiates two simplifying assumptions.

First, as with a first-order Markov chain, the probability of a particular state depends only on the previous state:

Second, the probability of an output observation o_i depends only on the state that produced the observation q_i and not on any other states or any other observations:

Slide 43:

Transition probability

The transition probability is the likelihood of a particular sequence for example, how likely is that a noun is followed by a model and a model by a verb and a verb by a noun.

This probability is known as Transition probability. It should be high for a particular sequence to be correct.

Emission probability :

These sets of probabilities are Emission probabilities and should be high for our tagging to be likely.

Now, what is the probability that the word Ted is a noun, will is a model, spot is a verb and Will is a noun.

Slide 50:

The components of an HMM tagger

An HMM has two components, the A and B probabilities

The A matrix contains the tag transition probabilities $P(t_i | t_{i-1})$ which represent the probability of a tag occurring given the previous tag.

For example, modal verbs like will are very likely to be followed by a verb in the base form, a VB, like race, so we expect this probability to be high.

We compute the maximum likelihood estimate of this transition probability by counting, out of the times we see the first tag in a labeled corpus, how often the first tag is followed by the second:

Slide 51:

The B emission probabilities, $P(w_i | t_i)$, represent the probability, given a tag (say MD), that it will be associated with a given word (say will).

The MLE of the emission probability is

Slide 52:

HMM tagging as decoding

HMM contains hidden variables, the task of determining the hidden variables sequence corresponding to the sequence of observations decoding is called decoding.

Slide 53:

For part-of-speech tagging, the goal of HMM decoding is to choose the tag sequence $t_1 \dots t_n$ that is most probable given the observation sequence of n words $w_1 \dots w_n$:

The way we'll do this in the HMM is to use Bayes' rule to instead compute:

Furthermore, we simplify Eq. Above by dropping the denominator $P(w_{n+1})$:

Slide 54:

HMM taggers make two further simplifying assumptions.

The first is that the probability of a word appearing depends only on its own tag and is independent of neighboring words and tags:

The second assumption, the bigram assumption, is that the probability of a tag is dependent only on the previous tag, rather than the entire tag sequence;

Slide 55:

Plugging the simplifying assumptions from Eq. X and Eq. Y into Eq. Z results in the following equation for the most probable tag sequence from a bigram tagger:

Slide 56:

Find the probability of the following sentence : Jane will spot Will

Slide 59:

In general, we from all the possible combination of parts of speech tags we will pick the one that generates the above sentence Jane will spot Will with the highest probability

How to find the correct path ?

Slide 62:

Which one of this generates the sentence with highest probability ?

Slide 68:

For every possible state we require the highest probability path that ends in that state .

This is known as Viterbi HMM Algorithm

Slide 70:

Emission Probability Matrix

Transition Probability Matrix

Slide 81:

We keep the edge which gives us a high value and remembering the edge which gives us a highest value .

We have all the probabilities and edges which gives those probabilities.

To get the optimal path, we start from end and trace backwards

Each state has only one incoming edge ,this will give us optimal path with highest probabilities .

Slide 83:

Consider the following data set

Apply HMM to find Tag Sequence for Will can spot Mary

Slide 84:

Transition Probability

Slide 85:

Emission Probability

5.4 4*ConstituencyGrammars*(1)(2)

Slide 1:

Constituency Grammars

Slide 2:

Context-free grammars

CFGs the backbone of many formal models of the syntax of natural language (and, for that matter, of computer languages).

they play a role in many computational applications, including grammar checking, semantic interpretation, dialogue understanding, and machine translation.

express sophisticated relations among the words in a sentence, yet computationally tractable enough that efficient algorithms exist for parsing sentences with them

Slide 3:

Constituency

Syntactic constituency is the idea that groups of words can behave as single units, or constituents.

For example, group of words come together to form a Noun Phrase.

Part of developing a grammar involves building an inventory of the constituents in the language.

Slide 4:

How do words group together in English?

Consider noun phrase the noun phrase, a sequence of words surrounding at least one noun.

Here are some examples of noun phrases

Slide 5:

How do words group together in English?

How these words form constituents or they group together ?

One evidence is that they can all appear in similar syntactic environments, for example, before a verb.

Slide 6:

But while the whole noun phrase can occur before a verb, this is not true for each of the individual words that make up a noun phrase.

The following are not grammatical sentences of English (we use an asterisk (*) to mark fragments that are not grammatical English sentences):

Slide 7:

Pre-posed or post-posed constructions

For example, the prepositional phrase on September seventeenth can be placed in a number of different locations in the following examples, including at the beginning (preposed) or at the end (postposed):

But again, while the entire phrase can be placed differently, the individual words making up the phrase cannot be

Slide 8:

Verb Phrase

Slide 9:

Adjective Phrase

Slide 10:

Context-Free Grammars

formal system for modeling constituent structure in English and other natural languages is the Context-Free Grammar, or CFG.

It is also called Phrase-Structure Grammars

Slide 11:

Context-Free Grammars

A context-free grammar consists of a set of rules or productions

Which expresses the ways that symbols of the language can be grouped and ordered together, and a lexicon of words and symbols

Following is the Example of production rules of - CFG

Slide 12:

Context-free Grammars

Context-free rules can be hierarchically embedded, so we can combine the previous rules with others, like the following, that express facts about the lexicon:

Slide 13:

Symbols in CFG

symbols that are used in a CFG are divided into two classes

Terminal non-terminal

The symbols

terminal that correspond to words in the language (“the”, “nightclub”)

symbols that express abstractions over these terminals are called non-terminals.

the item to the right of the arrow ($\rightarrow$) is an ordered list of one or more terminals and non-terminals;

to the left of the arrow is a single non-terminal symbol expressing some cluster or generalization.

Slide 14:

Two ways of CFG

a device for generating sentences and

a device for assigning a structure to a given sentence.

Slide 15:

CFG as a generator

we can read the $\rightarrow$ arrow as “rewrite the symbol on the left with the string of symbols on the right”.

a flight can be derived from the non-terminal NP

Slide 16:

CFG as a generator

Thus, a CFG can be used to generate a set of strings.]

This sequence of rule expansions is called a derivation of the string of words.

It is common to represent a derivation by a parse tree (commonly shown inverted with the root at the top).

Figure above “A parse tree for “a flight”” shows the tree representation of this derivation

Slide 17:

Start Symbol

The formal language defined by a CFG is the set of strings that are derivable start symbol from the designated start symbol.

Each grammar must have one designated start symbol, which is often called S.

Since context-free grammars are often used to define sentences, S is usually interpreted as the “sentence” node, and the set of strings that are derivable from S is the set of sentences in some simplified version of English.

Slide 18:

Few additional rules

The following rule expresses the fact that a sentence can consist of a noun phrase followed by a verb phrase:verb phrase

$S \rightarrow NP VP$ I prefer a morning flight

A verb phrase in English consists of a verb followed by assorted other things;for example, one kind of verb phrase consists of a verb followed by a noun phrase:

$VP \rightarrow Verb NP$ prefer a morning flight

Slide 19:

Few additional rules

the verb may be followed by a noun phrase and a prepositional phrase:

$VP \rightarrow Verb NP PP$ leave Boston in the morning

the verb phrase may have a verb followed by a prepositional phrase alone:

$VP \rightarrow Verb PP$ leaving on Thursday

A prepositional phrase generally has a preposition followed by a noun phrase.

For example, a common type of prepositional phrase in the ATIS corpus is used to indicate location or direction:

PP → Preposition NP from Los Angeles

Slide 20:

Example of lexicons

Slide 21:

The grammar for L0, with example phrases for each rule

Slide 22:

The parse tree for “I prefer a morning flight” according to grammar L0

Slide 23:

Formal Definition of Context-Free Grammar

A context-free grammar G is defined by four parameters: N , Σ , R , S (technically this is a “4-tuple”).

The problem of mapping from a string of words to its parse tree is called syntactic parsing;

Slide 24:

Constituency Parsing

Syntactic parsing is the task of assigning a syntactic structure to a sentence

Parse trees can be used in applications such as grammar checking sentence that cannot be parsed may have grammatical errors (or at least be hard to read).

Parse trees can be an intermediate stage of representation for semantic analysis

Slide 25:

Ambiguity

Slide 28:

Explanation

Two parse trees for an ambiguous sentence.

The parse on the left corresponds to the humorous reading in which the elephant is in the pajamas,

the parse on the right corresponds to the reading in which Captain Spaulding did the shooting in his pajamas

Slide 29:

Types of ambiguity

Two common kinds of ambiguity are attachment ambiguity and coordination ambiguity.

A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place.

The Groucho Marx sentence is an example of PP-attachment ambiguity. Various kinds of adverbial phrases are also subject to this kind of ambiguity.

For instance, in the following example the gerundive-VP flying to Paris can be part of a gerundive sentence whose subject is the Eiffel Tower or it can be an adjunct modifying the VP headed by saw:

We saw the Eiffel Tower flying to Paris.

attachment ambiguity

In coordination ambiguity phrases can be conjoined by a conjunction like and coordination ambiguity

For example, the phrase old men and women can be bracketed as [old [men and women]], referring to old men and old women, or as [old men] and [women], in which case it is only the men who are old.

coordination ambiguity

Slide 30:

CKY Parsing: A Dynamic Programming Approach

Slide 31:

How to Convert CFG to CNF

Slide 34:

CKY Algorithm

Slide 35:

CKY Algorithm

Slide 37:

Cocke-Younger-Kasami Algorithm

It is used to solve the membership problem using a dynamic programming approach.

The algorithm is based on the principle that the solution to problem $[i, j]$ can be constructed from solution to subproblem $[i, k]$ and solution to subproblem $[k, j]$.

The algorithm requires the Grammar G to be in Chomsky Normal Form (CNF). Note that any Context-Free Grammar can be systematically converted to CNF.

This restriction is employed so that each problem can only be divided into two subproblems and not more – to bound the time complexity.

Slide 38:

How does the CYK Algorithm work?

For a string of length N , construct a table T of size $N \times N$. Each cell in the table $T[i, j]$ is the set of all constituents that can produce the substring spanning from position i to j .

The process involves filling the table with the solutions to the subproblems encountered in the bottom-up parsing process.

Therefore, cells will be filled from left to right and bottom to top.

Slide 39:

In $T[i, j]$, the row number i denotes the start index, and the column number j denotes the end index.

Slide 40:

The algorithm considers every possible subsequence of letters and adds K to $T[i, j]$ if the sequence of letters starting from i to j can be generated from the non-terminal K .

For subsequences of length 2 and greater, it considers every possible partition of the subsequence into two parts, and checks if there is a rule of the form $A \rightarrow BC$ in the grammar where B and C can generate the two parts respectively, based on already existing entries in T .

The sentence can be produced by the grammar only if the entire string is matched by the start symbol, i.e., if S is a member of $T[1, n]$.

Slide 45:

CYK Algorithm

5.5 5_Probabilistic Context Free Grammar

Slide 1:

Probabilistic Context Free Grammar

Slide 2:

USE OF PCFG

Slide 3:

Probabilistic CFG

Slide 6:

Parse Tree

Slide 7:

Probabilities of The above Tree

5.6 6_Dependency Parsing(2)

Slide 1:

Dependency Parsing

Slide 2:

Introduction

In some languages, the same statement can be written in such a way that on translating in English (word to word translation), it might yield 'He is standing wall behind' i.e the word order is free.

It isn't necessary the adjective has to appear just before the noun it like 'Handsome boy'.

It can be 'Boy handsome'.

Hence, using rules for parsing such languages is problematic!!

And that is why dependency parsing is required.

Slide 3:

Dependency Parsing

Dependency parsing helps us build a parsing tree with the tags used determining the relationship between words

in the sentence rather than using any Grammar rule as used for syntactic parsing which gives a lot of flexibility even when the order of words (like 'boy handsome' or 'handsome boy') get changed.

A sample dependency tree can be observed below:

Slide 4:

Below is an example of Dependency parsing for:

'I prefer the morning flight through Denver'

Slide 5:

terminologies as well used for Dependency parsing

Head-Dependent:

In the arrows representing relationship, the origin word is the Head the destination word is Dependent.

For example: (I, 'nsubj', prefer), 'prefer' is Head 'I' is Dependent.

Root: Word which is the root of our parse tree. It is 'prefer' in the above example.

Slide 6:

terminologies as well used for Dependency parsing

Grammar Functions and Arcs: Tags between each Head-Dependent pair is a grammar function determining the relation between the Head Dependent.

The arrowhead carrying the tag is called an Arc. Below is the table that interprets the meaning of some common grammar functions:

Slide 7:

Some examples for Grammar Functions are given below:

Slide 8:

Some examples for Grammar Functions are given below:

Slide 9:

Understanding the Dependency parse tree

we represent dependencies as a directed graph $G = (V, A)$

where V(set of vertices) represents words (and punctuation marks as well) in the sentence

A(set of arcs) represent the grammar relationship between elements of V.

Slide 10:

Understanding the Dependency parse tree

A dependency parse tree is the directed graph mentioned above which has the below features:

Root has no Incoming arcs (can only be Head in Head-Dependent pair)

Vertices(except Root) should have only one incoming arc (Only one Parent/Head)

A Unique path should exist between Root each vertex in the tree.

Slide 11:

Projectivity

Projective arc: An arc/arrows_with_tag are projective when 'Head' associated with the arc has a path to reach each word that lies between 'Head' 'Dependent'.

Slide 12:

Explanation

Here, the arc between 'the' 'flights' is projective as the 'Head' i.e 'flights' has a path to 'morning' as well which lies between 'the' 'flights'. Similarly, arc between 'canceled' 'flights' is also projective as 'canceled' has a path to 'the'(canceled → flights → the) 'morning' (canceled → flights → morning) which lies between Head 'canceled' Dependent 'flights'

Slide 13:

Projective and Non-Projective Parse Tress

Projective parse tree: A parse tree with all its arcs projective. The above tree is projective.

Non-projective parse tree: A tree with at least one of the arcs as non-projective. Have a look at the below example

Slide 14:

Non-Projective Parse Tree Example

Here, the arc between 'flight' 'was' is non-projective as there exists no path between 'flight' 'morning'

Slide 15:

how to create a dependency parse tree-Shift Reduce Parsing (Arc standard):

We need some setup for this algorithm:

A Stack

Buffer of input words

Oracle function/parser set of dependency relations (like obj, iobj, etc tags that are used to represent arcs).

Slide 17:

Initially, the Stack has [Root], a dummy variable.

All the words of the sentence to be parsed are in the buffer.

A word is popped from buffer pushed in Stack.

As a word is pushed in Stack, the Oracle/parser takes the top two elements of the stack outputs/predict any of the below 3 transition with relationship type:

LEFTARC_FUNCTION:

RIGHTARC_FUNCTION

SHIFT:

Slide 18:

Hence, the actual prediction has two parts:

Determining Head Dependent using LEFTARC/RIGHTARC

Determining their relation type i.e functions like conj, iobj, etc.

Hence the actual predictions done by Oracle are something like this LEFTARC_IOBJ, RIGHTARC_CONJ, etc.

For simplicity purposes, I will be using just LEFTARC RIGHTARC notations in the below example.

We will stop as when the buffer is empty stack has only [Root] left.

Slide 19:

Some restrictions are also considered:

No LEFTARC, if we have [Root] as 2nd element of the stack buffer, is non-empty

At least 2 elements should be Stack for LEFTARC/RIGHTARC transition.

[Root] can never be a dependent (as it is a dummy variable)

Slide 20:

A Dependency Treebank is a text corpus in which each sentence has a corresponding dependency tree which is usually either extracted using Syntactic Treebanks(like PennTreebank) or manually marked by humans.

Slide 21:

Examples

consider the statement: 'book me the morning flight'. We will be creating a dependency parse tree for this sentence

Slide 22:

Step-0: As Stack has only one element [root], SHIFT is predicted(exception cases mentioned above) 'book' is pushed.

Step-1:[root, book] are popped out from Stack, Oracle predicts SHIFT hence they are put back, and 'me' is pushed in Stack.

Step-2:[book, me](top 2 elements of the Stack currently) are popped, Oracle predicts RIGTH-ARC, hence declaring 'book' as Head 'me' as dependent removing 'me' from Stack; pushing back 'book' adding relation:(Book→me)

At last, we can pop 'root' 'book' as the buffer is empty(which we avoided in Step-0 due to the exception mentioned). RIGTHARC is the relation detected with this the parse tree is obtained. Using the relations found, a directed graph can be easily established with 'book' as the root node.